NETWORK SECURITY · PRACTITIONER HANDBOOK

Network Security Assessment: A Practitioner's Handbook

One training path from the fundamentals of networking to expert command of every assessment domain - network, wireless, web, API, cloud and directory - the delivery architecture we run engagements from, and the complete technical reference for the host assessment toolkit. Written so a competent engineer can be brought to operating standard from it alone.

Classification	Standards base	Prerequisite	Scope of the toolkit
Training & reference	NIST SP 800-115 · PTES · OWASP · MITRE ATT&CK	Basic networking literacy	Assessment & identification - not exploitation

This handbook is deliberately complete. It assumes only that the reader knows what an IP address and a firewall are. Parts I-II build the frame; Part III is the delivery architecture *and the actual plan* (§3.8-3.9), so you can tell the generic field from what we specifically do; Parts IV-VI are the working methodology for each domain, with tools, example commands and the errors juniors make; Part VII is the toolkit's full specification; Part VIII is engagement management and reporting; Part IX is five worked scenarios; Part X is the desk reference; Part XI distils the field's hard-won lessons and Part XII walks a first engagement from the scope call to the report; Parts XIII to XVI are the deep reference on protocols, network attacks, databases and tools; Part XVII shows how real corporate and banking networks are built and how a tester gets access to them; Part XVIII is a set of short use-case scenarios, Part XIX gathers verified further-reading resources. A standing rule runs throughout: our toolkit performs assessment and identification (NIST SP 800-115 §3.4 + §4.3), never exploitation (§5) - the offensive techniques are taught so the reader understands the control being verified, not to run in a vulnerability-assessment scope without separate written authorisation.

Contents

Every section links to its page. Eleven parts, foundations through expert practice.

PART I · FOUNDATIONS OF NETWORKING

1.1 Why an assessor must understand the network first 5
1.2 The devices on a network 5
1.3 Addressing - MAC, IP, ports, NAT 7
1.4 How data moves - the layered model 9
1.5 The protocols an assessor meets 10
1.6 Segmentation and zones 12
1.7 IPv6, the DMZ and architecture 14

PART II · ASSESSMENT & STANDARDS

2.1 Scan → assessment → pentest → red team 16
2.2 The five domains, on one page 17
2.3 The standards landscape 18
2.4 The engagement lifecycle 20
2.5 Authorisation and scope 21
2.6 Test types & threat modeling 22

PART III · DELIVERY ARCHITECTURE

3.1 The testing environment 24
3.2 How external testing connects 25
3.3 Internal testing - VPN & jump-box 26
3.4 Cloud region and source IP 26
3.5 Scheduled scanning & scripts 26
3.6 Platform security & data handling 27
3.7 Advantages of this architecture 28
3.8 The delivery programme - the plan 28
3.9 Engagement & network types 30

PART IV · NETWORK PENETRATION TESTING

4.1 The two-pass methodology 32
4.2 External methodology, end to end 33
4.3 Internal & the AD attack primitives 35
4.4 Nmap in depth 37
4.5 The tool catalogue 38
4.6 Scoring - CVSS, EPSS, KEV, CWE 40
4.7 Segmentation testing - the matrix 41
4.8 Configuration & passive planes 42
4.9 Media and Apple environments 43
4.10 Service enumeration cheat-sheet 43
4.11 Pivoting, tunnelling & passwords 44

PART V · WIRELESS ASSESSMENT

5.1 801.11 fundamentals 46
5.2 The handshake, PMKID & encryption 47
5.3 The three layers of assessment 49
5.4 Wireless attacks and checks 49
5.5 The wireless toolkit & adapter 51
5.6 Standards and the remote limit 52
5.7 Beyond Wi-Fi - BLE & rogue devices 53

PART VI · WEB, API & CLOUD

6.1 Web application testing 54
6.2 API testing 55
6.3 Cloud assessment 56
6.4 Mobile, containers & the wider surface 58

PART VII · THE HOST ASSESSMENT TOOLKIT

7.1 Product shape - one workflow 60
7.2 Design philosophy 61
7.3 The collector & its transport 62
7.4 The 41 evidence sources 63

7.5 Evidence integrity & semantics 65
7.6 The 37 configuration rules 66
7.7 Patch determination 67
7.8 Active Directory evidence 69
7.9 Deployment, return & scale 70
7.10 The findings register & GRC 71
7.11 Capability and maturity 71
7.12 Production & licensing gates 73

PART VIII · ENGAGEMENT MANAGEMENT

8.1 Methodology & senior validation 75
8.2 Deliverables, scanning & retest 75
8.3 What we need from the client 76
8.4 Compliance mapping 76
8.5 Writing a finding 77
8.6 Pre-engagement checklist 78

PART IX · WORKED SCENARIOS

9.1 External web + API - SaaS portal 79
9.2 Internal network + AD - corporate HQ 79
9.3 Segmentation validation - PCI 80
9.4 Cloud posture - Azure / Entra 80
9.5 Wireless - a branch office 81

PART X · DESK REFERENCE

10.1 Ports and protocols 82
10.2 Tool matrix 82
10.3 Standards crosswalk 83
10.4 Glossary 84
10.5 The first hour - a command reference 88
10.6 Common findings and their fixes 89
10.7 Reading a CVSS vector at a glance 90
10.8 The engagement at a glance 90

PART XI · LESSONS FROM THE FIELD

11.1 Scoping without regret 92
11.2 The pre-engagement conversation 93
11.3 First, do no harm 93
11.4 Reading between the scan lines 94
11.5 Prioritising what actually matters 94
11.6 Writing findings that get fixed 94
11.7 Talking to clients and executives 95
11.8 The mistakes everyone makes once 95
11.9 How to keep learning - a skill ladder 96

PART XII · A FIRST ENGAGEMENT, WALKED THROUGH

12.1 The setup - the scope call 98
12.2 Getting in and looking around 98
12.3 Running the toolkit and reading results 98
12.4 One finding, start to finish 99
12.5 The debrief 99
12.6 When something looked wrong 99
12.7 What the report looked like 100

PART XIII · PROTOCOLS IN DEPTH

13.1 How to read any protocol 101
13.2 Layer 2 and 3: ARP, IP, ICMP 102
13.3 Transport: TCP and UDP 103
13.4 Naming and addressing: DNS and DHCP 104
13.5 The web trio: HTTP, HTTPS and TLS 105
13.6 The Windows enterprise stack 106
13.7 Management and remote access 107
13.8 File, mail and voice 108
13.9 Windows and Linux on the same wire 108
13.10 The protocol quick card 109

PART XIV · NETWORK ATTACKS AND VULNERABILITIES

14.1 How to think about attacks 111
14.2 Interception: poisoning, relay, MITM 112
14.3 Credential attacks 113
14.4 Active Directory and Kerberos attacks 113

14.5 Service exploitation 114
14.6 Layer 2 attacks 115
14.7 Denial of service and amplification 116
14.8 Web and application vulnerabilities 116
14.9 The attack-to-control map 117

PART XV · DATABASES ON THE NETWORK

15.1 Where a database belongs 118
15.2 The database quick reference 118
15.3 How databases are reached and abused 119
15.4 What we check 120

PART XVI · TOOLS IN DEPTH

16.1 How to think about tooling 121
16.2 Discovery and scanning: Nmap 121
16.3 Capture and analysis: Wireshark, tcpdump 122
16.4 Windows and Active Directory enumeration 123
16.5 Credential and hash tooling 123
16.6 Web and API tooling 124
16.7 Wireless tooling 124
16.8 Vulnerability scanners and reference databases 124
16.9 The tool selection map 125

PART XVII · REAL CLIENT NETWORKS AND ACCESS

17.1 The zone model of an enterprise network 126
17.2 Segmentation, micro-segmentation, zero-trust 127
17.3 Banking and regulated networks 127
17.4 When there is OT: the Purdue model 129
17.5 How a tester actually gets in 129
17.6 Inside versus outside; the reachability matrix 130
17.7 The engagement, on the ground 131

PART XVIII · USE CASES AND MINI-SCENARIOS

18.1 Discovery and recon 132
18.2 Internal and Active Directory 133
18.3 Segmentation and Layer 2 133
18.4 Configuration and services 134
18.5 Web, cloud and wireless 135
18.6 Judgement and professionalism 136

PART XIX · RESOURCES AND FURTHER READING

19.1 Standards and methodology 137
19.2 Protocol specifications 137
19.3 Vulnerability intelligence 138
19.4 Web and application 138
19.5 Tool homepages 139
19.6 Technique references and learning 139
19.7 A self-study path 140

PART I

Foundations of networking

You cannot assess what you cannot picture. Before any tool is useful, the assessor must see how the devices connect, how a packet travels, and which protocol carries which risk. This part builds that picture from the ground.

1.1

Why an assessor must understand the network first

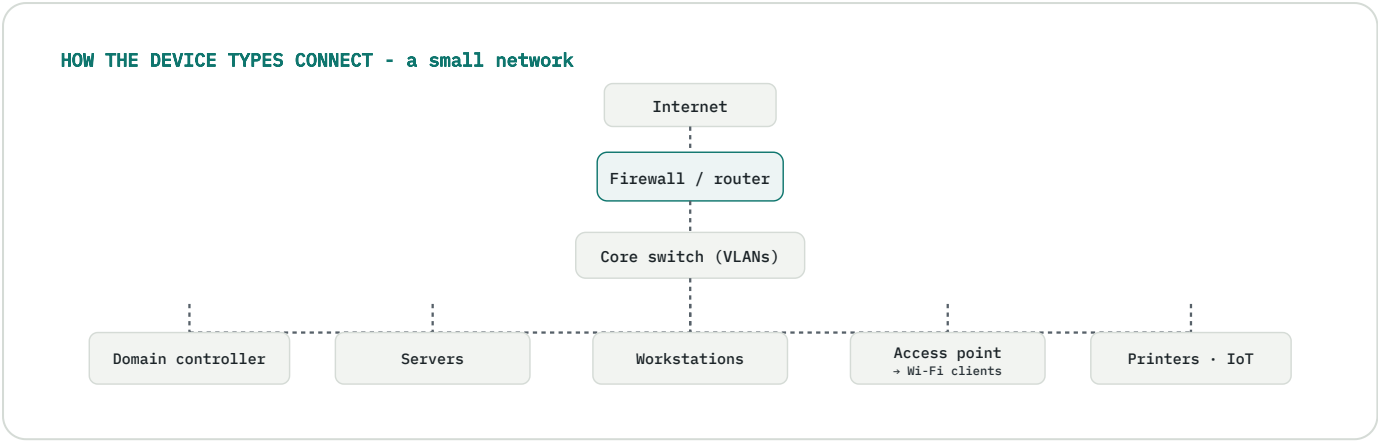
A scanner produces a list. Only an engineer who understands the network can say which items matter, which are false, and which point at a deeper structural weakness the scanner never named. Three questions run through every engagement, each a networking question before it is a security one: **Where does the traffic go?** (topology and routing) · **Who may talk to whom?** (segmentation and firewalls) · **What is listening, and does it prove its identity?** (services, authentication, encryption).

> FROM THE FIELD - PRACTITIONER'S NOTE
>
> Running the scanner is the easy 20% of the job - anyone can paste 200 results into a template. What a client actually pays for is the other 80%: knowing which three of those 200 matter **on this network**, which forty are false alarms, and which one dull "informational" line is really the crack an attacker walks through. That judgement comes only from understanding how the network is built - which is why this whole Part comes before any tool.

1.2

The devices on a network, and what each means to a tester

Every network is a small set of device types repeated. Learn these and you can read any topology diagram - each in plain terms, then in terms of why the assessor cares.



Every network is these same building blocks. The internet meets a firewall, a switch fans out to the VLANs, and the domain controller, servers, workstations, access point and network-only devices hang off it. Learn to spot each and you can read any topology.

DEVICE	WHAT IT DOES (PLAIN)	WHY THE ASSESSOR CARES
Switch	Connects devices *inside one local network*, forwarding each frame to its destination's port by MAC address (Layer 2). A managed switch can split ports into VLANs.	Where **segmentation** is enforced or lost. A flat switch lets one poisoned host reach everything on the segment; VLANs keep guest traffic off servers.
Router	Connects *different networks* and picks each packet's next hop toward its destination IP (Layer 3).	Routing decides what reaches what across subnets. A permissive route is often why an "isolated" zone is actually reachable.
Firewall	A gate between networks that allows/denies by rule (source, destination, port, sometimes application).	The ruleset *is* the segmentation policy written down. Reviewing it says what should be blocked; a live test says what is.
Access point (AP)	Bridges Wi-Fi (802.11) clients onto the wired network - the radio edge.	The wireless attack surface: encryption, SSIDs, and whether guest is truly separated from corporate.
Wireless controller	Centrally manages many APs - SSIDs, channels, VLAN mappings, security policy.	One controller misconfiguration propagates to every AP; its config is the fastest route to assess wireless segmentation.
Load balancer	Spreads connections across backend servers, presenting one address; often terminates TLS.	What the outside sees differs from the backend; weak TLS or exposed management is common here.
Proxy / gateway	Forwards traffic on a client's (forward) or server's (reverse) behalf.	Can hide or expose internal structure; a frequent place for auth and header-handling flaws.
VPN concentrator	Terminates encrypted remote-access tunnels, putting outside users on the inside network.	A public VPN is a prime external target: its patch level, its authentication (MFA?), its exposed management decide whether the perimeter holds.

DEVICE	WHAT IT DOES (PLAIN)	WHY THE ASSESSOR CARES
DHCP / DNS server	DHCP hands out IPs; DNS resolves names to addresses.	Trust infrastructure - poisoning name resolution is one of the most productive internal attacks.
Domain controller	The Active Directory server that authenticates users/computers and applies Group Policy.	The keys to the Windows kingdom - an assessment *target* and directory *source*, but never the machine you scan from.

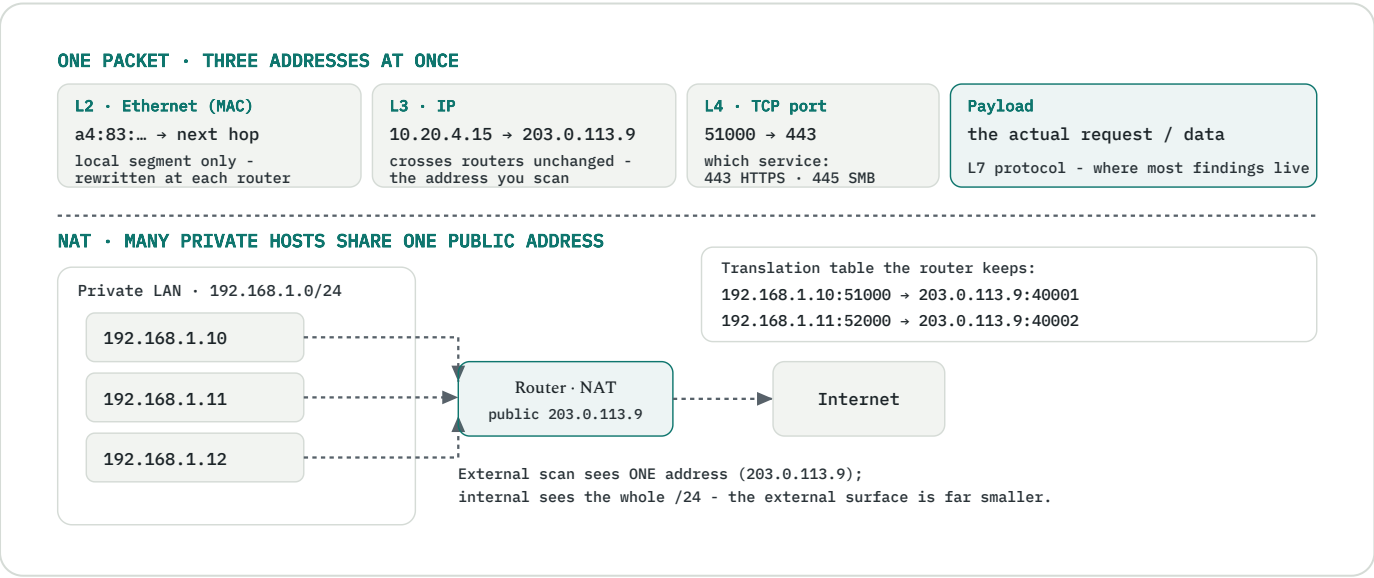
Beyond these sit **endpoints** (workstations/servers, assessed from the inside by the host toolkit) and **network-only assets** (printers, NAS, cameras, OT - no host agent, assessed gently over the network because active probing can disrupt them; tagged `NetworkOnly`).

1.3

How machines are addressed - MAC, IP, ports, NAT

- **MAC address** - a permanent hardware address (`a4:83:e7:2b:19:c0`), used *within one local segment* by the switch; it does not cross a router.
- **IP address** - a logical address that *does* cross routers (`10.20.4.15`); what discovery and scanning work with.

Private vs public, and CIDR. RFC 1918 ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) are private - reused inside every organisation, not routable on the internet. `10.20.4.0/24` is the 256 addresses `.0` - `.255`; scanning it means "this whole subnet." **Ports** (0-65535) say which service a packet is for (443 HTTPS, 445 SMB, 3389 RDP). **NAT** lets many private hosts share one public address, which is why the external view is far smaller than the internal one.



A packet carries three addresses at once - MAC (local hop only), IP (end-to-end, what you scan), and port (which service). NAT hides an entire private range behind one public address, which is why the external and internal views differ so sharply.

Reading a subnet - the one calculation to master

The `/24` in `10.20.4.0/24` is the *prefix length*: how many of the 32 bits are the fixed **network** part; the rest are **host** addresses. A smaller number after the slash = a bigger network. Two addresses in every subnet are reserved - the first (network) and the last (broadcast) - so a `/24` gives 254 usable hosts, not 256.

CIDR	MASK	ADDRESSES	USABLE HOSTS
`/24`	255.255.255.0	256	254
`/25`	255.255.255.128	128	126
`/26`	255.255.255.192	64	62
`/27`	255.255.255.224	32	30
`/28`	255.255.255.240	16	14
`/30`	255.255.255.252	4	2 (point-to-point links)

Worked read: `10.20.4.0/26` is 64 addresses (`.0` - `.63`), with `.0` the network, `.63` the broadcast, and `.1` - `.62` usable; the next subnet starts at `.64`. Reading this correctly lets you size a scan range and spot when a client's "one subnet" is really several.



Splitting a /24 into /26s. Each prefix step halves the block; the first and last address in each block are always reserved (network and broadcast).

MEMORY TIP - THE SUBNET LADDER

Each step up the prefix (`/24→/25→/26…`) **halves** the network: 256 → 128 → 64 → 32 → 16. Usable hosts are always **total minus 2** (network + broadcast).

IDENTITY TRAP

The same private address can mean a different machine at every site. **IP is never a stable identity.** Correlate a host by asset ID + hostname/FQDN + system UUID/MachineGuid + site - never merge two records because they share an IP.

1.4

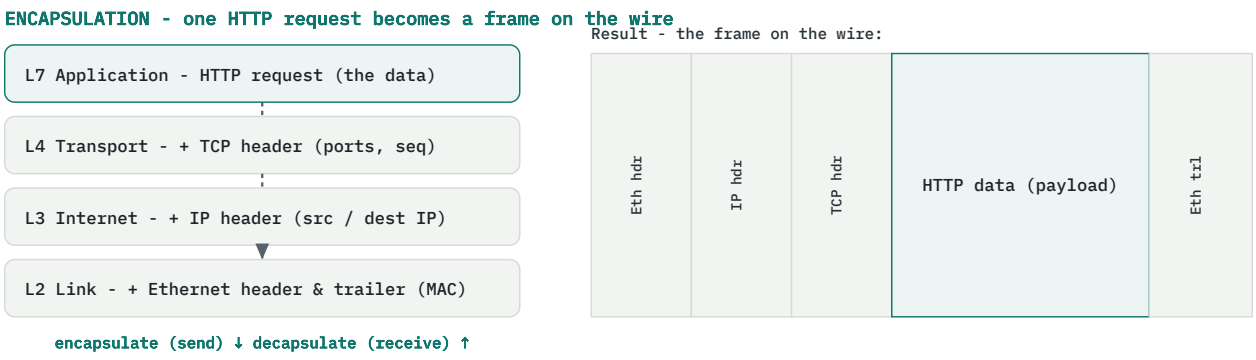
How data actually moves - the layered model, with one packet's journey

IN PLAIN WORDS

A network moves data in *layers*, like posting a parcel. You write a letter (the data), seal it in an envelope with an address (one layer), the post office drops that into a larger inter-city bag (another layer), and a van drives the bag down one street (another layer). Each layer only does its own job and trusts the layer below to carry it. Testers care because each layer can be attacked on its own - the street van, the city routing, or the letter itself.

Networking is layered so each layer solves one problem. Five ideas matter to a tester (a *frame* is the parcel on the local wire, a *packet* is it once it is being routed between networks):

LAYER	JOB	ADDRESS / UNIT	ASSESSOR'S INTEREST
L2 · Data-link	Move frames on one segment	MAC / frame	ARP spoofing, VLAN hopping, rogue devices
L3 · Network	Route between networks	IP / packet	Reachability, segmentation, discovery
L4 · Transport	Deliver to the right service	Port / TCP · UDP	Port scanning, the TCP handshake
L5-6 · Session/Presentation	Sessions & encryption	TLS	Weak TLS, certificate validation
L7 · Application	The service protocol	HTTP, SMB, DNS…	Where almost all findings live

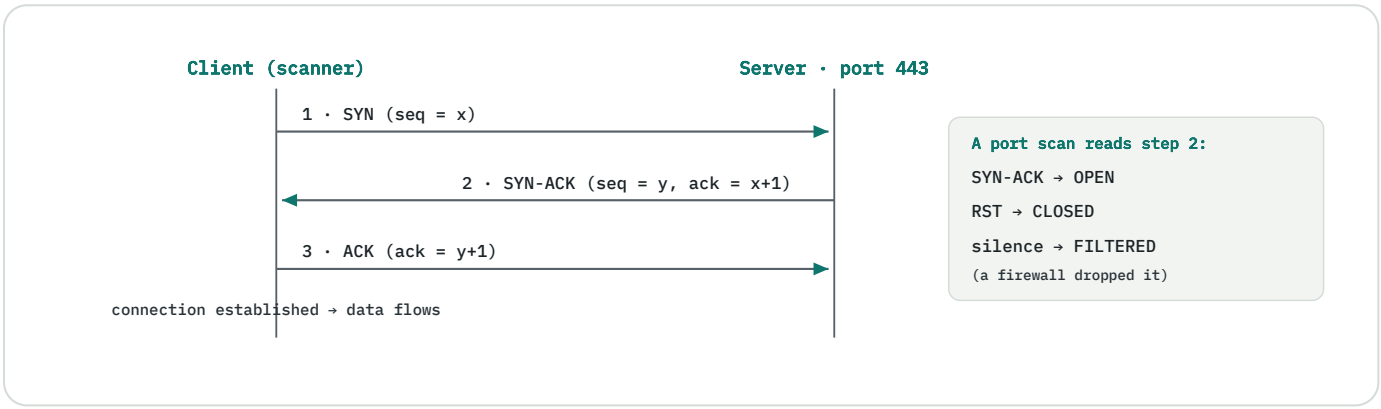


Going down the stack, each layer wraps the data in its own header; the receiver unwraps them going up. A scanner or firewall reads the outer IP and port headers, but the finding usually lives in the payload.

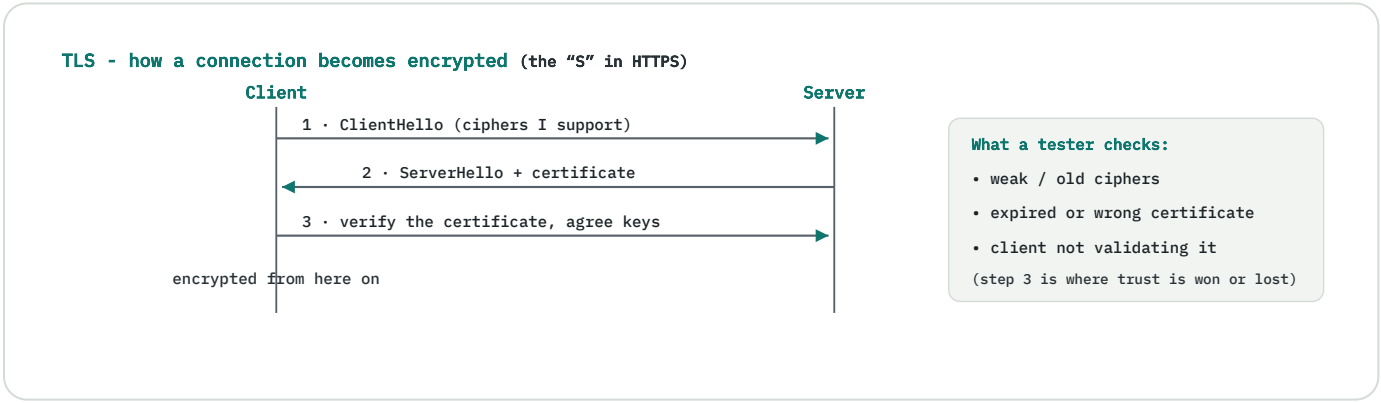
A worked example - you type a hostname and press enter

1. **DNS (L7)** turns `intranet.corp.local` into `10.20.4.15`.
2. **Routing (L3)** decides on-segment or across a router, and the next hop.
3. **ARP (L2)** finds the MAC of that next hop for physical delivery.
4. **TCP (L4)** opens the connection with the three-way handshake: `SYN` → `SYN-ACK` → `ACK`.
5. **TLS (L6)** negotiates encryption and checks the server certificate.
6. **HTTP (L7)** finally carries the request and response.

Every scanning technique is a deliberate manipulation of one of these steps. A port scan is just sending the `SYN` to many ports and watching which answer `SYN-ACK` (open), which answer `RST` (closed), and which stay silent (filtered by a firewall).



The TCP three-way handshake. Every port scan is a deliberate read of step 2 - a SYN-ACK means open, a RST means closed, and silence means a firewall filtered it.



TLS runs right after the TCP handshake: the two sides agree on encryption and the client checks the server's certificate. Most TLS findings are a weak cipher, a bad certificate, or a client that never checks it.

1.5

The protocols an assessor meets constantly

A **protocol** is just an agreed language two machines use to do one job - asking for a web page, logging in, looking up a name. Each one below has a *port* (the numbered door it listens on) and a security angle. The

words in that last column repeat all through this book, so learn them once here:

> **IN PLAIN WORDS - THE ATTACK VOCABULARY**
> *Spoofing* = pretending to be another device. *Poisoning* = feeding a computer a false answer to a "who is X?" question so it talks to the attacker instead. *Relay* = catching a login as it happens and passing it, unbroken, to a different server to sign in there (the password is never cracked). *Challenge-response* = a login where the server sends a puzzle and the client proves it knows the password by answering, without ever sending the password. *Community string* = SNMP's password - the factory default is literally the word `public`.

PROTOCOL	PORT	JOB	SECURITY ANGLE
ARP	-	Map IP → MAC on a segment	Spoofable; underlies LAN man-in-the-middle
ICMP	-	Diagnostics (ping, traceroute)	Host discovery; can leak topology
DNS	53	Names → addresses	Poisoning, zone-transfer leaks, tunnelling
DHCP	67/68	Hand out IP config	Rogue DHCP, DNS/gateway injection; DHCPv6 → mitm6
HTTP / HTTPS	80 / 443	Web traffic (plain / TLS)	The entire web/API surface
TLS	(wraps others)	Encryption + server identity	Weak ciphers, invalid certs, no validation
SMB	445	Windows file/printer sharing	SMBv1 (WannaCry); unsigned sessions → relay
RDP	3389	Remote graphical login	Exposure + weak auth = entry; needs NLA (Network Level Authentication)
LDAP	389 / 636	Query the directory (AD)	Anonymous binds, enumeration, cleartext on 389
Kerberos	88	Ticket-based domain auth	Kerberoasting, ticket attacks; needs FQDN + DNS
NTLM	(over SMB/HTTP)	Older challenge-response auth	Relay and cracking; legacy NTLMv1 weak
SNMP	161 (UDP)	Manage network devices	Default community `public` leaks config

PROTOCOL	PORT	JOB	SECURITY ANGLE
SSH	22	Encrypted remote shell	Weak/keyless auth, exposed management
WinRM	5985 / 5986	Windows remote management	The toolkit's transport. Kerberos or Negotiate over 5985 is message-encrypted; the risks are Basic authentication and the AllowUnencrypted setting, not the port number

Kerberos vs NTLM recur throughout Part VII. Kerberos is modern Windows login: prove identity once to the domain controller, receive time-limited *tickets*; it needs correct DNS and the machine's FQDN - an IP cannot be a Kerberos identity. NTLM is the older method still used with bare IPs and in workgroups; it is weaker and is the mechanism behind "relay" attacks, where an attacker who poisoned name resolution forwards a victim's authentication to another server.

MEMORY TIP - THE FOUR PORTS TO KNOW COLD

`445` SMB (relay & SMBv1) · `3389` RDP (exposure) · `88` Kerberos (needs a **name**, not an IP) · `161` SNMP (default community `public`). Learn these four before the rest.

FROM THE FIELD - PRACTITIONER'S NOTE

When you first land on a network, glance at these four ports before anything clever. `445` open with signing not required means you may be able to **relay** a login. `3389` reachable across a whole subnet is someone's forgotten remote desktop. `88` tells you a Windows domain exists (and a domain controller worth finding). `161` still answering to `public` hands you a device's entire configuration in one query. Those four tell you "is this network soft?" faster than any full scan.

1.6

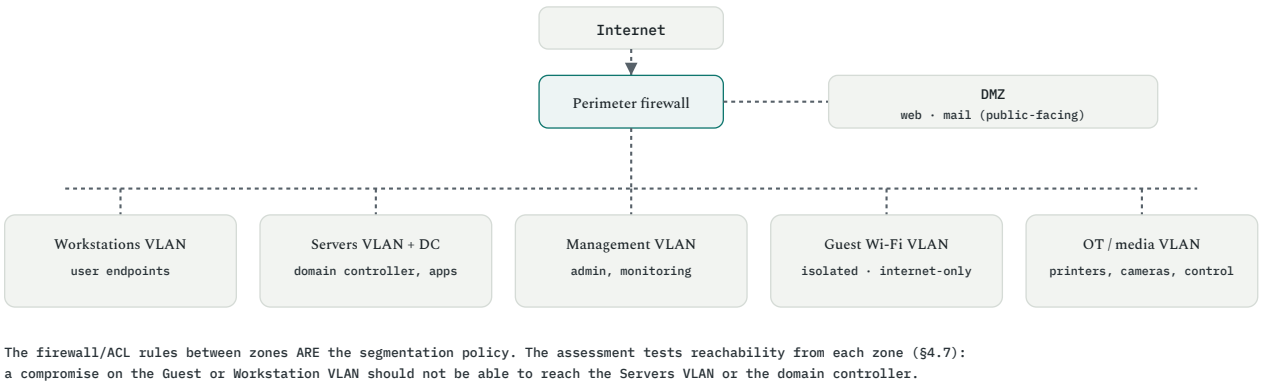
Segmentation and zones - the backbone of internal security

FROM THE FIELD - PRACTITIONER'S NOTE

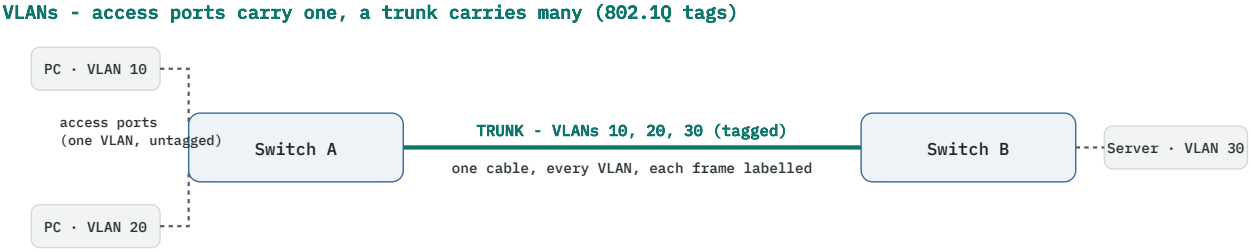
The fastest read of a new internal network is one question: **can an ordinary workstation reach a domain controller?** If it can reach services on the domain controller beyond the ones domain membership needs (Kerberos, DNS, LDAP, SMB for policy), such as RDP, WinRM or the management ports, the network is flat in the way that matters, and one phished laptop has a direct path to the crown jewels. Reaching the authentication ports alone is normal and proves nothing. That single reachability test (§4.7) tells you more about the client's real risk than a hundred-host vulnerability scan.

Segmentation is the most important structural control on an internal network. Divide it into **zones** (guest, corporate, servers, management, OT) and permit only the traffic the business needs. A well-segmented

network means a compromised guest laptop cannot reach a domain controller. Zones are built with **VLANs** on switches and enforced with **firewall/ACL rules**. Testing segmentation is not a scan - it is deliberate reachability tests from a representative position in each source zone (§4.7).



A segmented corporate network. Flatten this into one VLAN and a single compromise reaches everything - which is why segmentation is a first-class test.



An access port hands a device one VLAN with no label; a trunk carries many VLANs between switches, labelling (“tagging”) each frame. Misconfigure the trunk and an attacker can jump VLANs - the classic way segmentation breaks.

THE RULE THAT GOVERNS EVERY SEGMENTATION CLAIM

A scanner allowed broadly through the firewall sees a *more privileged* view than an ordinary user in that zone. You can never prove segmentation for a zone from a different, more-trusted vantage - and a failed connection is *Inconclusive* until a known-up control corroborates the cause.

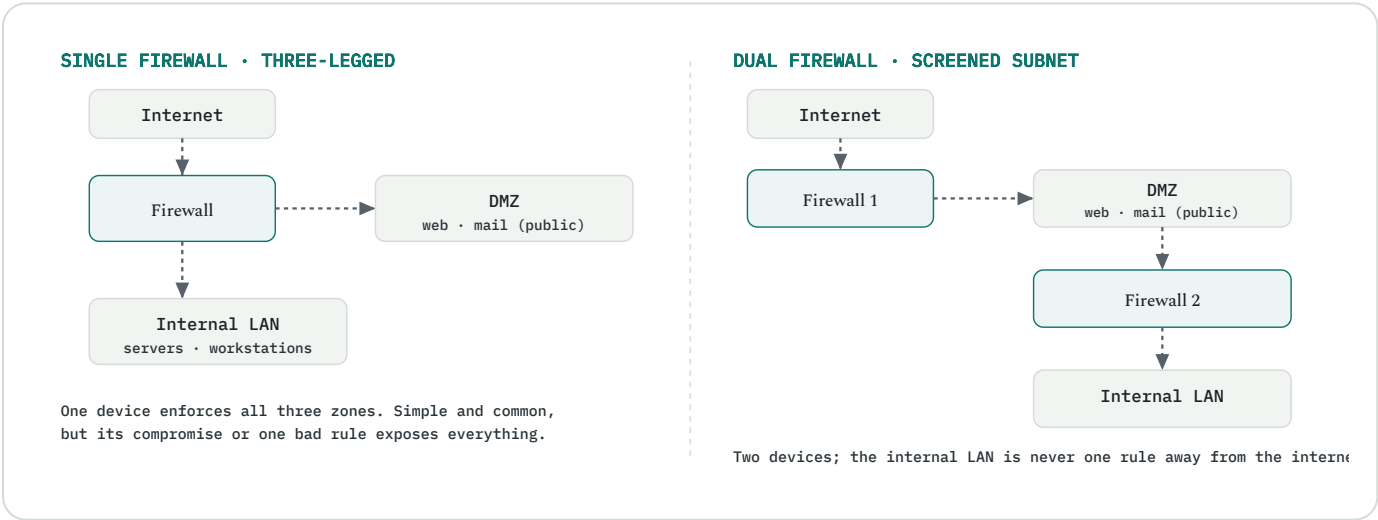
1.7

IPv6, the DMZ, and modern architecture

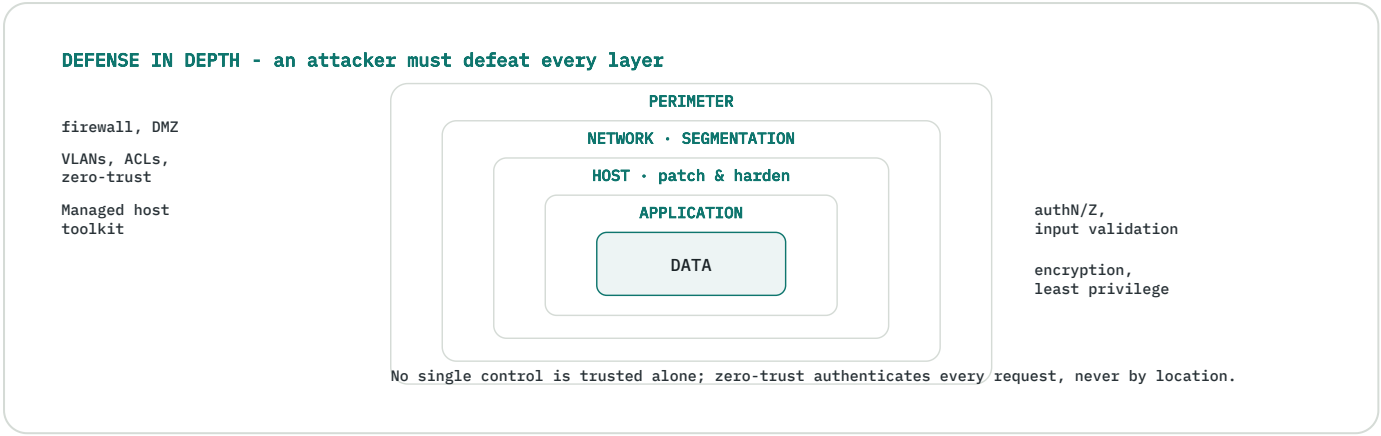
> **IN PLAIN WORDS**
>
> Most networks now speak two address languages at once - the old one (*IPv4*, like `10.20.4.15`) and the newer one (*IPv6*, like `fe80::1c2b`). Running both together is called *dual-stack*. The security catch: teams often watch and firewall only the old language and forget the new one, leaving a second, unguarded door into the same building. A *DMZ*, below, is just a fenced-off yard where you put public-facing servers so a break-in there cannot reach the offices behind.

IPv6 is the newer addressing scheme - 128-bit, written in hex (`fe80::1c2b:…`; *hex* just means counting with the digits 0-9 and a-f). Link-local addresses (ones only valid on the local wire) begin `fe80::`, and hosts auto-configure themselves via *SLAAC* (the IPv6 way of getting an address with no DHCP server). It matters to a tester for one blunt reason: most networks run *dual-stack* (IPv4 and IPv6 at once) but monitor and firewall only the IPv4 side, leaving IPv6 an unwatched parallel surface - the basis of the mitm6 attack (§4.3). Include or exclude IPv6 *explicitly* in the signed scope; never silently assume IPv4-only.

Three architecture words to hold: a **DMZ** (demilitarised zone) is a buffer segment for public-facing services (web, mail), isolated so a compromise there cannot reach the internal network. **Defense-in-depth** is layering controls so no single failure is fatal. **Zero-trust** is the modern stance that trust is never granted by network location - every request is authenticated and authorised regardless of origin - which raises the bar for segmentation and identity and is increasingly what a mature client is measured against.



Two ways to build a DMZ. The three-legged firewall is common; the dual-firewall screened subnet is stronger because a single failure at the edge does not reach the internal LAN.



Defense in depth: controls layered around the data so no single failure is fatal. The host assessment toolkit lives at the host layer - patch state and configuration hardening.

PART II

Assessment, standards & the engagement lifecycle

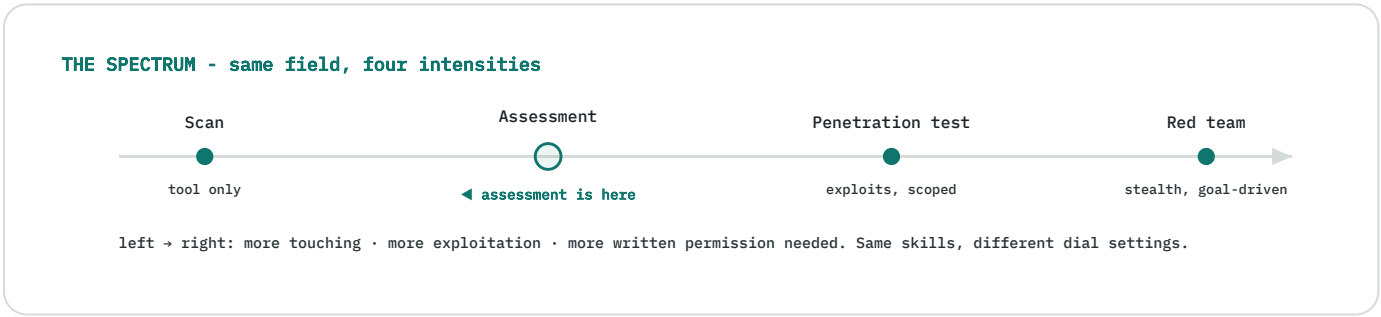
Testing is a spectrum of intensity governed by standards and by authorisation. Placing our work precisely on that spectrum keeps it legal, repeatable and defensible.

2.1

Scan, assessment, penetration test, red team - the spectrum

IN PLAIN WORDS

These four words are not the same job. A *scan* is a tool checking a list automatically. An *assessment* is that scan plus a human who confirms what is real and ranks it - this is what the toolkit does. A *penetration test* goes further and actually breaks in to prove the impact. A *red team* is a stealthy, goal-driven mock attack that also tests whether anyone notices. They differ in three things: how much you touch, whether you exploit, and how much written permission you need.



The four words are points on one dial. Turning it right means touching more, exploiting more, and needing more written permission. the toolkit sits at “assessment” - validated findings, no exploitation.

These four words are not synonyms. They differ in human involvement, in whether weaknesses are exploited, and in the authorisation each needs.

ACTIVITY	WHAT IT IS	EXPLOITS?	OUTPUT
Vulnerability scan	An automated tool checks hosts/services against a known-issue database.	No	A candidate list, with false positives
Vulnerability assessment	Scanning plus human scoping, validation and prioritisation - often *credentialed*. `HOST ASSESSMENT TOOLKIT`	No	A validated, prioritised, control-mapped finding set

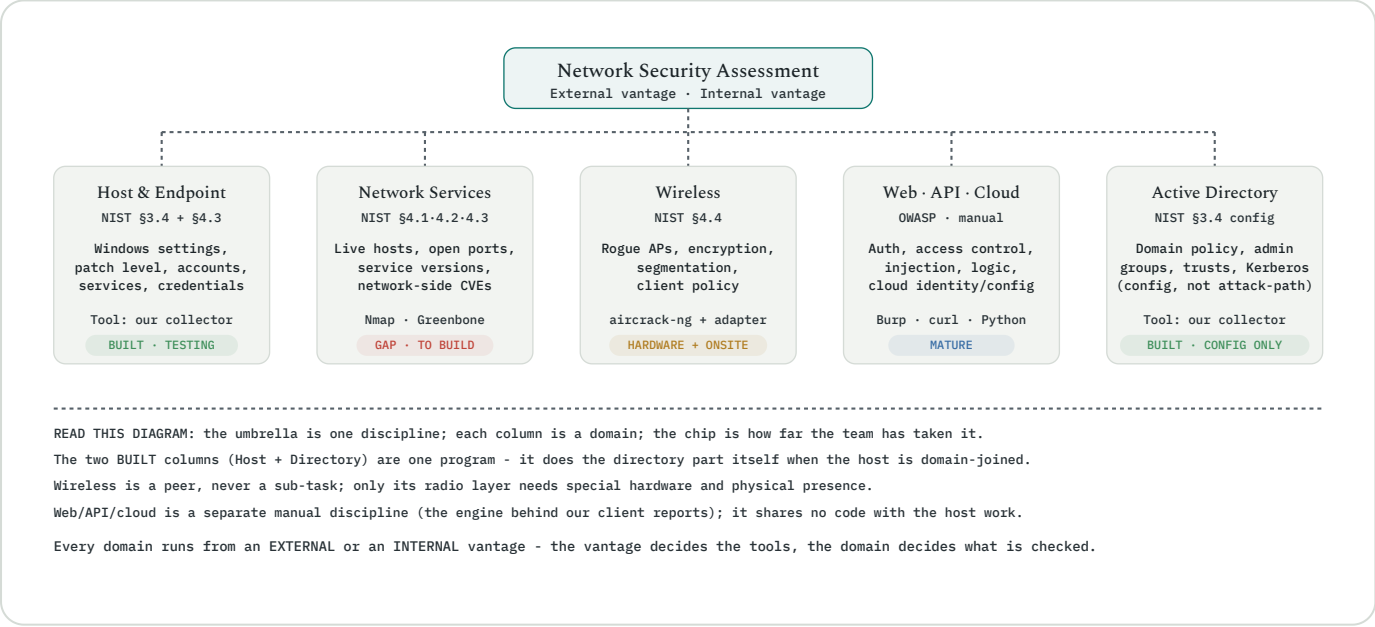
ACTIVITY	WHAT IT IS	EXPLOITS?	OUTPUT
Penetration test	An assessment that *proves* selected weaknesses by exploiting them under authorisation, showing impact.	Yes (scoped)	Proven findings with demonstrated impact
Red team	Objective-driven, stealthy adversary simulation testing detection and response.	Yes	An end-to-end attack narrative and defensive gaps

The host assessment toolkit performs a **credentialed vulnerability assessment** - the middle of the spectrum, where accuracy is highest and authorisation lightest, because nothing is exploited. Exploitation is separately authorised and never the toolkit's default.

2.2

The five domains of a network assessment, on one page

Everything the programme delivers - network, wireless, web/API, cloud and directory - is one discipline sliced three ways: by **vantage** (external / internal), by **technique family** (NIST §3 review / §4 analysis / §5 validation), and by **domain**. Wireless is its own domain (NIST §4.4), a peer of the others, not a sub-task.



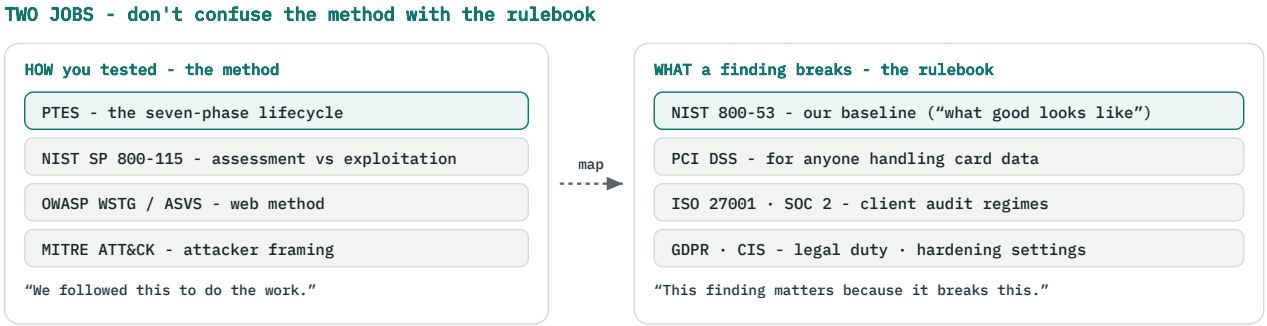
The five domains of a network assessment, with the toolkit's current coverage of each.

2.3

The standards landscape

IN PLAIN WORDS

There is a lot of alphabet soup here, but every “standard” does one of just two jobs. Some tell you *how to test* - a method to follow (PTES, NIST 800-115, the OWASP testing guides). Others tell you *what a finding breaks* - a rulebook a client must meet (PCI DSS for card data, ISO 27001, SOC 2, NIST 800-53). We follow a testing *method* to do the work, then map each finding to the client's *rulebook* so they see why it matters to them. Don't memorise the soup - just learn which of those two jobs each name does.



The whole standards landscape is just these two columns. On the left, the methods you follow to test; on the right, the rulebooks a client is measured against. You test with a method, then map each finding to their rulebook.

Professional testing follows published methodologies so results are comparable and defensible. Two axes must never be confused: **methodologies describe how you tested** (800-115, PTES, WSTG); **control catalogues describe what a finding violates** (800-53, ISO 27001, CIS).

STANDARD	WHAT IT IS	WHERE WE USE IT
NIST SP 800-115	The technical testing guide. Three families: §3 review (config/log/ruleset), §4 identification & analysis (discovery, port ID, vuln scanning §4.3, wireless §4.4), §5 validation (exploitation).	Primary methodology; the assessment-vs-exploitation line.
PTES	Seven phases: pre-engagement, intelligence gathering, threat modeling, vulnerability analysis, exploitation, post-exploitation, reporting.	The shape of an engagement.
OSSTMM	Metrics-driven operational testing across channels; introduces the RAV attack-surface metric.	Rigor reference for operational/segmentation testing.
OWASP WSTG / ASVS / API Top 10	WSTG = the web test-case methodology; ASVS = verification requirements (L1-L3); the Top 10s are awareness lists, *not* coverage standards.	Web/API method and checklist; Top 10 for reporting language.

STANDARD	WHAT IT IS	WHERE WE USE IT
MITRE ATT&CK	Knowledge base of real adversary tactics and techniques (T-IDs), incl. a Cloud matrix.	Framing findings by attacker use; detection gaps.
NIST SP 800-53 Rev 5	~1000 controls in families (AC, IA, SC, AU, CM, SI, RA…). A neutral catalogue - "what good looks like," not a pass/fail exam.	The **baseline** every toolkit rule maps to.
CIS Benchmarks / Controls v8	Prescriptive per-platform hardening; Controls = 18 Controls / 153 Safeguards (IG1-3). SecureSuite licence needed for commercial use of the content/CIS-CAT.	Hardening reference, licence permitting.
NIST CSF 2.0	Six functions: Govern (new), Identify, Protect, Detect, Respond, Recover.	Executive roll-up.
ISO/IEC 27001 (27002:2022)	ISMS certification; Annex A reorganised into 4 themes (Organizational, People, Physical, Technological), 93 controls.	Client-facing control mapping.
SOC 2 · PCI DSS · GDPR	SOC 2 = Trust Services Criteria attestation; PCI DSS v4.0.1 for cardholder data (Req 11 scans/pentest); GDPR Art. 32 for EU personal data.	Client regimes overlaid on the baseline.
NIST OSCAL	Machine-readable model for controls, baselines, assessment plans/results, POA&M.	The shape our evidence contract maps onto.

Where each applies - matching the standard to the situation

SITUATION	THE STANDARD THAT GOVERNS
How we tested (any engagement)	NIST SP 800-115 + PTES
A web or API application	OWASP WSTG / ASVS / API Top 10
Client handles cardholder data	PCI DSS (Req 11 scans/pentest, segmentation)
Client pursuing an ISMS certification	ISO/IEC 27001 (Annex A)
SaaS provider needing an attestation	SOC 2 (Trust Services Criteria)
Client or data subjects in the EU	GDPR Art. 32
US federal / government-aligned	NIST SP 800-53 (native) + CSF for executives
Hardening a specific platform	CIS Benchmark for that platform
Framing findings for defenders	MITRE ATT&CK technique IDs

THREE WORDS JUNIORS CONFUSE

800-53 is a control *catalogue* (the dictionary). **CIS Benchmarks** are concrete hardening *settings*. **PCI/HIPAA/ISO** are the *regimes a client is audited against* (the exam). We map once to 800-53, then overlay the client's regime - the baseline+overlay pattern of §8.4.

MEMORY TIP - TWO AXES

A **methodology** says HOW you tested (800-115 · PTES · WSTG); a **catalogue/regime** says WHAT a finding violates (800-53 · ISO · PCI · SOC 2). Never mix the two axes in one sentence.

2.4

The engagement lifecycle

FROM THE FIELD - PRACTITIONER'S NOTE

Agree the emergency-stop **before** you need it: a named person, a number that is actually answered, and a shared understanding of what triggers a stop. When something looks wrong at 2 a.m., that is not the moment to be working out who to call.

---

01 **Pre-engagement**

Scope, authorised CIDRs, no-scan assets, timing windows, rules of engagement, emergency-stop contact, evidence handling. Nothing starts until signed.

---

02 **Intelligence gathering**

Passive/open-source collection about the target without touching it aggressively.

---

03 **Discovery**

Find live hosts within authorised ranges. §4.1

---

04 **Enumeration**

Identify open ports, services, versions, OS. §4.2

05 **Vulnerability identification**

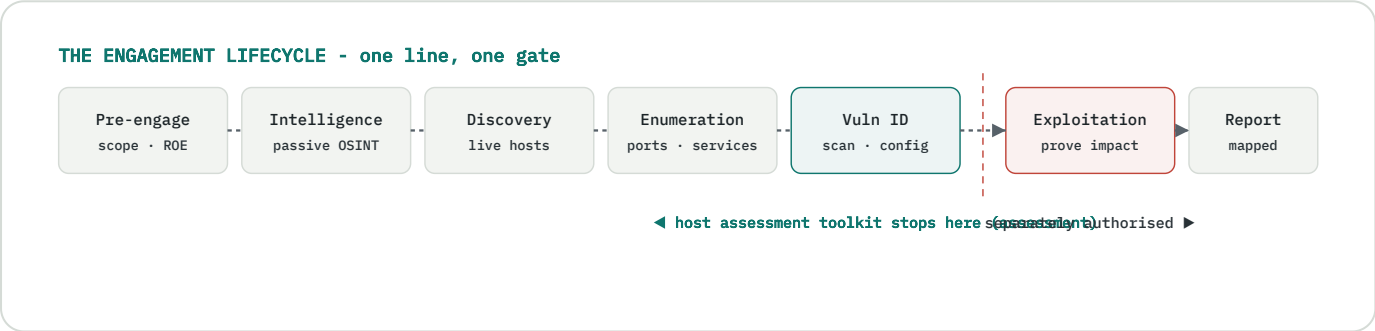
Match to known weaknesses by uncredentialed scan, credentialed scan, or config review. §3.4 + §4.3 ASSESSMENT STOPS HERE

06 **Exploitation / validation**

Prove selected weaknesses under authorisation, show impact. §5 SEPARATELY AUTHORISED

07 **Reporting**

Turn validated evidence into a control-mapped, defensible deliverable.



The lifecycle as a single pipeline. the toolkit runs through vulnerability identification (NIST SP 800-115 §3.4 + §4.3); exploitation is a separate, separately-authorised step.

2.5

Authorisation and scope - the spine that must never bend

FROM THE FIELD - PRACTITIONER'S NOTE

The moment that catches every junior: mid-test you discover the client owns more addresses than the paperwork lists. The instinct is "great, let me scan those too." **The answer is always no.** Finding extra systems is a valuable result to report - but touching them is unauthorised access, which is a crime, not initiative. Write down what you found, and ask for it in writing before a single packet goes near it. The signed scope is the one line you never cross to look good.

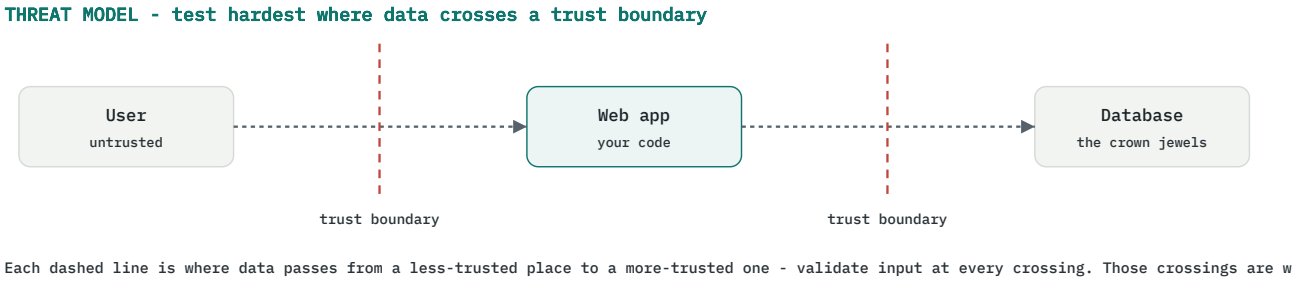
- **Exact architecture before production** - sites, authorised CIDRs, VLANs/zones, routing/firewalls, AD, management/security platforms, intended segmentation, critical/no-scan assets, evidence handling, emergency stop.
- **Stay inside authorised ranges.** Discovering more surface is a *finding to report*, never authorisation to scan it.

- **Unknown assets are not auto-escalated** - recorded `Discovered-Unmapped`, not subjected to deep testing.
- **Fragile assets get a safe profile** - printers, medical and OT devices get passive/config-only or an owner-approved window.
- **Credentials are scoped and vaulted** - temporary identities, validated on a pilot, never Domain Admin for convenience, stop on repeated auth failures (lockout).
- **Evidence is client-confidential** from collection - encrypted storage/transfer, explicit retention and destruction.

2.6

Test types, threat modeling, and the human layer

> **IN PLAIN WORDS**
> *Black-box* = we start knowing nothing, like an outside attacker. *White-box* = we get full information and logins up front. *Grey-box* = in between (say, one normal user's login) - and it usually finds the most real problems per hour, because we stop guessing and spend the time actually testing. *Threat modeling* is just asking, before we start, “who would attack this, and what are they after?” so the test aims at what matters instead of everything at once.



A threat model is this simple to start: draw where data flows, mark the lines where it crosses from less-trusted to more-trusted, and test those crossings hardest. That is where real attacks happen.

Before a technique is chosen, the *knowledge model* is agreed - it changes coverage and cost.

TYPE	WHAT THE TESTER IS GIVEN	TRADE-OFF
Black-box	Nothing beyond scope - models an external stranger.	Realistic but slow and shallow; misses what an attacker with time would eventually find.
Grey-box	Partial knowledge and/or low-privilege credentials.	The usual choice - best coverage per hour; models a foothold or insider. The two-pass method (§4.1) is grey-box.

TYPE	WHAT THE TESTER IS GIVEN	TRADE-OFF
White-box	Full knowledge - architecture, configs, sometimes source.	Deepest coverage; least like a real attacker, most like an audit. Config review (§4.8) is white-box.

Threat modeling (PTES phase 3) precedes the technical work: identify the assets that matter, the plausible threat agents, and the attack surface, then *prioritise testing by business impact* rather than testing everything equally. A crown-jewel database earns more effort than a marketing page.

THE HUMAN LAYER - SOCIAL ENGINEERING & PHISHING

People and process are a testable surface in their own right: **phishing** (email), vishing (voice) and pretexting probe whether staff and controls resist manipulation, and they measure the same email-security posture the external test checks technically (SPF, DKIM, DMARC), plus awareness and the reporting path. This is a §5 (validation) activity that touches real employees, so it is run *only* under explicit written scope with management sign-off - never bolted onto a technical engagement. It is named here because a complete programme includes it; it is not part of the host toolkit.

PART III

Delivery architecture - where we test from

Before the domain methodology, the operational setup: the environment we run engagements from, how external and internal work connect, how recurring scans are delivered, and how the platform itself is secured. One environment, built once, pointed at whatever the client authorises.

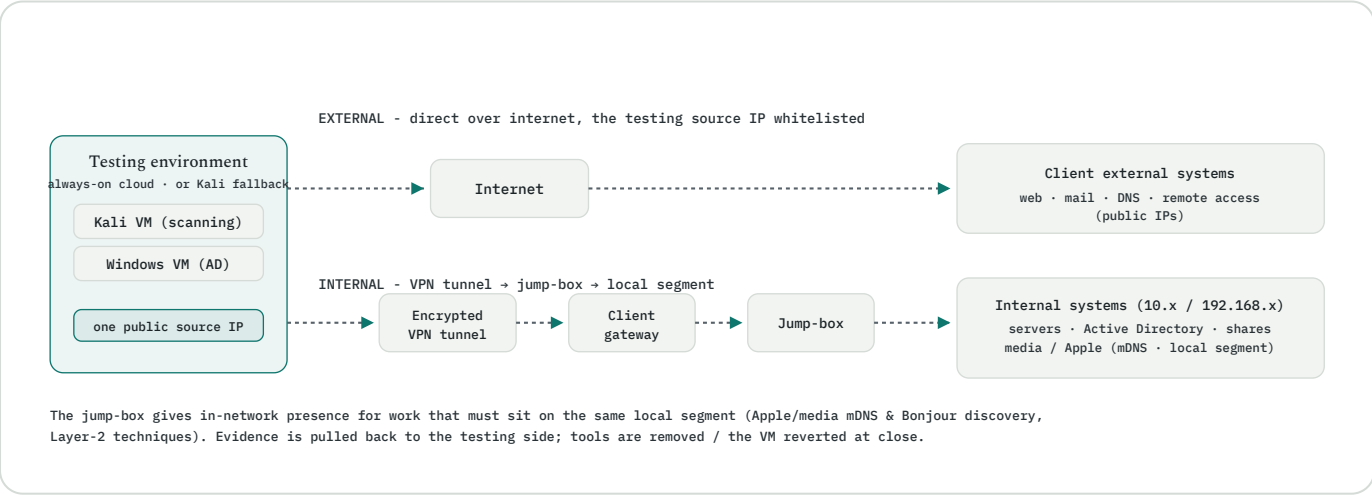
3.1

The testing environment - cloud preferred, testing machine as fallback

IN PLAIN WORDS

This section is about *where we run the tools from*. Two choices: a small cloud machine we spin up for the job (preferred - clean, disposable, and a fixed address the client can allow-list), or an testing laptop as a fallback. For internal work we reach the client's network through a *VPN* (an encrypted tunnel) into a *jump-box* - one agreed machine that sits inside their network and runs the local-only tests that can't travel down a tunnel.

We operate one testing environment, keep it ready, and point it at authorised targets. External work reaches the client over the internet; internal work connects through the client VPN to a jump-box inside their network.



Connection architecture - external traffic goes direct over the internet; internal traffic runs through an encrypted VPN tunnel to a jump-box inside the client network.

Cloud environment (preferred)

A small always-on cloud environment holds two VMs behind one public IP: a **Kali Linux VM** (primary scanning and tooling) and a **Windows VM** (Windows/AD work and any tool that runs best on Windows).

One shared public IP gives the client a single address to whitelist and us one clean source. Because it is always on, scheduled scans run on a timer with no manual start. It is built once and reused for every client (starting cost ~US$25/month).

Fallback from an testing machine

If the cloud is not approved or not yet ready, the same work runs from a testing Kali machine: external work scans from the machine; internal work connects through the client VPN, or logs into the client jump-box and runs the tools there. This works, but the machine must stay on for scheduled scans and the build-once benefit is lost.

SETUP	WHEN USED	TRADE-OFF
Cloud (preferred)	Default once approved	Always on, runs scheduled scans, one whitelistable IP, built once and reused
testing machine over VPN	Cloud not approved/ready	No monthly cost, but the machine must stay on and is not reusable
testing machine to jump-box	Internal work needing in-network presence	Best for heavy and local-segment scans; tools removed at close

---

3.2

How external testing connects

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> Send the client your exact source IP address before day one and get it allow-listed. Skip this and your first scan hits their firewall or web filter, you get blocked, and you burn half a day - and a little trust - working out why nothing is responding.

Our environment and the client's external systems are both public, so traffic goes straight over the internet from our address to theirs. We give the client our source IP first so their team allows it instead of blocking an unknown source. External targets are the internet-facing systems: web, mail, DNS and remote-access (VPN) gateways.

3.3

How internal testing connects - VPN and jump-box

> **IN PLAIN WORDS**
> To reach systems inside a client's private network we build a *VPN* (an encrypted tunnel) to a *jump-box* - one agreed machine sitting inside their network - and run from there. A few tests only work on the local wire and can't cross the tunnel, so those run on the jump-box itself; everything else runs through it. One agreed way in, fully logged, that the client can unplug at any time.

Internal systems use private addresses (10.x / 192.168.x) the internet cannot reach. The client provides **VPN** access, building an encrypted tunnel across the internet so both ends behave as one local network: our environment receives a private address inside the client range and routes to internal systems through the tunnel, which the client gateway decrypts on the far side. Routed scanning runs over the VPN.

Work that must sit on the *same local segment* - Apple and media discovery (mDNS / Bonjour) and certain Layer-2 techniques - runs from a **jump-box** inside the client network. A jump-box (bastion) is a single hardened host we route through, giving controlled, logged in-network presence; evidence is pulled back to our side and the report is produced there. On a client jump-box, all tools are removed at close or the VM is reverted. “VPN monitoring” means logging our own testing traffic so it can never be mistaken for a real attack and so we hold an audit trail.

3.4

Cloud region and source IP

- **US client** - host in a US or EU region so the source IP is readily accepted.
- **Data-residency requirement** - use a Pakistan region only when data must stay in-country.
- **Always** - avoid regions with weak data-protection rules. Same environment, region chosen per engagement.

3.5

Scheduled scanning and the wrapper scripts

> **IN PLAIN WORDS**
> Some checks (like a network vulnerability scan) run on a timer, not by hand. We set them to run only inside the agreed testing windows and never inside the client's *blackout windows* - the times the client says are unsafe - so a heavy scan never fires during a busy period or wakes the on-call team at 2 a.m. The wrapper scripts simply put standard tools on that timer and make every run produce the same tidy, comparable output.

A small script set wraps the tools into a repeatable run. Each reads one config file and writes structured output, so runs are consistent and re-runnable on a schedule:

SCRIPT	JOB
Discovery	Live host and port discovery; tags the asset inventory.
Scan orchestration	Runs the unauthenticated pass, then the authenticated pass.
Segmentation probing	Tests zone-to-zone reachability and builds the matrix.
Evidence collection	Saves and organises output per finding.
Report preparation	Assembles structured findings for the report.

Scheduled scanning runs on a timer (cron) on the always-on environment, scheduled inside the agreed testing windows and kept out of client blackout windows. Internal scheduled scans route through the VPN; external ones run directly. This is how recurring scanning is delivered without manual effort, adding a regular change summary between formal test windows.

3.6

Platform security and data handling

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> Hold your own kit to the standard you sell. A tester's laptop or cloud box - full of a client's evidence and credentials - is itself a juicy target. Encrypt it, use key-only access, and wipe it between engagements. Getting breached because **your** own machine was soft is the one incident a consultancy does not survive.

The environment holds client scan data and credentials, so it is locked down: **key-only access, host firewall on, encrypted storage, activity logging, and each client's data kept separate.** Credentials are removed at the end of every engagement. We hold our own platform to the standard we recommend to clients - the same discipline as the evidence contract in Part VII: raw evidence encrypted in transit and at rest, explicit retention and destruction.

> **ONE PLATFORM, EVERY SERVICE LINE**
> The same two machines also run web/API testing (Burp and API tooling) and cloud reviews (Azure/AWS/GCP) by installing the relevant toolchain - so this is one platform for every service line, not a single-purpose box. The Windows VM is especially useful for Azure/Entra work, which relies on PowerShell and Entra tooling.

3.7

Advantages of this architecture

Always-on, scheduled
Recurring scans run on a timer with no manual start, between formal windows.

One whitelistable source IP
The client allows a single clean address; we present one consistent origin.

Build once, reuse
The environment is built and hardened once, then pointed at every client.

One platform, all service lines
Network, web/API and cloud testing from the same two machines.

Region flexibility
Host per engagement for source-IP acceptance or data residency.

Clean separation
Per-client data isolation; credentials removed at close; jump-box tools reverted.

Evidence integrity (host plane)
Collect-vs-judge separation and sealed evidence make findings defensible (§7.2).

Low run cost
Open-source-first tooling and ~US$25/mo cloud keep the delivery cost near zero.

3.8

The delivery programme - the actual plan

> **IN PLAIN WORDS**
> This is the plan in real terms, not a generic textbook one. The short version: we read a machine's *real* state by logging in (never guessing from outside), a senior reviews the results before anything ships, and the evidence is sealed so no one can quietly change it. Those three habits are what make a finding hard to argue with.

Everything elsewhere in this handbook is the *generic field* - how network assessment is done anywhere. This section is the **specific plan**: what we actually deliver, and where our method differs from the textbook. Read the rest as knowledge; read this as the operating model.

What we deliver - five service lines, one platform

SERVICE LINE	WHAT WE DO	STATUS
External network	Test internet-facing systems (web, mail, DNS, remote access) from outside.	DELIVERED
Internal network + host	Assess the LAN over VPN/jump-box, plus credentialed host evidence via the toolkit.	HOST TOOLKIT IN VALIDATION
Web & API	Manual Burp-driven testing - the mature engine behind our client reports.	DELIVERED
Cloud	Read-only posture & identity review of Azure/AWS/GCP tenants.	DELIVERED
Wireless	Segmentation & client policy remotely; RF layer on-site with a monitor-mode adapter.	CONFIG NOW · RF ON-SITE

Our method vs the field - where we are different

The field runs scans and writes them up. We add three things that make the work defensible and cheap to run:

- **Evidence integrity.** On the host plane, collection and judgement are separate programs and every artifact is sealed at collection - so a finding traces to a control and a tamper-evident file, and the operator cannot influence the verdict (§8.2). Most scanners cannot claim this.
- **Off-host patch truth, for free.** Windows patch state is derived from the exact build+UBR against Microsoft's free MSRC feed and CISA KEV - *no Windows scanner purchase required* (§7.7).
- **One always-on platform, credentialed two-pass, senior-validated.** One environment serves every service line (§4); every engagement runs unauthenticated then authenticated (§4.1); every important finding is confirmed by a senior before it ships (§10.1).

> **REMEMBER - THE TOOLKIT EDGE IN ONE LINE**
>
> **Seal it, separate it, and read the truth from inside - then a person signs it off.**
> Evidence integrity + credentialed inside view + human validation is what a scanner alone never gives.

3.9

Engagement types & network types

FROM THE FIELD - PRACTITIONER'S NOTE

You can read a network's whole shape in the first hour, and it tells you the engagement. Flat and Windows-heavy? The workstation-to-domain-controller path will be the story. Well-segmented? The value moves to the reachability matrix and its exceptions. Cloud-hybrid? Half the risk lives in identity, not the wire. Spot the shape early and you know where to spend your days.

"Network penetration test" is not one thing. The network's *shape* and the engagement's *type* decide the vantage, the tools and the method. First, the three network shapes you will meet:



The three network shapes. The assessment's job on a flat network is often to show *why* it must be segmented.

And the engagement types, each with its vantage, tools and this method:

ENGAGEMENT	VANTAGE	PRIMARY TOOLS	THIS METHOD
External network	Internet	Nmap, Greenbone, testssl	From the cloud platform; source IP whitelisted; two-pass.
External web / API	Internet	Burp, curl, Python	Manual OWASP/WSTG; authz-first (BOLA/BFLA).
Internal network + host	LAN / jump-box	Nmap, netexec, the toolkit	Over VPN to jump-box; credentialed host evidence, sealed.
Segmentation	Each source zone	Nmap `-Pn`, hping3	Reachability matrix from representative positions (§4.7).
Wireless	On-site (RF) / network (seg.)	aircrack-ng, adapter	Survey → posture → validate → segmentation.
Cloud	Tenant API (read-only)	ScoutSuite, Prowler, ROADrecon	Benchmark config & identity; provider RoE.

ENGAGEMENT	VANTAGE	PRIMARY TOOLS	THIS METHOD
Retest	Same as original	Targeted re-run	Fixed only on direct retest evidence (§10.2).
Scheduled	Platform, on a timer	Orchestrated scan scripts	Cron between windows; change summary.

PART IV

Network penetration testing

The core domain: assessing hosts and services from outside and inside. The working methodology, the tools (open-source and commercial), example commands, and the errors that trip juniors.

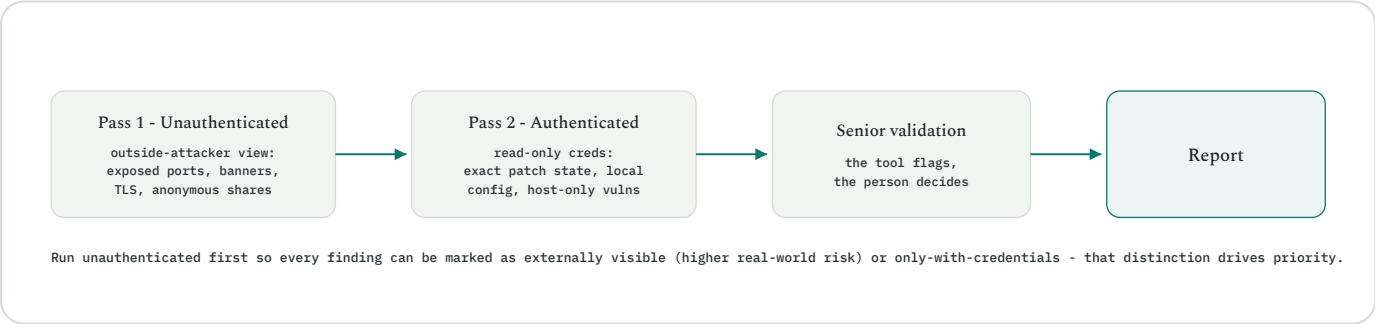
4.1

The two-pass methodology

FROM THE FIELD - PRACTITIONER'S NOTE

The credentialed pass's biggest trap is thinking it worked. A wrong password or a firewalled WinRM makes the tool quietly fall back to the weak outside view and report almost nothing - which reads like “all clear.” Always confirm authentication actually **succeeded** on each host before you trust a quiet result; an empty report from a failed login is the most dangerous kind of false comfort.

Every network engagement runs in two passes. The first models the outside attacker with no credentials; the second logs in with read-only credentials and reads the truth. The authenticated pass finds *materially more* real issues - commonly several times as many - so credentials are requested early.



The two-pass method - authenticated scanning reads exact patch and config state instead of inferring from a network banner.

THE CREDENTIALED-SCAN TRAP

A large share of “authenticated” scans silently fail authentication (wrong account type, no admin rights, RemoteRegistry/WMI disabled, 445/135 blocked) and quietly downgrade to unauthenticated. **Verify authentication success per host** (e.g. Nessus “Credentialed checks: yes”). A credentialed scan that did not authenticate is just a loud unauthenticated one.

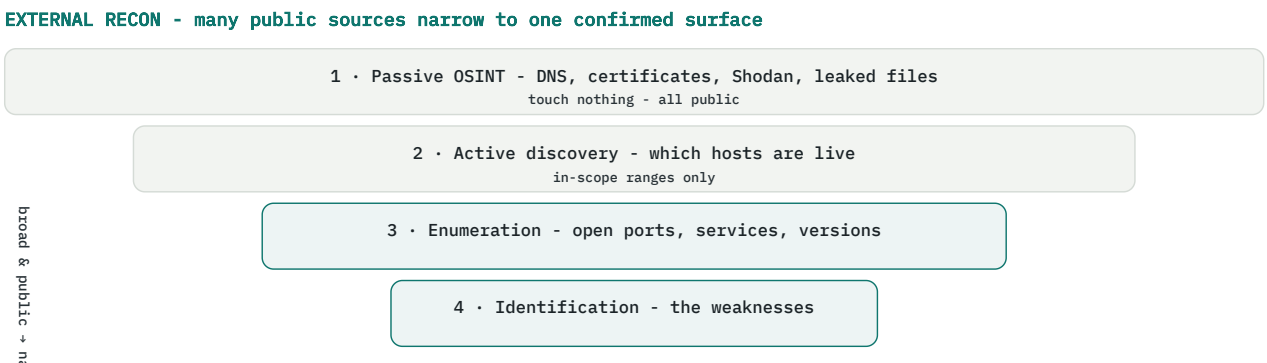
4.2

External methodology, end to end

IN PLAIN WORDS

“External” means we act like a stranger on the internet with no inside access. The job has two halves. First, *passive OSINT* (open-source intelligence) - gathering what is *already public* about the target without touching it: domain names, certificates, leaked files. It is like learning everything about a building by walking past it. Then *active discovery* - carefully knocking only on the doors the signed scope allows, to see which are open and what is behind them. Most of the odd tool names below are just different ways of finding the company's public footprint.

The external assessment models a stranger on the internet. The goal is to enumerate the exposed surface completely and identify its weaknesses.



External reconnaissance is a funnel: start wide with everything already public, then narrow - always staying inside the signed scope - down to the confirmed weaknesses worth reporting.

PHASE A - PASSIVE OSINT (NO AGGRESSIVE CONTACT)

- **DNS footprint** - `whois`, `dig`, DNSDumpster, `dnsrecon`. Records: A/AAAA, MX, NS, TXT (SPF/DMARC/DKIM leak infra), CNAME (SaaS → subdomain-takeover candidates).
- **Subdomain enumeration** - passive (`amass -passive`, `subfinder`, `crt.sh`, SecurityTrails) and active (`amass -active`, `puredns`, `ffuf` vhost).

- **Certificate Transparency** - `crt.sh` , `certspotter` reveal subdomains issued certs, including staging/dev names accidentally certed.
- **Internet-scan datasets** - Shodan / Censys by org, ASN, cert or favicon hash; reveals exposed services and shadow IT.
- **ASN / netblock discovery** - `whois` → org → `asnmap` / `bgp.he.net` maps the full owned surface.
- **Leak & content OSINT** - `theHarvester` (username format), `waybackurls` / `gau` (old endpoints), `trufflehog` / `gitleaks` (exposed secrets, `.git` , buckets).

> TWO OSINT ERRORS
> (1) A passive/dataset hit is a *historical* name - resolve and confirm it is live and in-scope before reporting; many CT entries are dead or internal. (2) Discovering more netblocks is a finding to report, not authorisation to scan them - only test signed ROE ranges.

PHASE B-D - DISCOVERY, ENUMERATION, IDENTIFICATION

Bounded to authorised ranges: `nmap -sn` / `masscan` / `fping` for liveness (external hosts often drop ICMP, so “no ping” ≠ down), then `masscan` for breadth and `nmap -sV` for accuracy, then TLS (`testssl.sh`), tech-detection (`httpx` , `whatweb`), and any web app handed to Part VI. Common external findings: weak/expired TLS, exposed management interfaces (RDP, SSH, DB, Jenkins, iLO/iDRAC, Citrix/Fortinet/Pulse VPNs), missing patches on internet-facing services, subdomain takeover, information disclosure (`.git` / `.env` /backups, stack traces), email-security gaps (SPF/DMARC), open SNMP/resolvers.

> VERSION-INFERRED ≠ CONFIRMED
> Vendors backport security fixes without changing the banner version (common on RHEL/Debian). Never assert a CVE from a banner alone - label findings “version-inferred” vs “validated.”

4.3

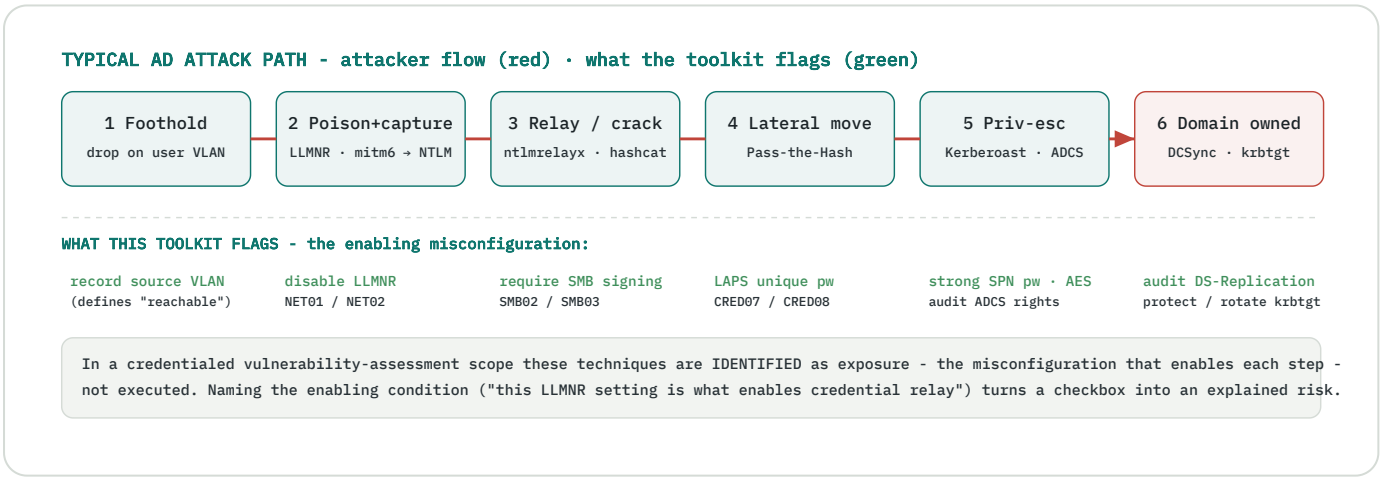
Internal methodology and the Active Directory attack primitives

IN PLAIN WORDS

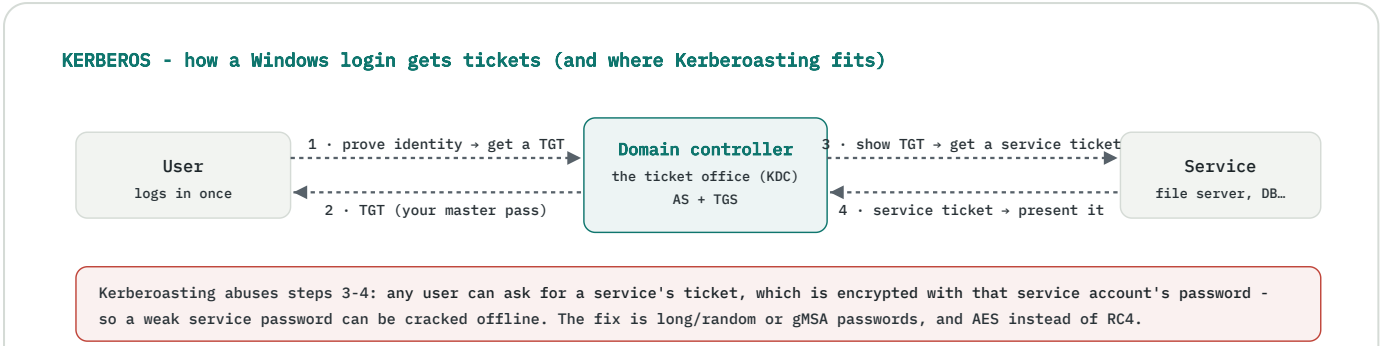
"Internal" means we test from *inside* the network, the way an attacker would after taking over one laptop. (*Active Directory*, or AD, is the Windows system that stores all the company's user accounts and computers and checks logins - the keys to the kingdom.) Almost every scary name below is a variation of one idea: **Windows machines constantly prove who they are to each other, and if that proof is done sloppily, an attacker can catch it, reuse it, or crack it.** A few in one line each - *Poisoning*: shout the wrong answer to a lost computer so it sends its login to you. *Relay*: pass that caught login straight to another server. *Kerberoasting*: ask the system for an encrypted ticket and crack its weak password offline. *Pass-the-Hash*: log in using the stored form of a password without knowing the password. *DCSync*: trick a domain controller into handing over everyone's password hashes. The toolkit never runs these - it finds the sloppy settings that let them happen (the diagram below maps the whole path).

The internal assessment models an attacker already inside, or assesses the estate from a management position. Positioning matters - a drop on a flat user VLAN sees far more than one in a segmented DMZ; record the source VLAN, it defines what "reachable" means. On-subnet discovery uses ARP (`arp-scan`, `nmap -sn` which auto-uses ARP on-subnet), which is authoritative and cannot be firewalled at Layer 2.

An assessor must understand the classic techniques even where the engagement only *identifies* the exposure - because the host toolkit's checks exist to find the conditions that enable them. Format: what it is · what enables it · what the defender checks.



The classic path from an inside foothold to domain dominance. The toolkit does not run these attacks - it finds and names the specific misconfiguration that would enable each step.



Kerberos is a ticket system: prove who you are once to get a master ticket (TGT), swap that for a per-service ticket, then use it. Kerberoasting is simply asking for a service's ticket and cracking its password offline.

TECHNIQUE	WHAT IT IS / ENABLES / DEFENDER CHECK
LLMNR/NBT-NS/mDNS poisoning (Responder)	Answer broadcast name queries when DNS fails, impersonating the host to capture NTLM challenge-response. *Enables:* LLMNR/NBT-NS/mDNS on by default. *Check:* disable via GPO (rules `NET01/NET02`), kill WPAD.
NTLM / SMB relay (ntlmrelayx)	Relay captured auth live to another service (SMB/LDAP/ADCS) - no cracking. *Enables:* SMB signing not *required*; LDAP signing/channel-binding off; coercion (PetitPotam). *Check:* require SMB signing (`SMB02/03`), LDAP signing + channel binding (EPA, Extended Protection for Authentication). *Note:* signing merely *enabled* ≠ *required*; you cannot relay back to the originating host (post-MS08-068).
Kerberoasting	Any domain user requests a service ticket for an SPN account, encrypted with that account's hash → crack offline. *Enables:* service accounts with weak passwords, RC4 tickets. *Check:* long/random or gMSA passwords, AES, monitor 4769.
AS-REP roasting	For accounts with pre-auth disabled, request an AS-REP and crack it offline. *Enables:* `DONT_REQUIRE_PREAUTH`. *Check:* no accounts without pre-auth; strong passwords.
Pass-the-Hash / Pass-the-Ticket	Authenticate with an NT hash (PtH) or inject a stolen/forged ticket - golden (krbtgt) / silver (service) (PtT). *Enables:* NTLM allowed, shared local-admin hashes, krbtgt compromise. *Check:* LAPS, disable NTLM, Credential Guard, Protected Users, protect/rotate krbtgt.
DCSync (concept)	Abuse directory replication to ask a DC for password hashes (incl. krbtgt). *Enables:* excess "Replicating Directory Changes" rights. *Check:* audit DS-Replication rights (BloodHound `DCSync` edge), monitor 4662.
Shared local-admin reuse	One identical local-admin password/hash across hosts → one compromise = whole fleet. *Enables:* no LAPS, gold-image reuse. *Check:* LAPS with unique per-host passwords (`CRED07/08`).
mitm6 / IPv6 DNS takeover	Answer Windows' default DHCPv6 solicits, become its IPv6 DNS, then relay to LDAP. *Enables:* IPv6 on but unmanaged. *Check:* RA Guard / DHCPv6 Guard at the switch - *not* wholesale-disabling IPv6, which breaks Windows.

Naming these to a client - “this LLMNR setting is what enables credential relay on your network” - is the difference between reporting a checkbox and explaining a risk. Enumeration tooling: `netexec` / `enum4linux-ng` (shares, sessions, users, policy), `certipy` (ADCS ESC1-ESC16), BloodHound to map who can reach tier-0.

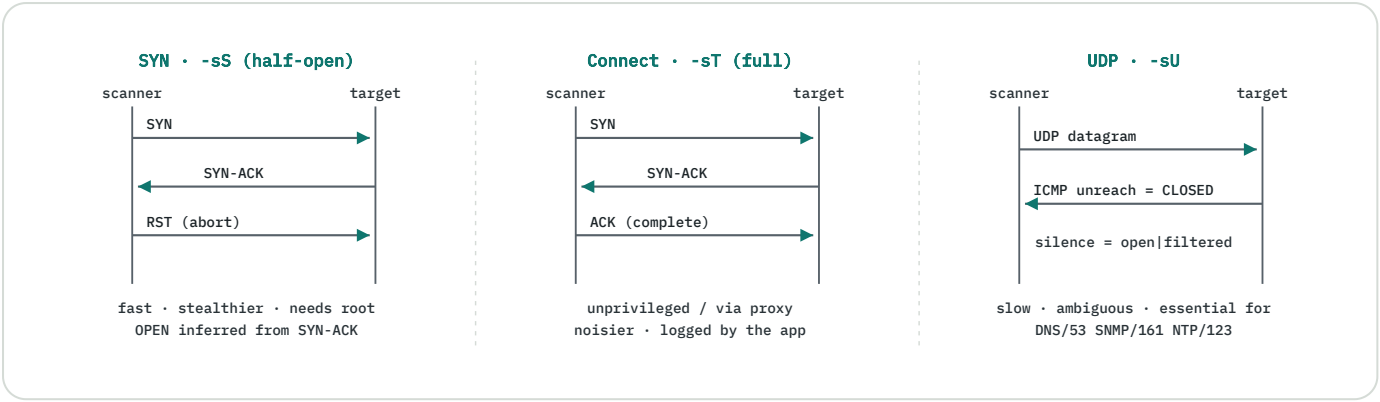
> SCOPE BOUNDARY
>
> Relay, roasting and pass-the-hash cross into exploitation (§5). In a credentialed vulnerability-assessment scope they are *identified as exposure* (the misconfiguration that enables them), not executed, without separate written authorisation.

4.4

Nmap in depth

> **FROM THE FIELD - PRACTITIONER'S NOTE**
>
> Three habits separate a real scan from a miss. Always add `-p-` or you will miss a service parked on 8443. Always save with `-oA` - no saved output means no re-checkable evidence later. And never skip UDP entirely: DNS, SNMP and NTP live there and often hand you the best findings of the day.

SCAN	WHAT IT DOES / WHEN
`-sS`	SYN half-open - default with root; fast, never completes the handshake. Needs raw-packet privilege.
`-sT`	Full TCP connect - when unprivileged or over a proxy; noisier, app-logged.
`-sU`	UDP - slow, ambiguous (`open|filtered`), but essential for DNS/53, SNMP/161, NTP/123, IKE/500. Skipping UDP loses high-value findings.
`-sn`	Host discovery only (ARP on-subnet, ICMP+TCP off-subnet).
`-sV` / `-O`	Version detection / OS fingerprint (needs an open + a closed port for reliability).
`-sC` / `-A`	Default NSE scripts / aggressive (`-sV -O -sC --traceroute`) - loud.



The three core scan types. A SYN scan reads the handshake (§1.4) but sends RST instead of completing it; a connect scan finishes it and is app-logged; UDP is slow and ambiguous but essential for DNS, SNMP and NTP.

Timing `-T0` - `-T5` (paranoid→insane); `-T3` default, `-T4` typical internal. `-T5` externally drops probes and causes false negatives. **NSE categories:** use `safe` / `default` / `discovery` / `version` in a VA scope; **avoid** `brute` , `dos` , `exploit` , `intrusive` unless the ROE permits - `dos` can crash services.

```
# 1. Fast internal host discovery (ARP on-subnet), no port scan
nmap -sn 10.10.10.0/24
# 2. Thorough TCP: all ports, version + default scripts, save all formats
nmap -sS -p- -sV -sC -T4 -oA scans/internal_full 10.10.10.5
#   -p- all 65535 ports   -oA writes .nmap/.gnmap/.xml for re-parsing
# 3. Targeted UDP on high-value services (never all UDP - too slow)
nmap -sU -p 53,69,123,161,500 --open -T4 10.10.10.5
# 4. SMB-signing posture (relay exposure) via NSE
nmap -p 445 --script smb2-security-mode 10.10.10.0/24
```

Junior errors: forgetting `-p-` and missing a service on 8443/9000; not saving `-oA` (no re-parseable evidence); trusting an OS guess made through a filter.

4.5

The tool catalogue - open-source vs commercial

FROM THE FIELD - PRACTITIONER'S NOTE

Tools change every year; the method does not. Learn what each tool is **for** - discovery, enumeration, a specific check - rather than memorising its flags, and you can pick up next year's replacement in an afternoon. And in a client report, name only the real, analyst-driven tools you actually used (§8.5) - never a scanner that ran unattended.

TOOL	CAPABILITY	LICENSE / COST	THIS TOOLKIT
Nmap	Discovery, port/service/OS ID, NSE scripts	Open source	USE
masscan / arp-scan / fping	Internet-scale port sweep / L2 discovery / fast ping	Open source	USE
Greenbone / OpenVAS (GVM)	Full network vuln scan, auth + unauth; Community Feed (no completeness warranty)	Open source (enterprise feed paid)	USE
Nessus Professional	Industry-standard credentialed scanner, compliance	Commercial ~US$3,990-4,790/yr	REUSE CLIENT'S
Tenable.io / Qualys VMDR / Rapid7 InsightVM	SaaS vuln management, asset-priced, dashboards/agents	Commercial (quote)	REUSE CLIENT'S
Microsoft Defender VM	Agent-native host/patch plane on Microsoft estates	Commercial add-on	REUSE CLIENT'S
netexec (nxc) / enum4linux-ng	AD/SMB/LDAP/WinRM enum: signing, shares, users, policy, spray	Open source	ENUM ONLY IN VA
impacket	Protocol toolkit (secretsdump, GetUserSPNs, ntlmrelayx...)	Open source	VALIDATION ONLY
Responder	Poison LLMNR/NBT-NS to capture auth (analyze mode is passive)	Open source	VALIDATION ONLY
BloodHound / SharpHound · certipy · kerbrute	AD attack-path mapping · ADCS abuse · Kerberos user-enum/spray (no lockout on enum)	Open source	OUT OF SCOPE*
testssl.sh / nuclei / nikto	TLS audit / template vuln matching / web-server misconfig	Open source	USE (TESTSSL); SCANNERS INTERNAL
Metasploit · Hydra · Hashcat / John	Exploitation · online brute · offline cracking	Open source (MSF Pro commercial)	NOT USED BY DEFAULT
Batfish · Zeek	Model device configs (static) · parse traffic to structured logs (passive)	Open source	USE (§4.8)

* Directory attack-path analysis (BloodHound) is a future workstream; the toolkit collects directory *configuration* only (§7.8). Junior errors: calling `netexec` enumeration "exploitation" (its module runner is, its enum is not); Hydra brute in AD triggers *lockouts* - prefer `kerbrute` enum + policy-aware spray.

CLIENT-FACING TOOL DISCIPLINE - ABSOLUTE

Whatever is used internally, the client report names only real analyst-driven tools: **Burp Suite Professional, curl, Python (custom scripts)** (plus `testssl.sh` / `ffuf` where genuinely used, and the maintained scanner where one was). Never name an unattended scanner (ZAP, nuclei, nikto, Nessus) or an anonymised stand-in in a deliverable, even when used internally.

4.6

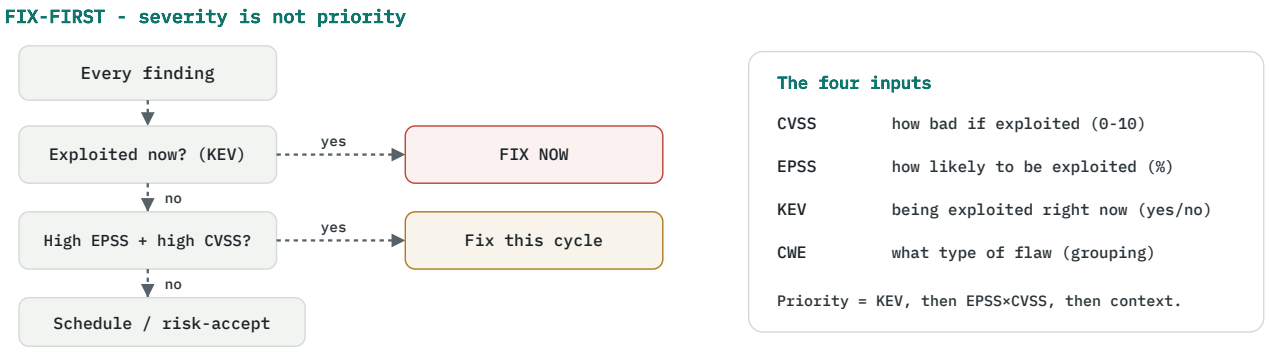
Scoring and prioritisation - CVSS, EPSS, KEV, CWE

IN PLAIN WORDS

Four short names decide what to fix first. *CVSS* = how bad is this if exploited? (a 0-10 severity score). *EPSS* = how likely is it to actually be exploited? (a probability, from real-world data). *KEV* = is it being exploited in the wild *right now*? (a yes/no list published by the US government). *CWE* = what *type* of flaw is it? (a category, useful for grouping). The key lesson: severity is not the same as priority. A medium-severity bug that is on the KEV list with high EPSS gets fixed before a high-severity one nobody is exploiting.

FROM THE FIELD - PRACTITIONER'S NOTE

Clients drown when you hand them 300 findings sorted by CVSS. Sort by **fix-first** instead: (1) anything on the KEV list, (2) then high-EPSS + high-CVSS, (3) then everything else on a schedule. A one-page "do these five this week" gets more fixed than a perfect 80-page report that overwhelms them.



Priority is not the same as severity. A medium-severity bug that is being exploited in the wild (on the KEV list) is fixed before a high-severity one that no one is attacking.

IDENTIFIER	WHAT IT IS	ANSWERS
CVE	The unique ID of one specific known vulnerability.	“Which flaw?”
CVSS	0-10 technical severity from a vector (v3.1; v4.0 exists - not interchangeable).	“How severe, abstractly?”
EPSS	Daily ML probability (0-1) a CVE is exploited within 30 days.	“How likely to be attacked?”
CISA KEV	Curated list of CVEs with confirmed active exploitation (binary; not exhaustive).	“Exploited today?”
CWE	The weakness *class* (one CWE → thousands of CVEs).	“What kind of flaw?”

The CVSS **vector**, not just the number, must be read and the score computed mechanically (*Roundup*, not ordinary rounding). v3.1 base metrics: **AV** (Network/Adjacent/Local/Physical), **AC** (external prerequisites, not exploit difficulty), **PR** (None/Low/High), **UI**, **Scope** (Changed only when a different security authority is impacted - the most-misused metric), and **C/I/A** impacts. So `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` = an unauthenticated remote attacker, no interaction, full impact → 9.8 Critical. **Fix-first stack:** in KEV → immediate; else high EPSS + high CVSS → next; else high CVSS/low EPSS → schedule; cluster fixes by CWE. Severity ≠ risk ≠ likelihood (EPSS) ≠ confirmed exploitation (KEV) - keep the four distinct.

MEMORY TIP - FOUR DIFFERENT QUESTIONS

CVSS = how bad if exploited · **EPSS** = how likely · **KEV** = happening right now · **CWE** = what kind of flaw. Never answer one with another; “fix first” = KEV, then high-EPSS-and-high-CVSS.

4.7

Segmentation testing - the reachability matrix

The goal is to prove whether traffic from zone A can reach zone B *from a source position representative of a real host in zone A*. Build an N×N matrix (source zone → destination zone), each cell recording which ports/protocols are reachable vs blocked; the matrix maps to the intended policy (e.g. PCI: CDE isolated from non-CDE). Test with `nmap -sS -Pn -p <policy ports>` (use `-Pn` because you are testing the firewall, not host liveness) plus `-sU` and ICMP as the policy requires; `hping3` confirms a single rule.

SEGMENTATION REACHABILITY MATRIX - source zone (row) → destination zone (column)

source ↓ / dest →	Guest	Wkstn	Server/DC	Mgmt	DMZ
Guest	-	blocked	GAP · 445 reached	blocked	blocked
Wkstn	allowed	-	allowed · 443/445	blocked	blocked
Server/DC	blocked	blocked	-	allowed	blocked
Mgmt	allowed	allowed	allowed	-	allowed
DMZ	blocked	blocked	blocked	blocked	-

green = allowed (expected) · grey = blocked (intended) · red = reached but policy says block (a finding) · always run a positive control.

The reachability matrix turns segmentation into evidence: each cell is a tested source→destination path. The red cell is the finding - a guest host reached a server on SMB/445, which policy forbids.

INTERPRETING RESULTS - THE LOAD-BEARING DISTINCTION

`closed` (RST) usually means the host itself answered and the path is open - a segmentation gap even though no service was listening on that port - but a firewall or load balancer can also send the reset, so corroborate (a second port you know is listening, the firewall policy, or a capture) before reporting the path as open. `filtered` /no-reply is **Inconclusive** - it could be a firewall drop (working), a down host, or ICMP filtering. Never call it “blocked” without corroboration: test a port you *know* is listening on the destination and confirm it is unreachable from the source zone but reachable from an authorized zone. Always include a **positive control** (a target/port you expect to reach) to prove your source position and tooling actually work. A scanner in a management VLAN with broad allow rules cannot prove user-zone segmentation - its vantage is wrong.

4.8

The configuration and passive evidence planes

IN PLAIN WORDS

Not everything is best learned by actively poking the network. Sometimes the client simply hands you their device configuration files, and a tool (*Batfish*) works out from them what can reach what - no live traffic at all. Other times you learn from records the network already keeps: firewall logs, connection summaries (*NetFlow*), or a central security monitor (*SIEM*). This “passive” evidence is quieter and safer on sensitive networks. One caution: a sensor only sees its own corner, so “I didn't see it” never proves “it is blocked.”

Not every fact is best learned by active scanning. **Network-configuration analysis:** where a client will not grant interactive switch/firewall access, they export device configs and **Batfish** models routing/ACL/firewall reachability across Cisco, Palo Alto and other platforms - *modelled/static* evidence,

correlated with live tests. **Passive evidence** (preferred order): existing firewall/session logs → DNS/DHCP and flow (NetFlow/IPFIX) → SIEM/NDR/EDR → client PCAP or a SPAN/TAP sensor only when justified; **Zeek** parses traffic at an authorised point into structured connection/DNS/Kerberos/LDAP logs. A sensor sees only what is visible at its position (absence ≠ blocked), and structured metadata is always preferred over raw payload capture on sensitive networks.

4.9

Media and Apple environments

FROM THE FIELD - PRACTITIONER'S NOTE

On Mac-heavy and media networks, device discovery leans on mDNS/Bonjour, which stays on the local wire and will not cross a router. Run that discovery from the jump-box on the **same** segment, not over the VPN - from the wrong side you will conclude “nothing's here” on a network that is actually full of devices.

Media-heavy and Apple-based environments (common in content clients) need extra care. Device discovery relies on **mDNS / Bonjour**, which is link-local - it does not cross routing boundaries, so that discovery must run from the *same local segment* (the jump-box), not over the routed VPN. Content networks also carry sensitive material, so structured metadata is preferred over payload capture and content-protection guidelines apply to evidence handling.

4.10

Service enumeration cheat-sheet

IN PLAIN WORDS

Once you find an open door (an open port), you first *ask it politely what it offers* before trying any password - this is “enumeration.” Many services will happily tell an anonymous stranger their shared folders, user lists, or entire configuration if they were set up loosely. That over-sharing is *itself* the finding: a *null session* (reading Windows shares with no login), an *anonymous LDAP bind* (reading the directory with no login), or a default *SNMP community* all hand the target's structure to anyone who asks.

Once a port is open, each service is enumerated for what it leaks *before* any credential attack. All of the below is identification (safe in a VA scope); spraying/brute is not.

SERVICE · PORT	ENUMERATE	TOOL / COMMAND
SMB · 445	Null-session shares, users, groups, password policy, signing	`enum4linux-ng`, `netexec smb`, `smbclient -L`

SERVICE · PORT	ENUMERATE	TOOL / COMMAND
LDAP · 389	Anonymous bind, naming contexts, users/computers	`ldapsearch -x`, `netexec ldap`
SNMP · 161/UDP	Community strings, full device config, interfaces, routes	`onesixtyone`, `snmpwalk -c public`
DNS · 53	Zone transfer (full record dump), version	`dig axfr @ns domain`
SMTP · 25	User existence (VRFY/EXPN/RCPT), open relay, banner	`smtp-user-enum`, `nc`
NFS · 2049	Exported shares and their access rules	`showmount -e`
RPC · 135	Endpoint mapper, interfaces, domain info	`rpcdump.py`, `rpcclient`
Kerberos · 88	Valid usernames (no lockout on enumeration)	`kerbrute userenum`
Web · 80/443	Directories, vhosts, tech stack, parameters	`ffuf` / `gobuster`, `whatweb`, `httpx`

A null session (unauthenticated SMB/LDAP that still returns data), an anonymous LDAP bind, and a default SNMP community are themselves findings - they leak the target's structure to any unauthenticated attacker.

4.11

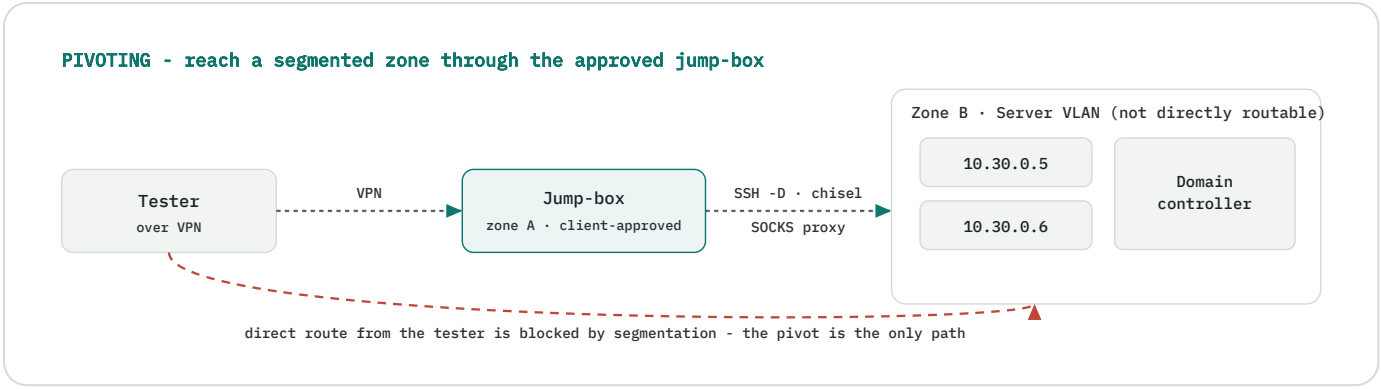
Pivoting, tunnelling & password attacks

FROM THE FIELD - PRACTITIONER'S NOTE

Before any password spray, establish the account-lockout policy - then spray low and slow, well inside it. Get this wrong and you lock out the client's real staff: a genuine operational incident and a very bad phone call. Enumerate usernames first (that does not trigger lockout); only then, carefully, try a single common password across them.

Reaching deeper - pivoting and tunnelling

Internal networks are segmented, so a position in one zone often cannot route to another directly. **Pivoting** uses an authorised foothold or the jump-box as a relay to reach otherwise-unreachable segments. The common mechanisms: an SSH dynamic tunnel (`ssh -D`) fronted by `proxychains`, or purpose-built tunnellers (`chisel`, `ligolo-ng`, `sshuttle`). In an assessment scope this is legitimate reach through the client-approved jump-box, correlated with the segmentation matrix (§4.7) - not exploitation; it is also how a probe in one zone is made to test from a representative position in another.



Pivoting is legitimate reach through the client-approved jump-box: the tester cannot route to zone B directly (segmentation blocks it), so a SOCKS tunnel on the jump-box relays traffic in - correlated with the reachability matrix (§4.7), not exploitation.

Password attacks - three kinds, and the lockout rule

KIND	WHAT IT IS	RISK
Brute-force	Many passwords against one account.	Noisy; triggers lockout fast.
Password spraying	One common password against many accounts, slowly.	The go-to - evades per-account lockout, but still needs the policy.
Credential stuffing	Breached username/password pairs replayed.	Depends on reuse; check against known leaks.

Always establish the account-lockout policy first. Enumerate usernames with `kerbrute` (no lockout on enumeration), then spray low-and-slow within the threshold and reset window. A careless spray locks out a client's staff - a real operational incident. Default and vendor credentials on exposed devices are identified as a matter of course.

PART V

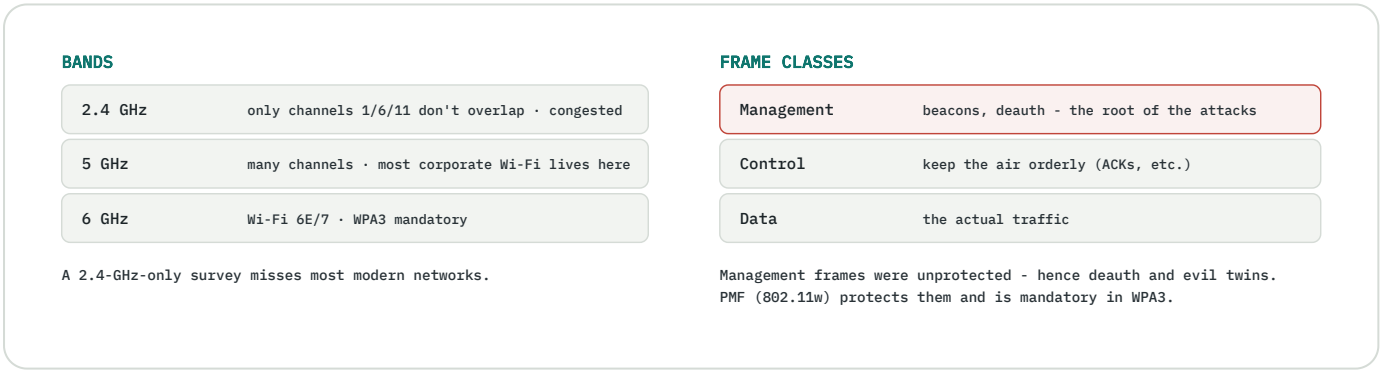
Wireless assessment

A peer domain, not a sub-task - because at its core it travels over radio, not IP. That single fact drives its methods, its hardware, and the limits on doing it remotely. Every technique here assumes a signed ROE naming the target SSIDs/BSSIDs and locations; RF attacks are active and can disrupt production and third-party devices in range.

5.1

802.11 fundamentals

> **IN PLAIN WORDS**
>
> Wi-Fi is just networking over radio instead of a cable, and its rulebook is called *802.11*. Two names to keep straight: the *SSID* is the network's name you see and pick (like “CorpWiFi”); the *BSSID* is the hardware address of one specific access point broadcasting that name. Wi-Fi runs on *bands* (2.4, 5 and 6 GHz) divided into *channels* - think radio stations. The security-relevant fact: some Wi-Fi “management” messages were historically unprotected, which is exactly what lets an attacker kick devices off the network or stand up a fake one.



Wi-Fi runs on three bands (survey all of them) and carries three classes of frame. The unprotected management frames are what enable deauth and evil-twin attacks - which is exactly what 802.11w (PMF) fixes.

Bands & channels: 2.4 GHz (only channels 1/6/11 non-overlapping; congested), 5 GHz (many channels, some DFS that must yield to radar), 6 GHz (Wi-Fi 6E/7; *mandates WPA3/OWE*, uses out-of-band discovery, and monitor/injection support is still immature). A 2.4-GHz-only survey misses most modern corporate infrastructure.

SSID vs BSSID: the SSID is the network name (not unique, can be hidden but leaks in probes - hiding is not security); the BSSID is a specific AP radio’s MAC. *You attack a BSSID, you enumerate by SSID.* **Frame classes:** management (beacons, probes, auth/assoc, deauth), control, data. Management frames were historically unauthenticated - the root of deauth and evil-twin attacks; **802.11w (Protected**

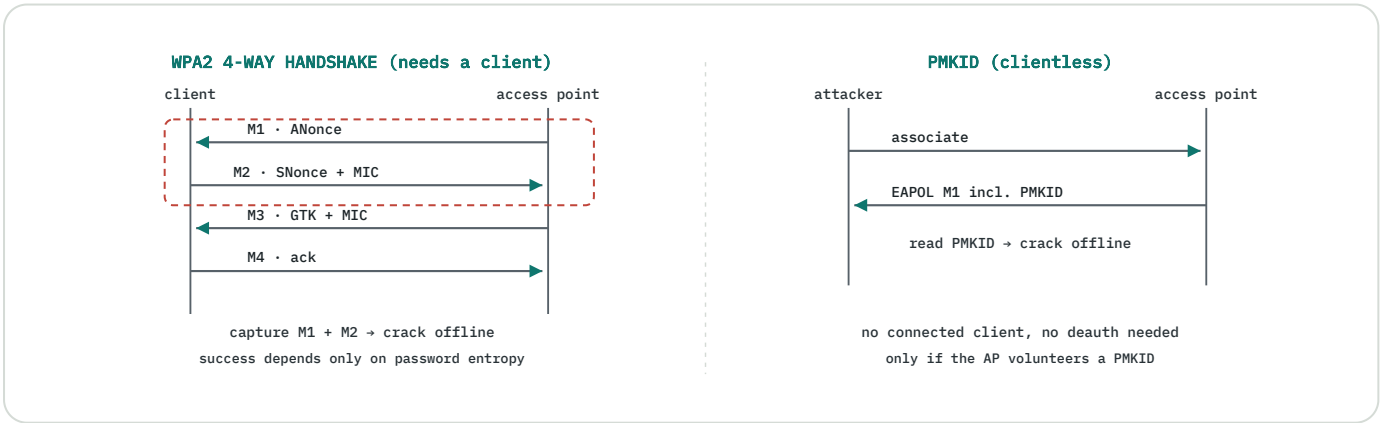
Management Frames) authenticates them and is mandatory in WPA3 and 6 GHz, defeating classic deauth. **Beacons** (~10/sec) carry the RSN Information Element - the cipher/AKM/PMF posture you fingerprint passively. **Probe requests** can leak a device's saved-network list (the basis of KARMA/known-beacon).

5.2

The handshake, PMKID, and the encryption generations

> **IN PLAIN WORDS**
>
> When a device joins a WPA2 Wi-Fi it does a *4-way handshake* - a quick back-and-forth that proves both sides know the password *without ever sending it*. If an attacker records that handshake they still can't read it, but they can guess passwords against it *offline* until one fits - which is why a long, random Wi-Fi password is everything. *PMKID* is a shortcut some access points allow that hands an attacker the same crackable material with *no other device connected*. The rest of this section is simply which Wi-Fi encryption generations are safe (WPA2 and WPA3) and which are broken (WEP, old WPA).

The 4-way handshake (WPA/WPA2) derives fresh session keys after association *without transmitting the password*: both sides already share the PMK (from PSK+SSID via PBKDF2, or from the enterprise EAP exchange). Capturing it gives an *offline oracle* - you recompute the key for each password guess and check the MIC. It does not reveal the password or decrypt traffic; success is entirely a function of password entropy. Messages 1+2 (or 2 of 4) with matching replay counters are typically enough - mixing frames from different handshakes yields an uncrackable capture, so verify on-site. **PMKID** is an optional value some APs volunteer in EAPOL message 1: `HMAC-SHA1(PMK, …)` - you associate yourself, read it, and crack offline with *no connected client and no deauth*. Not all APs include it, so both techniques stay in the kit.



Two ways to get an offline cracking oracle. The 4-way handshake needs a real client (capture M1+M2); PMKID needs only the attacker and a willing AP - no client, no deauth. Neither reveals the password or decrypts traffic; both just let you guess offline.

SCHEME	STATUS & NOTE
WEP	Broken (RC4, 24-bit IV reuse; FMS/PTW). Cracked in minutes - treat as open. Legacy OT/IoT only.
WPA-TKIP	Interim RC4 patch; deprecated, its own attacks (Beck-Tews). “WPA2-TKIP”/mixed is weaker and down-negotiable.
WPA2-CCMP (AES)	The sound baseline. The capture-then-crack attack is a pure offline brute force - the finding is “weak PSK,” not “WPA2 broken.” PBKDF2 uses only 4096 iterations with the SSID as salt, so identical SSIDs share precomputable tables. (KRACK was an implementation flaw, client-patched, not a password attack.)
WPA3-Personal (SAE)	Replaces the crackable handshake with a password-authenticated key exchange - no captured value tests passwords offline; each guess needs a live, rate-limited interaction, plus forward secrecy. *Dragonblood*: side-channels and, crucially, **transition mode** (accepting WPA2+WPA3) lets an attacker force the WPA2 path and capture a crackable handshake - a config exposure to test for.
WPA2/WPA3-Enterprise (802.1X/EAP)	No shared PSK; each user authenticates to a RADIUS server. Security hinges on **client-side server-certificate validation** - see below.

THE ENTERPRISE WI-FI MISCONCEPTION

“Enterprise is safe because there’s no shared password” is false. With **PEAP-MSCHAPv2** (the enterprise default), if the client does not validate the RADIUS server certificate, an attacker stands up a rogue RADIUS (eaphammer/hostapd-wpe), the client hands over the MSCHAPv2 challenge/response, and MSCHAPv2 reduces to single-DES - crackable regardless of password strength. The flagship enterprise finding is “clients don’t validate the RADIUS certificate” - a client-config issue. EAP-TLS (mutual certs) avoids it.

REMEMBER - CAPTURE ≠ PASSWORD

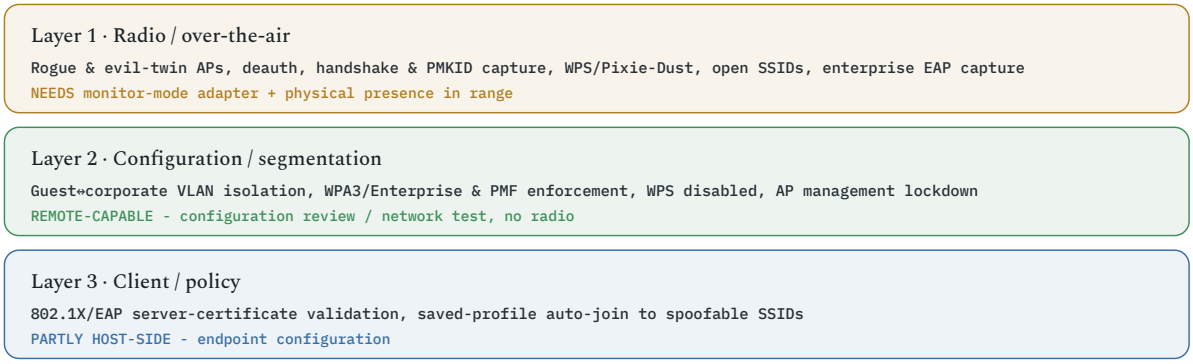
A captured handshake is only an **offline guessing oracle**; its success equals password entropy. WPA3-SAE removes the oracle - but **WPS**, **transition mode** and **no server-cert validation** put a crackable path back.

5.3

The three layers of a wireless assessment

FROM THE FIELD - PRACTITIONER'S NOTE

Before you book travel, split the wireless work into what needs the radio and what doesn't. Controller configuration, guest-isolation checks and policy review can often be done remotely from an export; only the over-the-air layer truly needs antennas and your feet on-site. Do the remote layers first, so the expensive on-site day is spent only on what actually requires it.



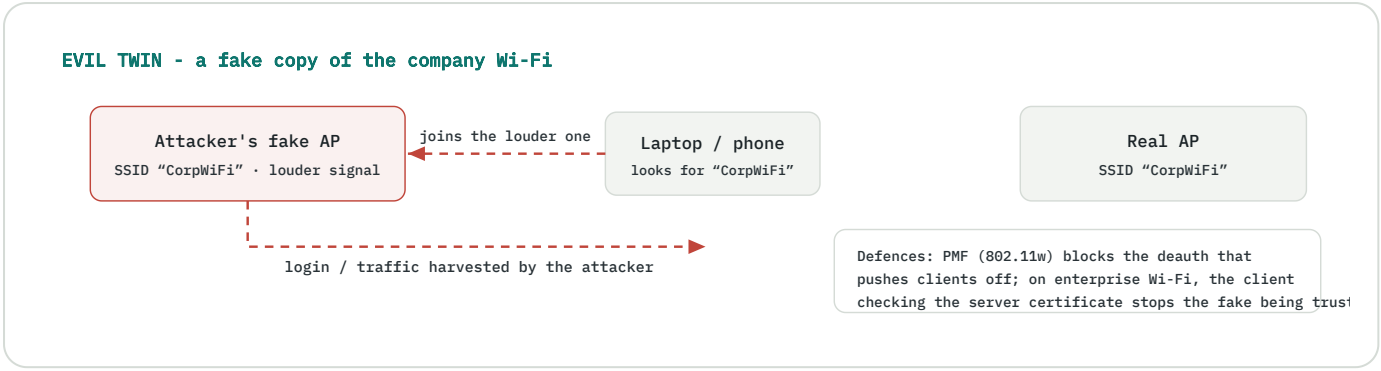
The three wireless layers - only Layer 1 needs the adapter and on-site presence; Layers 2-3 are configuration and network tests, doable remotely.

5.4

Wireless attacks and checks

IN PLAIN WORDS

These are the actual moves: capture a handshake, grab a PMKID, or stand up an *evil twin* - a fake copy of the company Wi-Fi that tricks devices into connecting so you harvest logins. You don't always need to kick anyone off the network; often the quiet PMKID grab is enough. The tool names (airodump, hcxdumptool, hashcat) just automate the capture and the offline password-guessing.



An evil twin is a fake access point using the real network's name. Devices set to auto-join “CorpWiFi” connect to whichever is loudest - so the attacker just needs a stronger signal, then harvests logins.

---

01 **Reconnaissance / survey (passive)**

Monitor-mode capture across *all bands*: BSSIDs, SSIDs (incl. hidden), channels, encryption/AKM/PMF from the RSN IE, connected clients. Non-disruptive; do it first and log neighbour (out-of-scope) networks explicitly since RF crosses property lines.

---

02 **Deauthentication**

Spoofed deauth forces a client to reconnect so the handshake can be captured - active, disruptive, a DoS; targeted single-client preferred. **Defeated by 802.11w/PMF** - then pivot to PMKID or wait for a natural join. You do not always need a deauth.

---

03 **Handshake / PMKID capture**

Lock airodump to the channel/BSSID and confirm “WPA handshake”; or use `hcxdumptool` for a client-free PMKID. Verify the capture is crackable on-site.

---

04 **Offline cracking**

Dictionary + rules + mask in hashcat (**mode 22000**; 16800/2500 are legacy) or aircrack-ng. The finding is “weak PSK.”

---

05 **WPS / Pixie-Dust**

The 8-digit WPS PIN collapses to ~11,000 guesses; **Pixie-Dust** recovers it offline in seconds from weak nonces, then the AP hands over the PSK *regardless of its strength*. Always check WPS is disabled - it bypasses “use a strong password” entirely.

06 **Evil twin / KARMA / captive portal**

Clone the SSID on a stronger signal (hostapd-mana/eaphammer/wifiphisher) for credential capture (personal), rogue RADIUS (enterprise), or captive-portal PSK phishing. KARMA/known-beacon/MANA exploit saved-network probing.

07 **Segmentation test (no RF)**

Join the guest SSID legitimately and test whether corporate/internal ranges, management interfaces or other clients are reachable - client isolation, DNS/DHCP leakage, lateral routes. Needs only network access; often the highest-severity finding ("guest Wi-Fi routes to the production LAN").

5.5

The wireless toolkit and the adapter

FROM THE FIELD - PRACTITIONER'S NOTE

"Alfa" is a brand, not a guarantee. What actually matters is the **chipset** and whether its driver supports monitor mode and packet injection on your system - the same product name has shipped with completely different chips (the classic TL-WN722N v1 versus v2). Run a quick injection test at your desk before you travel; discovering a dead adapter on the client's floor is a wasted trip.

The adapter: wireless work needs *monitor mode* (hear all raw frames, not just your own network) and *packet injection* (transmit crafted frames). These depend on the **chipset + driver, not the brand** - "Alfa" is a vendor, not a guarantee; verify with `aireplay-ng --test`. Known-good: **Atheros AR9271** (2.4 GHz only, rock-solid; note TP-Link TL-WN722N *v1 only* - v2/v3 changed chip and lost injection), **MediaTek MT7612U** (dual-band, mainline `mt76` driver - the recommended modern choice), **Realtek RTL8812AU** (dual-band, out-of-tree driver). A 2.4-GHz-only card silently misses the 5/6 GHz estate; 6 GHz monitor/injection support is still poor.

TOOL	ROLE (ALL OPEN-SOURCE)
aircrack-ng suite	`airmon-ng` (monitor), `airodump-ng` (survey/capture), `aireplay-ng` (deauth/injection), `aircrack-ng` (crack).
hcxdumptool / hcxtools	Client-free PMKID + handshake capture; `hcxpcapngtool` → hashcat mode 22000.
hashcat	GPU cracker (mode 22000 for WPA).

TOOL	ROLE (ALL OPEN-SOURCE)
Kismet	Passive survey/IDS; hidden-SSID discovery, device tracking (no injection).
wifite / bettercap	Automation wrapper / modular Wi-Fi framework (recon, deauth, capture, MITM).
hostapd-mana · eaphammer · wifiphisher	KARMA/MANA · enterprise EAP evil-twin & credential capture · captive-portal phishing.
Wireshark	Frame analysis - verify a handshake, inspect RSN IEs (not an attack tool).
Reaver / Bully	WPS PIN brute + Pixie-Dust.

```
# WPA2-PSK handshake capture + crack (authorized)
airmon-ng check kill; airmon-ng start wlan0
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
aireplay-ng --deauth 5 -a AA:BB:CC:DD:EE:FF -c <CLIENT> wlan0mon   # force reconnect
hcxpcapngtool -o hash.22000 capture.cap
hashcat -m 22000 hash.22000 wordlist.txt -r rules/best64.rule
```

5.6

Standards and the remote limit

FROM THE FIELD - PRACTITIONER'S NOTE

Be honest and early about the one hard limit: the over-the-air part of wireless cannot be done down a VPN - radio needs a body and an antenna in the building. Put that in the scope conversation up front so the client plans an on-site day, rather than expecting it remotely and being disappointed halfway through.

Wireless anchors to **NIST SP 800-115 §4.4** (passive scanning for rogue/AP enumeration and encryption posture; active scanning against discovered devices; wireless device location). 800-115 (2008) predates WPA3/6 GHz/PMF, so pair it with **NIST SP 800-153** (WLAN hardening), the Wi-Fi Alliance specs (WPA3/OWE/PMF), and - for cardholder environments - **PCI DSS Req 11** (quarterly rogue-AP scanning). Frame the work as *survey → posture from the RSN IE → targeted validation → impact via segmentation → report*, with the standing rule that the finding is almost always a **configuration or password-entropy weakness** (weak PSK, WPS enabled, PMF off, no server-cert validation, flat guest VLAN), not "the encryption is broken."

THE HARD REMOTE LIMIT

Handshake capture, deauth, PMKID and evil-twin all need the adapter *within radio range* - they cannot be done over a VPN unless a box with its own local adapter is physically there. RF work is on-site work, which is a scoping and logistics fact for the ROE. The Layer-2 segmentation test is the remote-capable exception.

5.7

Beyond Wi-Fi - Bluetooth, BLE and rogue devices

FROM THE FIELD - PRACTITIONER'S NOTE

In sensitive areas - cardholder floors, R&D labs - do a quiet rogue-device sweep. A cheap Bluetooth/BLE scanner catches drop-boxes, unexpected trackers and personal hotspots that policy forbids. It is five minutes of walking, and every so often it finds the single thing that matters most.

The wireless domain extends past 802.11. NIST SP 800-115 §4.4 explicitly includes **Bluetooth / BLE**, assessed with `bluetoothctl`, **Ubertooth** and `btlejack` for discovery, pairing security and sniffing; mature programmes also consider Zigbee and other RF, and **rogue-device detection** - unauthorised APs, beacons and drop-boxes on or near the estate (a PCI DSS requirement in cardholder environments). The same hard rule holds: RF is local, so this is on-site work requiring the right radio within range.

PART VI

Web, API & cloud assessment

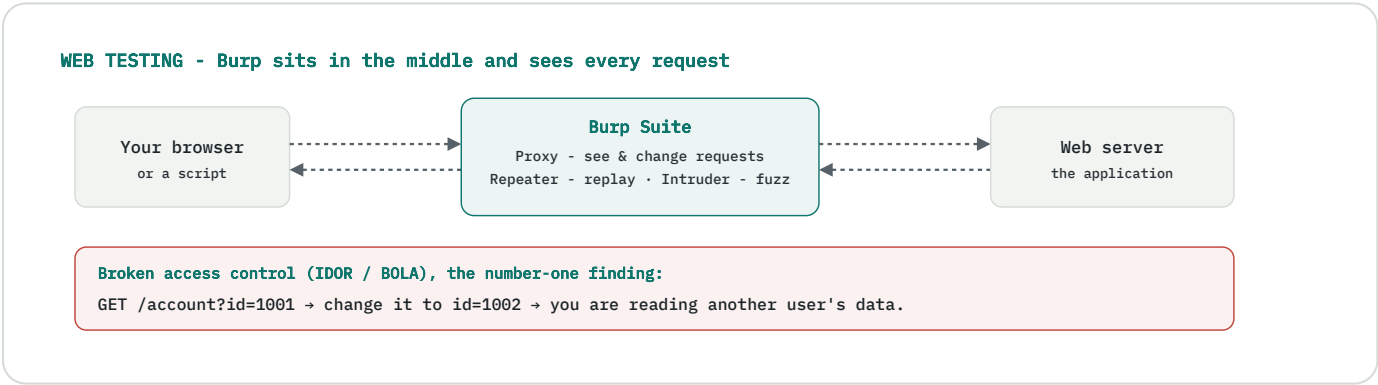
Adjacent, manual-heavy disciplines - the web/API engine is the one already mature in-house. They share the evidence contract and reporting pipeline but need different technical methods, and share no code with the host toolkit.

6.1

Web application testing

IN PLAIN WORDS

A web-app test looks for ways a normal-looking website lets you do things you shouldn't. The classic tool is *Burp Suite*, which sits between your browser and the site so you can see and change every request. The biggest category is not fancy code exploits - it is *broken access control* (also called IDOR or BOLA): change an ID in the address from `1001` to `1002` and you are suddenly reading someone else's data. A scanner rarely catches that; a human replaying the same request as different users does.



Burp Suite sits between the browser and the site, so you can read and edit every request. The classic finding needs no exploit code - just changing an ID from 1001 to 1002 and seeing someone else's data come back.

Human-driven analysis of an application's logic, following the OWASP **WSTG** (the methodology) - not a scan. The OWASP **Top 10 (2021)** is an awareness/reporting list, not a coverage standard: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection (incl. XSS), A04 Insecure Design, A05 Security Misconfiguration (incl. XXE), A06 Vulnerable Components, A07 Auth Failures, A08 Software/Data Integrity, A09 Logging/Monitoring Failures, A10 SSRF. WSTG phases run information-gathering → config → identity → authentication → **authorization** → session → input validation → errors → crypto → **business logic** → client-side.

The Burp Suite Professional workflow

Proxy (MITM backbone) → **Target/site map** (scope tightly to the ROE) → **Repeater** (manual single-request tampering - where IDOR/BOLA, authz and injection are confirmed by hand) → **Intruder** (parameterised fuzzing: Sniper/Battering-ram/Pitchfork/Cluster-bomb) → **Scanner** (crawl-and-audit). **curl** captures clean copy-pasteable PoCs; **Python** handles multi-step logic. The high-value triad is authentication (enumeration, MFA bypass, reset-token strength), session (cookie flags, fixation, token entropy via Sequencer), and access control (a role matrix with ≥2 accounts per role; the cleanest PoC is "same request, victim's object ID, attacker's session → 200 + victim's data").

THE LOAD-BEARING MISCONCEPTION

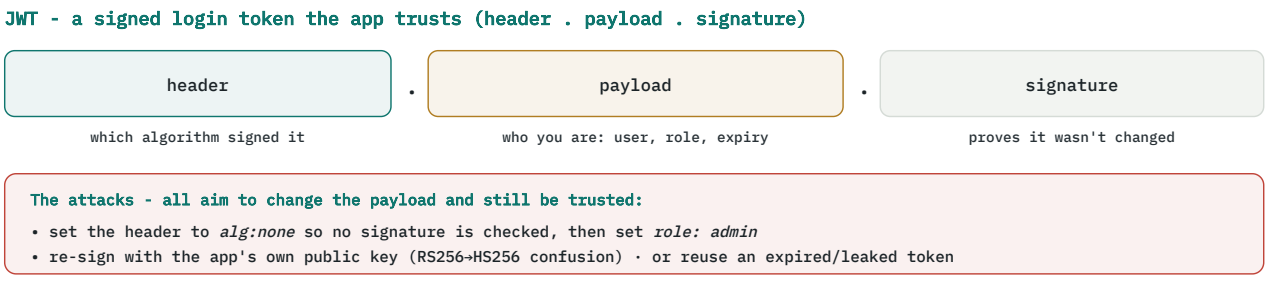
The scanner does **not** find broken access control, BOLA/IDOR or business-logic flaws - it has no model of who *should* be authorized. Those phases are manual replay with a second identity, and they are the value that separates a pentest from a scan. Every scanner-derived finding is reproduced in Repeater before it is reported.

6.2

API testing

IN PLAIN WORDS

An *API* is the machine-to-machine version of a website - the same app's data, spoken in raw requests instead of pages on a screen. Because nothing visual is hiding things, the number-one problem is *authorization*: can user A fetch user B's data just by changing an ID (*BOLA*), or call an admin-only action (*BFLA*)? Logins often carry a *JWT* - a signed token the app trusts; a whole class of bugs is getting the app to accept a token it should have rejected. With seven roles and many endpoints the combinations explode, which is why API testing is mostly patient, methodical permission-checking.



A JWT is three parts joined by dots. Every classic attack is a way to change the middle part (your identity and role) and still have the app accept the token - which is why the signature settings matter so much.

The OWASP **API Security Top 10 (2023)** is dominated by authorization: **API1 BOLA** (swap an object ID, get another tenant's object - the #1 API risk), API2 Broken Authentication, **API3 Broken Object Property Level Authorization** (excessive data / mass assignment, e.g. injecting `"role":"admin"`), API4 Unrestricted Resource Consumption, **API5 BFLA** (reach a function above your role), API6 Sensitive Business Flows, API7 SSRF, API8 Misconfiguration, API9 Improper Inventory (shadow/zombie APIs), API10 Unsafe Consumption.

Authorization dominates because APIs expose object identifiers directly and are consumed by clients the server can't trust - every object reference is an authz decision the server must re-make on every call, and UI restrictions mean nothing. With RBAC via token claims (roles in a JWT), three test classes define the engagement: **BOLA/BFLA** per role/endpoint/verb; **cross-role/cross-tenant** (an N×endpoints matrix; a 7-role engagement is combinatorially authz-heavy); and **JWT tampering** (`alg:none`, RS256→HS256 confusion, unverified signature, edited role/tenant claims, stripped `exp`). Tools: Burp (Repeater, Intruder to sweep IDs, Authorize extension to diff a low-priv vs high-priv session, JWT Editor), Postman (import OpenAPI to build the inventory and per-role token environments), Python (drive the full matrix), curl (PoC). Postman/Swagger give the inventory; the authorization verdict is always manual replay - and always test API9 for endpoints the spec omits.

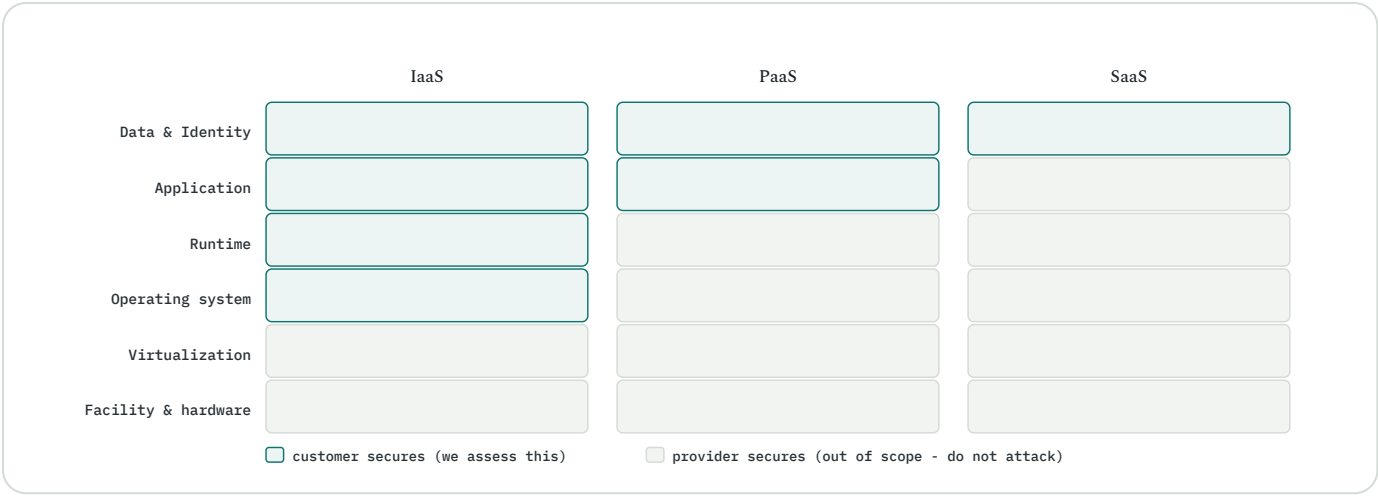
6.3

Cloud assessment

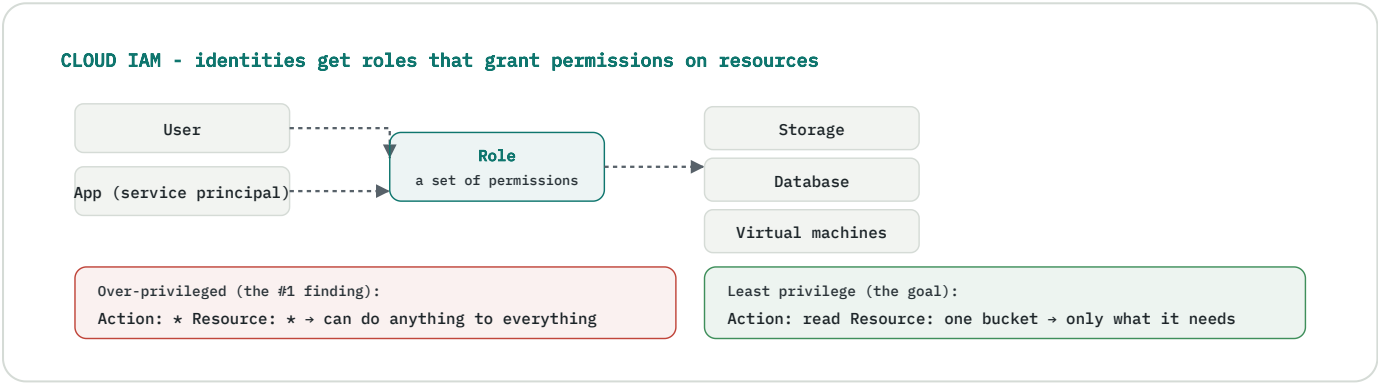
> **IN PLAIN WORDS**
> Testing "the cloud" (AWS, Azure, Google) is not about breaking the provider - you never attack Amazon. It is an *identity and configuration audit*: are permissions too broad (an account that can do *everything*)? Is storage or a database left open to the internet? Is admin access protected with extra login checks? The *shared responsibility* idea (diagram below) is the key - the provider secures the building; the customer secures how they set up their rooms, and the customer's half is all we test.

Cloud is **predominantly a credentialed configuration and identity review**, not remote exploitation: usually authenticated read-only tenant access, almost entirely via provider APIs, governed by the provider's rules of engagement. Findings are public storage buckets, over-permissive IAM (`*:*`), missing MFA on privileged/root accounts, unencrypted storage, exposed management ports in security groups/NSGs, disabled logging (CloudTrail/Activity Log), stale credentials, conditional-access gaps.

What the customer owns - and therefore what we assess - depends on the service model. This is the **shared-responsibility** split:



The shared-responsibility split. The lower the service model, the more the customer owns - and the assessment always targets the customer's half.



In the cloud, identities are given roles, and roles grant permissions on resources. The most common finding by far is an identity whose role can do far more than it ever needs - the gap between the red box and the green one.

PROVIDER	KEY THINGS TO CHECK
Azure / Entra ID	Privileged roles (Global Admin, PIM), Conditional Access, app registrations & service-principal consent, legacy auth, guest/B2B access, ARM vs data-plane.
AWS	IAM users/roles/policies (wildcards), S3 public exposure, security groups, KMS, CloudTrail, Organizations/SCPs, IMDSv1 vs IMDSv2 (SSRF).
GCP	IAM bindings, service-account key sprawl, org policies, public GCS buckets, firewall rules, project hierarchy.

TOOL	CLOUD	WHAT IT DOES
ScoutSuite	AWS/Azure/GCP	Multi-cloud read-only config auditor → HTML risk report.
Prowler	AWS (+Azure/GCP/M365)	Read-only checks mapped to CIS/PCI/HIPAA; compliance-oriented.
ROADtools / ROADrecon	Azure/Entra	Enumerates Entra (users, apps, roles, service principals) into a queryable DB.

TOOL	CLOUD	WHAT IT DOES
ScubaGear	M365	CISA tool checking M365/Entra against SCuBA baselines.
Pacu	AWS	Offensive post-credential framework - RoE-sensitive, not a posture tool.

NOT "ATTACKING THE CLOUD"

You do **not** attack AWS/Azure/GCP infrastructure - that violates provider RoE and is their half of the *shared-responsibility model* (the provider secures the cloud; the customer secures what's in it). All three providers prohibit DoS/stress testing and testing other tenants or managed control planes. Cite the current provider policy in the ROE. Benchmark against the relevant CIS Benchmark for the platform.

REMEMBER - CLOUD IS IDENTITY, NOT PACKETS

A cloud review is a read-only **configuration & identity audit** against a CIS benchmark. You never attack the provider; you check the customer's half of the shared-responsibility line.

6.4

Mobile, containers & the wider surface

IN PLAIN WORDS

The same skills stretch to other surfaces. *Mobile* apps are checked for secrets baked into the app and for weak connections. *Containers* and *Kubernetes* (the system that runs lots of small app pieces called "pods") are checked for over-privileged pods and default settings. *OT* - operational technology, the computers that run factories, power and other physical processes - is *passive-only*: you look but never actively scan, because a single stray packet can stop a production line or a turbine.

- **Mobile applications** - a separate workstream under OWASP **MASVS / MASTG**: static analysis (insecure storage, hardcoded secrets, weak crypto), dynamic analysis (certificate pinning, runtime), and the API backend - which is the API testing of §6.2. The client app is fully attacker-controlled, so the server remains the enforcement boundary.
- **Containers & Kubernetes** - a config-review discipline against the **CIS Kubernetes Benchmark**: exposed dashboards/kubelet, over-broad RBAC, privileged pods, image vulnerabilities (`kube-bench`, `kube-hunter`, `Trivy`). Cloud-adjacent, mostly identity and configuration.

- **OT / ICS / SCADA** - industrial control systems are *passive-only*: active probing can disrupt physical processes, so these are assessed by configuration review and passive observation under a strict safe profile, never active scanning on production.

PART VII

The host assessment toolkit - full specification

The instrument itself, from design philosophy to protocol detail: the 45 evidence sources, the 45 rules, the patch engine, deployment, and validation status.

7.1

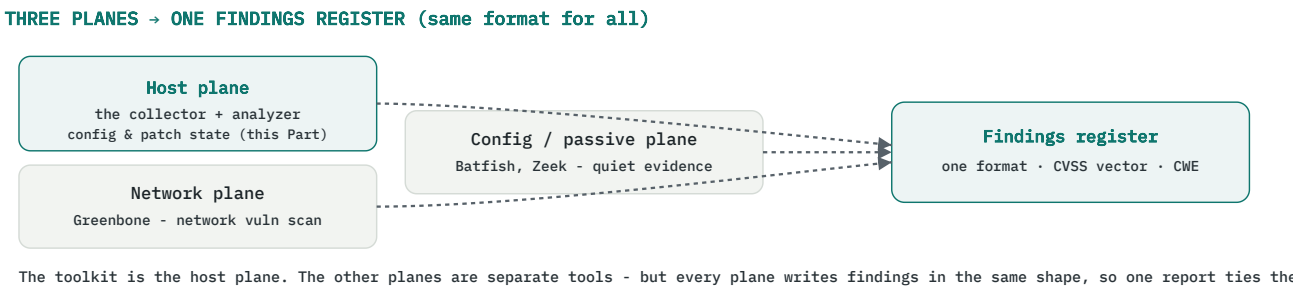
Product shape - one workflow, several planes

IN PLAIN WORDS

The toolkit is not one giant script - it is a small assembly line. One part *collects* evidence from a Windows machine, another *seals* it so it cannot be secretly changed, another *judges* it against rules, and a last part turns the results into a report. Splitting the jobs is deliberate: the part that collects can never influence the verdict, and that separation is what makes the findings trustworthy.

THE CODE IS PUBLIC

The reference implementation described in this Part is published as open source under the MIT licence: PostureKit on GitHub. It contains the same collector, launcher, sealed-batch model, analyzer, rule set and importers this Part specifies, plus a synthetic offline demonstration you can run without touching any host. Read the code alongside this Part; every section below names the file it describes.



The toolkit is one of three “planes” of evidence - host, network and passive. Each uses different tools, but all write findings in one shared format so a single report can combine them.

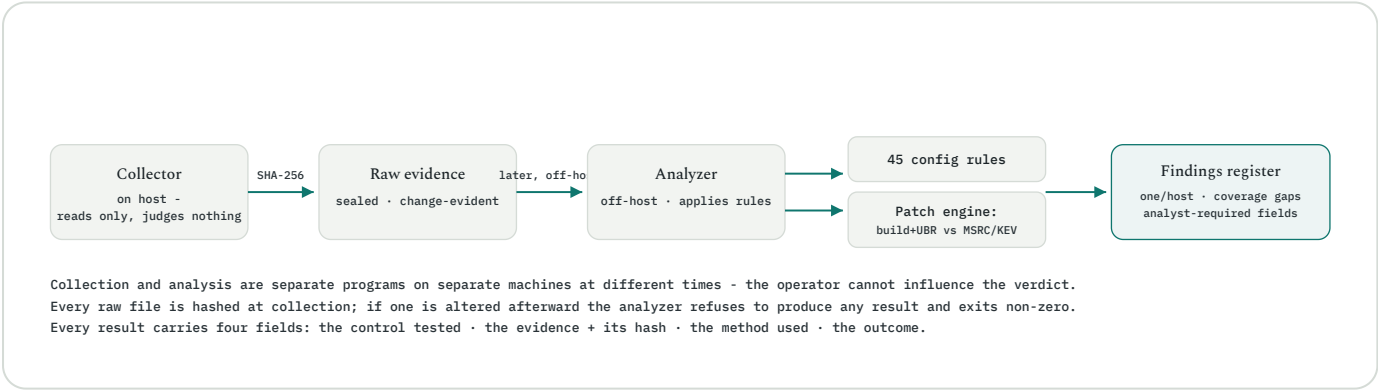
The toolkit is **software implemented as scripts and modules**, not a single script. Its architecture decision is deliberate: one topology-aware, evidence-first internal assessment toolkit - not one giant vulnerability script, and not a permanent endpoint agent as the default.

PLANE	RESPONSIBILITY
Controller / Planner	Scope, inventory, execution planning, job and coverage tracking.
Windows Evidence Module	The collector: native PowerShell local/WinRM + a reduced agentless CIM mode. Host, security-config and directory evidence in one collection.
Update Determination	Outstanding Microsoft OS updates derived off-host from the collected servicing level vs vendor data. No the CVE database; nothing executed on the host.
Network Plane	Separately installed Nmap + a maintained vulnerability engine (Greenbone or the client's scanner).
Segmentation · Passive · Network-Config planes	Source-zone probes; client firewall/DNS/flow/SIEM first, SPAN/TAP + Zeek when justified; exported device configs + Batfish.
Evidence Plane	Change-evident raw + hashes → normalised Evidence.json / Tests.csv / Coverage.csv → drafted findings → senior validation → reporting.

7.2

Design philosophy - why it is defensible

The value over a black-box scanner is not more checks; it is **evidence integrity**, produced by three principles.



The toolkit data flow - separation of collection from judgement, plus sealing, is what makes a finding defensible.

Collect and judge are separate programs (the operator cannot influence the verdict); **evidence is sealed at collection** (a tampered file makes the analyzer refuse); **credentialed, from the inside** (reads the exact registry value and build - the most accurate position, almost no false positives). Every result carries the **four fields**: the control tested, the evidence and its hash, the method used, the outcome - so any finding traces back to a control and a tamper-evident artifact.

7.3

The collector and its transport

> **IN PLAIN WORDS**
> To read a Windows machine's settings you have to log in and run the checks. The toolkit does this over *WinRM* - Windows' own built-in remote-management channel - using the client's admin account, running in memory and writing nothing to their disk. On a domain it logs in by computer name (Kerberos); on a standalone machine it uses an encrypted (HTTPS) login. The key discipline: we never switch remote access on ourselves - if it is off, the client turns it on, because quietly enabling it would itself be a security change.

01 **Transport: WinRM (WS-Management)**

Built-in Windows remote management (5985 plain / 5986 TLS), the channel PowerShell Remoting uses. Nothing new is enabled on the client; WinRM/firewall are never auto-enabled to get in.

02 **Authentication: Kerberos (domain) / validated HTTPS (workgroup)**

Domain: DNS/FQDN + Kerberos - never an IP literal as a Kerberos identity. Workgroup: validated HTTPS with the real certificate and explicit credentials - never wildcard `TrustedHosts` or disabled cert validation.

03 **Execution: the script text runs in the remote session**

The collector's text is sent down the session and executed in memory (`Invoke-Command -FilePath`). No file copied, no service installed, no agent left; evidence returns through the same session - sidestepping the second-hop delegation problem.

04 **Invocation: non-interactive**

In production, started by Configuration Manager, Intune, RMM (remote monitoring and management), Group Policy or a scheduled task (`-NonInteractive -File`) - never a human console. Two defects were found that appear *only* under non-interactive invocation, which is why it is tested that exact way.

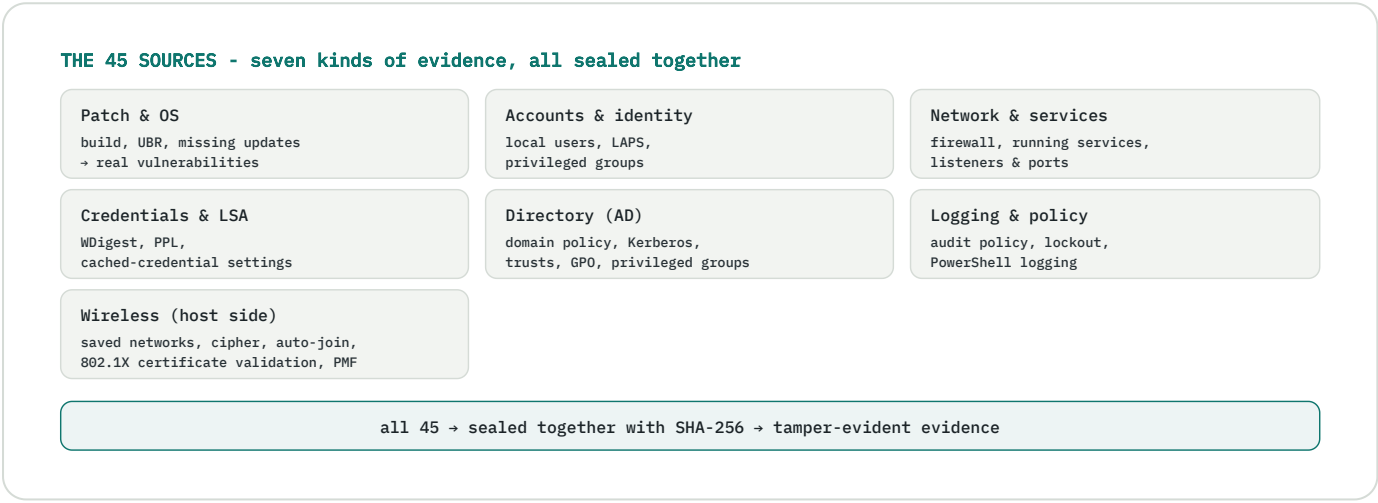
Collection profiles bound what is gathered: **Minimal** (identity + selected controls/listeners), **Standard** (+ bounded service/software/update inventory), **Deep** (+ targeted hardening/role evidence). Collection must be justified by an objective and constrained by per-host size budgets.

7.4

The 45 evidence sources

IN PLAIN WORDS

A “source” is one category of evidence the tool reads - patch level, local accounts, firewall state, running services, and so on (45 in all). It records *facts*, not opinions, and only what is needed: for example it notes *whether* an auto-login password exists, never the password itself. How you read a setting matters more than it sounds - the tables below flag several Windows traps where the obvious command returns a wrong answer.



The 45 sources are seven kinds of fact - patch state, accounts, network, credentials, directory, logging, and the host's wireless configuration. They read facts, not opinions, and all seal together so the evidence cannot be quietly changed.

Grouped with what each reveals. Each records a status and a plain caveat; only whether an auto-logon password *exists* is recorded - never the value.

IDENTITY, INVENTORY & BUILD

`patchlevel`	Exact servicing level (build + UBR) - the authoritative patch state (§7.7).
`software` · `hotfixes`	Installed software from the uninstall registry (never `Win32_Product`); hotfix list is inventory only, not patch state.
`services`	Services with binary path + logon account; an unquoted-path candidate flag (candidate only).
`addresses` · `defender` · `optionalfeatures` · `bootintegrity`	Local IPs; Defender state; installed features/roles; Secure Boot + volume encryption (need elevation - null on a non-elevated run means *not determined*).

EXPOSURE - LISTENERS, CONNECTIONS, ACCOUNTS

`tcplisteners` · `udpendpoints` · `connections`	Local listening ports (a listener ≠ remote exposure); point-in-time connections (no payloads).
`localadmins` · `localguest`	Direct local-admin membership (no nested expansion); built-in Guest by RID.

SECURITY CONFIGURATION (THE RULE SURFACE)

`firewall`	Domain/Private/Public profile state.
`smbserver` · `smbclient` · `smb1driver`	SMB signing and SMBv1 state, both sides + client driver.
`rdp` · `uac` · `uacpolicy`	RDP/NLA config; UAC master switch and policy detail.
`lsa` · `wdigest` · `logonpolicy`	LSA protection, LM-hash, legacy-auth level; WDigest caching; interactive-logon/auto-logon presence.
`nameresolution` · `installerpolicy` · `pointandprint`	LLMNR/NetBIOS; Installer “always elevated”; printer-driver install (PrintNightmare).
`powershelllogging` · `pscorelogging` · `autorunpolicy` · `passwordpolicy`	PS script-block/module logging (WinPS + PS7); AutoRun; local password/lockout policy (English-parsed).
`laps` · `lapsconfig` · `updatesource` · `winrmconfig` · `credentialguard`	LAPS presence + `BackupDirectory`; update server over http; WinRM cleartext/basic; Credential Guard.

ACTIVE DIRECTORY (DOMAIN-JOINED ONLY)

`domainidentity` · `domainpolicy`	Identity + functional level; default password/lockout policy and `MachineAccountQuota`.
`domainprivilegedgroups` · `domaintrusts` · `kerberospolicy` · `gpoapplied`	Privileged membership (by RID, a floor); trusts; Kerberos/secure-channel; applied GPOs.

WIRELESS CONFIGURATION (HOST SIDE, SEE PART V)

`wirelessadapters` · `wirelessinterface`	Whether a wireless adapter exists and, if connected, the current network's authentication, cipher and management-frame protection. A host with no adapter records the wireless rules as *Not Applicable*, never a false pass.
`wirelessprofiles` · `wirelessposture`	Every saved network profile (open, legacy WEP/WPA, TKIP, auto-join, pre-shared key or 802.1X, and whether an 802.1X profile validates the server certificate) and a one-row summary the eight WLAN rules evaluate. No pre-shared key is ever read or exported.

The six directory sources read through libraries on every Windows host - no RSAT, no ActiveDirectory module. On a workgroup host all six record *Not Applicable*. The three domain-

level sources (policy, privileged groups, Kerberos policy) are read once from the domain controller; a member host collected through a remote session records them as *Not Applicable* with the reason, because a remote session cannot bind the directory on the host's behalf (the credential second hop).

7.5

Evidence integrity and result semantics

FROM THE FIELD - PRACTITIONER'S NOTE

A “hash mismatch” is not always tampering. On Windows, a file path longer than 260 characters makes the verifier report files as missing or invalid - which looks exactly like altered evidence. Always re-check from a short path (like `C:\r2\`) before you ever accuse a batch of being tampered with; the panic is nearly always a path-length problem, not an attacker.



A check never just says “bad.” It records one of six honest states - and a Fail is only an observation until a human reviews it against the client's baseline. That discipline is what keeps false findings out of a report.

Four evidence layers, strictly bounded: **Raw** (change-evident once hashed), **Derived** (regenerable Evidence.json/Tests.csv/Coverage.csv), **Manual** (verification + notes, each hashed), **Report** (only after validation). Sealing: every raw file is SHA-256-hashed at collection; the analyzer re-hashes - match proceeds, mismatch stops with zero results. Raw evidence is never edited; a correction is a new run.

VERIFY FROM A SHORT PATH

A Windows path over 260 characters makes a sealed file read as `MISSING OR INVALID` - a path problem masquerading as tampering. Verify manifests from a short working path; otherwise a real `HASH MISMATCH` cannot be told apart from a path-length artifact.

STATE	MEANING
Pass / Fail	Expected condition observed / a configuration observation (not automatically a vulnerability or severity).

STATE	MEANING
Unknown / Error	Insufficient evidence / could not run (provider message preserved).
Not tested / Not applicable	Not attempted (visible in coverage) / does not apply (e.g. domain check on a workgroup).
Inconclusive	Several possible causes - a failed connection is Inconclusive, never Blocked and never Pass.
Observation / Candidate	Seen (not automatically bad) / needs applicability validation before reporting.

The coverage denominator comes from the *approved inventory*, not only hosts that returned evidence - so “zero findings” can never hide missing assets - and a CVE is never inferred from a missing KB, a port, or an unverified banner.

7.6

The analyzer and the 45 configuration rules

> **IN PLAIN WORDS**
> The “analyzer” is the part that judges the collected evidence against 45 *rules* - each rule a single yes/no question like “is SMB signing required?” A rule can also say what a *missing* setting means, because sometimes absence is safe and sometimes it is the risk. Importantly, a failed rule is an *observation*, not yet a finding: a human still checks it against the client's own agreed baseline before it goes in a report.

45 rules across nine categories. It is a lab reference profile, explicitly not a certification: a *Fail* is a configuration observation requiring analyst review against the client’s approved baseline before it becomes a finding. Severity is never assigned by the rule set.

CATEGORY	RULES	REPRESENTATIVE CHECK (800-53)
Firewall	FW01-03	Domain/Private/Public profile enabled (SC-7)
SMB	SMB01-04	SMBv1 disabled; signing required (CM-7, SC-8/23)
RDP · UAC · Accounts	RDP01, UAC01-03, SAM01	NLA; admin-approval mode; remote local-account filter; Guest disabled (AC-6/17, AC-2)
Credential protection	CRED01-09	WDigest, LSA PPL, LM hash, cached logons, auto-logon, LAPS, LmCompatibility (IA-5, SC-39)
Name resolution	NET01-02	LLMNR disabled; NetBIOS off per interface (SC-20)

CATEGORY	RULES	REPRESENTATIVE CHECK (800-53)
Privilege escalation	PRIV01-02, PRN01-02	Installer-always-elevated; printer-driver install restriction (AC-6, CM-11)
Audit logging	LOG01-03	PowerShell script-block/module logging (AU-2, AU-12)
Account policy · media · patch · remote mgmt	PWD01-02, MEDIA01, UPD01-02, WRM01-02	Password length/lockout; AutoRun; update source https; WinRM no cleartext/basic (IA-5, AC-7, MP-7, SI-2, SC-8)
Wireless (host side)	WLAN01-08	No open network on auto-join; no legacy WEP/WPA or TKIP; 802.1X profiles validate the server certificate; corporate networks not on a shared key; management-frame protection on the connected link (AC-18)

Rule anatomy - a worked example

```
// CRED02 - LSA runs as a protected process
source: "lsa"   field: "RunAsPPL"
type:   "int_any"   expected: [1, 2]      // 1 and 2 both enable protection
absent_means: { result: "Fail" }         // no value → LSASS exposed
control_refs: [ SC-39 Process Isolation, IA-5 Authenticator Mgmt ]
```

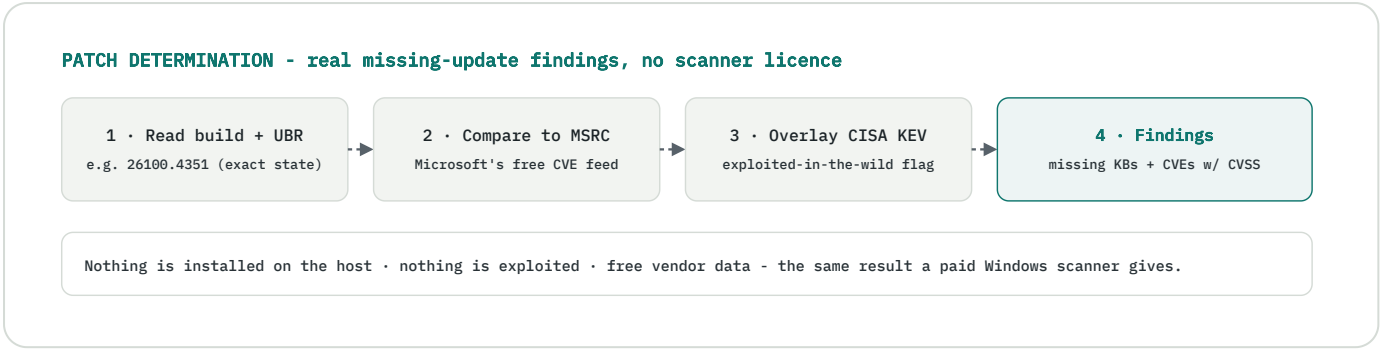
Comparisons go beyond equality - an accepted set (`int_any`) and `numeric_max` / `min` / `between` - because LSA protection 2 is as valid as 1, and a lockout threshold of 0 *disables* lockout (weaker than any positive value, so `PWD02` declares the range 1-5). **`absent_means`** interprets a missing value only when the rule states what its absence means: `CRED01` (WDigest) is version-dependent, so an absent value reports *Unknown* with the dependency stated, not a false Pass. Every rule carries its 800-53 control references - the first of the four fields.

7.7

Patch determination - finding real vulnerabilities without a Windows scanner

IN PLAIN WORDS

This is how the tool finds *real* missing-update vulnerabilities without buying a scanner licence. It reads the exact Windows build number - including the small part after the dot (the *UBR*), which is what actually separates patched from unpatched - then compares it against Microsoft's own free published list of fixes (*MSRC*) and the US government's list of bugs being exploited right now (*KEV*). Same result a paid scanner gives, from free vendor data. One rule: never judge patch state from the list of installed updates - on modern Windows that list is incomplete and will mislead you.



The patch engine reads the exact build (including the UBR revision), compares it to Microsoft's own free list of fixes, flags anything on the government's exploited-now list, and outputs findings - no commercial scanner needed.

THE TRAP - NEVER DO THIS

Reading the installed-hotfix list (`Get-HotFix` / `Win32_QuickFixEngineering`) and checking missing KBs. Modern Windows ships *cumulative* updates that supersede hundreds of KBs; the hotfix list names the cumulative packages that were installed but cannot tell you what they superseded or what is still missing, so it understates patch state on Windows 10, 11 and Server 2016+. Never use it for patch state.

---

01 **Record the exact servicing level**

Build + update revision (UBR) together, e.g. `10.0.26200.9457`, read from `HKLM\…\CurrentVersion` - the unambiguous patch state.

---

02 **Compare against Microsoft’s free feed**

The MSRC CVRF data publishes, per CVE, the fixed build per product - **free, no API key**. Build lower than the fix → the vulnerability is present. An unidentifiable product is *Unknown*, never patched.

---

03 **Report with the vendor’s severity, overlay CISA KEV**

Each item carries Microsoft’s CVSS vector and the update required; KEV marks which are exploited now → “fix these first.”

Performed **off-host** against vendor data - a §4.3 assessment, never an exploit. A validation lesson: a three-month window hid 15 of 16 exploited CVEs a twelve-month window found, so the default is twelve months with a truncation warning.

TWO DELIBERATE BOUNDARIES

No the CVE database, and no third-party CVE matching in the collector. For non-OS software, version-string matching is a graveyard (backported fixes flagged vulnerable, ~missing CPEs, a naïve Chrome match tagging dozens of products), so that work is left to a maintained scanner; the toolkit flags end-of-life software and CISA-KEV presence + curated version floors instead. A local, offline WUA module with a dated Microsoft-signed `Wsusscn2.cab` exists for deeper applicability - but its scan API cannot be called remotely, so it runs on the endpoint even when a platform launches it.

7.8

Active Directory evidence

FROM THE FIELD - PRACTITIONER'S NOTE

The tool reports a privileged-group count as a **floor**, not the whole truth - it does not expand nested groups. If “Domain Admins” shows four members but three of them are themselves groups, the real number is larger. Report it as “at least,” and let the attack-path mapping (BloodHound) fill in who can actually reach admin.

On a domain-joined host, the six directory sources populate automatically - no separate script, they switch on by themselves, reading through libraries present on every Windows host (no RSAT). One check to teach specifically: **`MachineAccountQuota`** - an AD default of 10 lets any authenticated user join ten computers to the domain, a known attack primitive.

THE BOUNDARY - CONFIGURATION, NOT ATTACK PATH

This is directory *configuration* evidence: who is in a privileged group, the domain policy, trusts, Kerberos. It does **not** perform attack-path analysis (the BloodHound question of who can *reach* Domain Admin) and does not expand nested groups - so a privileged-group count is a *floor*. A domain controller is a legitimate target and source but is never the scanner/orchestrator.

7.9

Deployment, evidence return, and scale

> **IN PLAIN WORDS**
>
> To run the tool across hundreds of machines, something has to *launch* it on each one and *bring the evidence back*. Clients already own tools that push a script to every machine (ConfigMgr, Intune, Group Policy), so we use those to launch, and the sealed evidence returns over an agreed file share. This section is really about the real-world limits of each channel - size caps, time-outs - and how we stay inside them.

One evidence model, several **execution adapters** tried in order - a pre-engagement decision, not a script's guess.

#	ADAPTER	BEST FIT
1	**Existing client management** (ConfigMgr, Intune, RMM, EDR)	Large estates - client admins target an approved collection; no extra login per endpoint.
2	**WinRM / PowerShell remoting**	Domain members (Kerberos + FQDN); workgroup over validated HTTPS.
3	**Agentless CIM**	A reduced read-only subset; no agent file on the endpoint.
4	**Offline / one-time collector**	Restricted networks; client staff run an approved package, no live session.
5	**Existing security-agent data**	Reuse current Defender VM/Qualys/Rapid7 evidence.
6-7	**Manual exception · Network-only asset**	Isolated endpoints (stays in the coverage ledger); appliances assessed by scan/config export.

Evidence return is designed separately from execution. A console status channel is often too small - ConfigMgr Run Scripts output truncates at 4 KB, Intune scripts have a 30-minute limit - so full evidence is staged and collected separately while the console carries compact status. The package never creates a share or service account on its own. **Scale:** the lab harness is a 1-20 sequential runner, not the 1,000-endpoint scheduler; use the client's fleet management first, or an optional temporary **ansible-core** controller over WinRM/PSRP; prove scale only with a representative pilot (runtime, evidence size, endpoint load, auth-failure behaviour), never a synthetic 1,000-row plan. **Failure modes** are named and answered by an alternate adapter - never by weakening a control (blocked WinRM is not auto-enabled; WDAC blocking the toolkit is recorded and worked with, never evaded).

7.10

The findings register and the GRC layer

FROM THE FIELD - PRACTITIONER'S NOTE

No tool's severity is your severity. A scanner shouting “critical” is an input, not a verdict. Re-score every finding from its CVSS vector against the client's real exposure - a “critical” sitting behind three other controls may be a medium in their world, and saying so, with reasons, is exactly what senior judgement looks like.

A findings register is drafted from the rule results and the outstanding-update result: outstanding updates collapse into *one finding per host*; an undeterminable patch state is a *coverage gap*, never silently dropped; judgement fields are explicit analyst-required markers; and no proof-of-concept text is generated (a PoC is only transcribed from a real capture). **Baseline + overlay:** map checks once to the neutral **NIST SP 800-53** baseline, then apply a per-engagement overlay re-labelling them to the client’s regime (PCI/ISO/SOC 2/HIPAA) - **test once, map many**. Non-public framework IDs are transcribed by an assessor, never guessed.

SEVERITY DISCIPLINE

The rule set assigns **no severity**. Another tool’s severity reflects its assumptions about the environment; severity is a judgement about *this* client’s risk, made by the assessor. No tool’s ratings are inherited as ours.

7.11

Capability and maturity

FROM THE FIELD - PRACTITIONER'S NOTE

Say plainly what you have validated and what you have not. “Built and proven on Windows 11; not yet validated on Server 2016” is a stronger, more credible line than a vague “supports all platforms.” Clients trust the tester who names the edge of their own knowledge - and distrust the one who claims there isn't one.

The toolkit is a **complete pipeline**: it collects 45 sealed evidence sources from a Windows host, evaluates 45 control-mapped configuration rules, determines outstanding Microsoft patches off-host against the free MSRC feed with CISA-KEV prioritisation, and drafts a findings register - all under the collect-vs-judge integrity model. The host side has now been validated end to end in a six-machine virtual lab; what remains is field coverage for the planes that need equipment or a real client environment. A defensible programme states both plainly.

VALIDATED - SIX-MACHINE VIRTUAL LAB: WINDOWS 10, WINDOWS 11, SERVER 2016, 2019, 2022 AND A DOMAIN CONTROLLER	STILL TO PROVE IN THE FIELD
All 45 sources return on every platform; batches sealed; manifests verify; a modified sealed file makes the analyzer refuse the batch	Over-the-air wireless capture (needs a monitor-mode adapter and a person in radio range)
Central collection over WinRM with Kerberos from one admin host to four servers in a single run, with the domain password ages, privileged groups and Kerberos ticket policy read from the domain controller in that run	Wireless-controller review against a real client export (proven so far on a lab TP-Link Omada software controller with a corporate and a guest WLAN)
A live Greenbone Community Edition scan of the server segment (four hosts) exported, imported and analyzed into candidate rows; a controller intake imported and the controller rules evaluated	Two-vantage segmentation across real network segments (the lab has one subnet; the before-and-after used a host firewall block and recorded the blocked path as inconclusive, as designed)
Credential-refused and unreachable targets record Error with the provider message and zero test rows; segmentation before-and-after on one port, with liveness corroborated; a German-language Windows 11 client recorded the account-policy rules as Unknown rather than a wrong value, and the collector now reads that policy through a language-independent API	Wireless host rules on a non-English host that has a wireless adapter (the German test host had none)
Offline update scan against the signed Microsoft catalogue; hardening-audit import; several sealed batches consolidated into one report; 225 automated tests pass, including the OS-update-aware Point and Print check (KB5005652, an absent value judged against the version-specific secure default rather than always failing) and the wireless-controller isolation and VLAN checks stated as configuration review rather than over-the-air enforcement	Estate scale: the launcher runs 1 to 20 hosts sequentially; a client fleet-management adapter is the next build

The lab exposed fourteen defects, every one caught by the evidence rather than by a client: two under non-interactive invocation (script-relative defaults in `param()`; strict-mode failure when the network block is absent), the patch engine's product matching and release ordering, an empty build number on older servers, directory reads through a remote session, an empty privileged-group list, wireless wrongly reported present on servers, a segmentation result that could only ever be inconclusive, and a local account policy that could only be read in English. All are fixed and covered by tests. It is an internal R&D build; production sign-off follows the gates in §7.12. **The Windows capability is proven end to end in the lab; the remaining items need equipment or a client environment, not code.**

REMEMBER - THE HONEST LINE

Capability proven in a lab is not the same as capability proven at a client. Say exactly which platforms and which planes have evidence behind them, and name the ones that still need equipment or a real environment - never just “done.”

7.12

Production and licensing gates

FROM THE FIELD - PRACTITIONER'S NOTE

The code-signing certificate is not optional paperwork - without it, a mature client's application-control (WDAC/AppLocker) simply refuses to run the tool, and you get nothing. Budget the small annual cost before the engagement, not on the client's floor when the collector won't launch.

- **Code trust.** Signed and hash-pinned, integrated with client App Control/WDAC. Execution policy is not a security boundary; Bypass, wildcard `TrustedHosts` and disabling app control are never prerequisites.
- **Evidence.** `Win32_Product` prohibited for routine inventory; hotfix list is inventory only; third-party CVE applicability comes from a maintained engine, never a home-grown match.
- **Lifecycle.** Exact edition/build/LTSC/ESU/support recorded; a generic “Windows 10” label is insufficient; lifecycle is distinct from patch status.
- **Licensing.** Microsoft baselines are the default free reference; CIS content needs SecureSuite; OpenSCAP is not a Windows candidate; the Greenbone Community Feed carries no completeness warranty (record edition/date); Nmap redistribution in a proprietary product needs a licensing decision.
- **Confidentiality.** Client-approved encrypted storage/transfer, explicit retention and destruction; hashes prove later change, not original truthfulness.

THE CODE-SIGNING CERTIFICATE, EXPLAINED

Enterprise deployment tools (Configuration Manager, Intune, Group Policy) and application-control policies (WDAC/AppLocker) trust code by its **digital signature**, not by where it came from. A **code-signing certificate** - from a public CA (roughly US$200-400/year) or an internal enterprise CA - lets the team sign each released script so it runs under the client's own controls *without* a security warning and *without* asking the client to weaken anything (no execution-policy Bypass, no allowlisting an unsigned file). It also hash-pins the exact reviewed version, so a tampered copy will not run. This is the difference between "please disable your security to run our tool" - which we never say - and "our tool is signed; verify the publisher." Buying the certificate and protecting its private key is a business decision that precedes production use.

PART VIII

Engagement management

How an engagement is run, validated, delivered and mapped to the client's compliance world.

8.1

Methodology and senior validation

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> The tool flags; the person decides. Every result gets a second, human read before it becomes a finding - is it real, is it in scope, does it matter for **this** client? That review is not bureaucracy; it is the difference between a report you can defend line by line and a scanner dump with your logo on it.

Two passes (unauthenticated then authenticated, §4.1); segmentation recorded as a reachability matrix (§4.7); configuration reviewed for open rules, exposed management, missing logging, weak settings and unsupported firmware. **Senior validation is a fixed step:** every important finding is confirmed by a senior team member before it enters the report - the tool flags, the person decides. Work follows PTES and NIST SP 800-115; findings are scored with CVSS (from the vector, mechanically).

8.2

Deliverables, scheduled scanning and retest

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> "They said they fixed it" is never "Fixed." On a retest, something is marked Fixed only when you re-ran the exact same test and watched it fail to work now. An email, a ticket marked done, or a screenshot of a config are not evidence - re-test it, or write "Not retested." Clients respect the discipline; proving the fix is the entire point of a retest.

The deliverable is a report with an **executive summary** for management and **detailed findings** for the technical team - each with affected systems, risk, evidence, and a prioritised fix. Scheduled scanning adds a regular **change summary** between formal windows. A **retest** is offered after fixes are applied -

and a finding is only marked Fixed on direct retest evidence, never on correspondence, an implementation statement, an inaccessible endpoint, or a risk acceptance.

8.3

What we need from the client before we start

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> Chase the boring prerequisites early - read-only credentials, the source-IP allow-list, config exports, blackout windows. These are what actually delay engagements, not the testing. Send the request list the day the ROE is signed, and keep following up until every item has landed.

Nothing is scanned until these are in place:

- Signed authorisation naming the exact systems in scope.
- In-scope asset list: domains/IPs for external, internal ranges for internal.
- Agreed testing window and any blackout periods.
- For internal work, VPN access and jump-box details.
- Read-only credentials for the authenticated pass, where in scope.
- Configuration exports if firewall/system/endpoint review is included.
- A notification destination and an emergency contact who can stop the work at any time.

8.4

Compliance mapping - one baseline, many client views

> **IN PLAIN WORDS**
> One test result can satisfy many rulebooks at once. We check a setting a single time against our master list (NIST 800-53), then *crosswalk* it - map it across - to whatever the client is actually measured by (PCI, ISO 27001, SOC 2). That saves testing the same thing five times. The caution: these maps are "close," not exact, so a human reviews the edge cases before anything goes into a compliance report.

Findings and control coverage map *once* to **NIST SP 800-53 Rev 5** (the richest superset, with published crosswalks), then overlay each client's framework rather than re-mapping from scratch: 800-53 → SOC 2 TSC, 800-53 → ISO 27001 Annex A, 800-53 → CSF function; PCI DSS requirement numbers where a cardholder environment is in scope; GDPR Art. 32 where EU personal data is exposed; content-protection guidelines for media environments. Emit coverage in an OSCAL-compatible shape so the

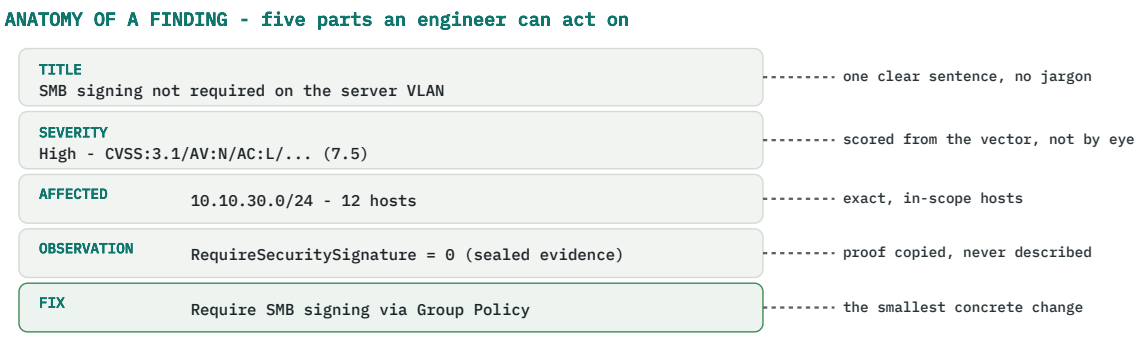
mapping is reusable. Crosswalks are approximate - a mapping is “related,” rarely 1:1 - so edge cases are reviewed, not trusted blindly.

8.5

Writing a finding

FROM THE FIELD - PRACTITIONER'S NOTE

A finding that actually gets fixed has four parts a busy engineer can act on in five minutes: **what** is wrong, **where** (the exact host or URL), **proof** (the real request and response, copied - never described from memory), and the **smallest change** that fixes it. Skip the essay. If they can't act on it without emailing you a question, it won't get fixed - and an unfixed finding helped no one.



A finding is a work order, not an essay. These five parts let a busy engineer act in five minutes - and the plain title doubles as the sentence an executive reads.

A finding is only as good as its write-up. Each carries the same anatomy, and each element has a rule behind it learned the hard way.

ELEMENT	THE RULE
Title	Specific and plain, never sensational - the weakness, not a headline.
Severity + CVSS	Computed mechanically from the vector (Roundup), never eyeballed; the vector travels with the number.
Affected assets	In-scope hosts only, taken from evidence - every asset traces to the ROE.
Description	What the weakness is, in the client’s language; observed vs inferred vs not-validated kept distinct.
Evidence / PoC	**Transcribed from a real capture, never reconstructed.** Every request line, status code and body fragment is copied from the archive; if the capture is not there, the PoC is not written.
Impact	The real-world business consequence - what an attacker gains - not a restatement of the CVSS.

ELEMENT	THE RULE
Remediation	Actionable and prioritised; a fix the client can actually execute.
References	CVE / CWE class / the mapped control.

NON-NEGOTIABLES

A rejected attempt is not a bypass; an HTTP error is not remediation. Dates are the signed ROE dates, stated plainly. A finding is marked *Fixed* only on direct retest evidence. And nothing internal - tooling names, model names, coverage counts, analyst paths - ever appears in the client document.

8.6

Pre-engagement operational checklist

FROM THE FIELD - PRACTITIONER'S NOTE

Treat this checklist as a hard go/no-go gate - not a formality. Do not send a single packet until every box is ticked: scope signed, source IP allow-listed, blackout windows agreed, emergency contact reachable, tool feeds current. Five minutes here prevents the two most common disasters in this whole book - testing out of scope, and taking something down at the worst possible moment.

Nothing is scanned until every box is ticked - the go/no-go gate before the first packet.

☐ Signed authorisation & scope confirmed; in-scope assets/CIDRs loaded	☐ Testing window agreed; blackout periods recorded
☐ Emergency-stop contact confirmed and reachable	☐ the testing source IP shared and whitelisted by the client
☐ VPN access and jump-box details verified (internal work)	☐ Read-only credentials validated on a small pilot
☐ Account-lockout policy known before any auth attempt	☐ No-scan / fragile / OT assets flagged with a safe profile
☐ Evidence storage encrypted; retention/destruction agreed	☐ Tooling and feeds updated; feed edition/date recorded
☐ A positive-control target identified (proves tooling works)	☐ IPv6 explicitly included or excluded in scope

PART IX

Worked scenarios

Five anonymised engagements that show the method end to end. No real client is named; these are representative composites for teaching, and any specifics stay confidential.

9.1

External web + API - a multi-role SaaS portal

Context	A cloud-hosted analytics portal with a browser UI and an admin Web API; seven user roles enforced by JWT claims.
Scope	Two test/dev hosts only; production out of scope; signed ROE supplying credentials for each role.
Approach	Grey-box, manual. Map endpoints from the UI and the API spec; build a role×endpoint matrix; replay each role against another's objects and functions; tamper JWTs.
Findings	BOLA (one tenant reads another's records by object ID), BFLA (a low role reaching an admin verb), a JWT accepting an unverified signature, permissive CORS.
Deliverable	Per-finding PoC transcribed from Burp/curl; CVSS from the vector; remediation per endpoint. Tools named to the client: Burp, curl, Python.

TEACHES

Authorisation is the whole game in APIs, and a scanner finds none of it - the PoC is "same request, victim's ID, attacker's token → 200."

9.2

Internal network + AD - a corporate HQ

TEACHES

Internal AD work is rarely one exploit - it is a **chain** of small misconfigurations (LLMNR on, SMB signing off, a reused local-admin password, one over-privileged account). The value is naming the chain and the single settings that break each link.

Context	A Windows domain of ~600 hosts across a workstation VLAN and a server VLAN; VPN + jump-box access.
Scope	Internal assessment, one read-only domain account, one workstation-VLAN position.
Approach	Two-pass. Unauthenticated discovery + SMB-signing sweep; then credentialed host collection with the toolkit across a piloted set; directory config sources on the domain.
Findings	LLMNR enabled (relay exposure), SMB signing not required on servers, WDigest caching on older hosts, no LAPS (shared local-admin), MachineAccountQuota at 10, hundreds of missing OS patches on a subset.
Deliverable	Sealed evidence, one patch finding per host, coverage measured against the approved inventory, controls mapped to 800-53.

9.3

Segmentation validation - a cardholder (PCI) environment

Context	A retailer needing evidence that its cardholder data environment (CDE) is isolated from the corporate network for PCI DSS.
Scope	Reachability testing from named source zones to the CDE; no exploitation.
Approach	From a representative position in each source zone, test policy-relevant ports to CDE targets (`-Pn`), with a positive control; build the N×N reachability matrix.
Findings	A management VLAN with an unexpected open path into the CDE; a “blocked” result correctly held Inconclusive until a known-up port confirmed the drop.
Deliverable	The matrix mapped to the intended policy and PCI requirement; each cell states observed vs inconclusive.

TEACHES

Source position is everything, and a no-connect is Inconclusive, not proof of isolation.

9.4

Cloud posture - an Azure / Entra tenant

TEACHES

Cloud is an identity and configuration audit, not an attack on the provider. The findings are almost always an over-privileged role, an open storage bucket, or admin access without extra login checks - all read from the tenant's own settings, never by breaking in.

Context	A SaaS company’s Azure subscription and Entra ID directory.
Scope	Read-only audit role; configuration and identity review; no exploitation, provider RoE observed.
Approach	Enumerate with ScoutSuite/Prowler and ROADrecon; benchmark against CIS Azure; review privileged roles, Conditional Access and app registrations.
Findings	A public storage blob, over-privileged service principals, Global Admins without phishing-resistant MFA, legacy auth allowed, disabled activity logging.
Deliverable	Posture report mapped to CIS Azure and 800-53; no provider infrastructure touched.

9.5

Wireless - a branch office

Context	A branch with a corporate WPA2-Enterprise SSID and a guest SSID.
Scope	On-site; signed ROE naming SSIDs/BSSIDs and the location; a technician physically present with a dual-band monitor-mode adapter.
Approach	Passive survey across all bands; check PMF, WPS and transition mode; test enterprise client certificate validation; join guest and test isolation.
Findings	Clients not validating the RADIUS certificate (crackable MSCHAPv2), WPS enabled on one AP, guest SSID routing to an internal print server.
Deliverable	Findings framed as configuration/entropy weaknesses; the guest-isolation gap as the highest severity.

TEACHES

The finding is almost always config or password entropy - “no server-cert validation,” “WPS on,” “flat guest VLAN” - not “the encryption is broken.”

PART X

Desk reference

Quick-lookup material to keep open during an engagement.

10.1

Ports and protocols

PORT	PROTOCOL	NOTES
22	SSH	Encrypted shell; keyless/weak auth, exposed management.
53	DNS	Zone-transfer leaks, poisoning, tunnelling.
67/68	DHCP	Rogue DHCP; DHCPv6 → mitm6.
80 / 443	HTTP / HTTPS	Web/API surface; on 443 also TLS quality.
88	Kerberos	Domain auth; needs FQDN + DNS.
135 / 139	RPC / NetBIOS	Legacy Windows; NetBIOS ties to name poisoning.
161	SNMP (UDP)	Default community `public` leaks config.
389 / 636	LDAP / LDAPS	Directory queries; 389 cleartext, anonymous binds.
445	SMB	SMBv1 and unsigned sessions are the classic internal risks.
3389	RDP	Needs NLA; exposure + weak auth = entry.
5985 / 5986	WinRM	The toolkit's transport; watch cleartext/basic auth.

10.2

Tool matrix - open-source vs commercial

PURPOSE	OPEN-SOURCE (PRIMARY)	COMMERCIAL OPTION
Discovery	nmap, masscan, arp-scan, fping	-

PURPOSE	OPEN-SOURCE (PRIMARY)	COMMERCIAL OPTION
Network vuln scan	Greenbone / OpenVAS	Nessus Pro (~$3,990-4,790/yr), Tenable.sc, Qualys, Rapid7
Enumeration	netexec, enum4linux-ng, ldap utils, impacket	-
Web / API	curl, Python, testssl.sh, ffuf/gobuster (plus ZAP, nuclei)	Burp Suite Pro (~$449/yr)
Active Directory	bloodhound, kerbrute, impacket, certipy	-
Wireless	aircrack-ng, hcxtools, hashcat, kismet, eaphammer	- (monitor-mode adapter required)
Cloud	ScoutSuite, Prowler, ROADrecon, ScubaGear	-
Config / passive	Batfish, Zeek, Lynis, OpenSCAP	Nipper Studio
Host plane (host toolkit)	the collector + analyzer (+ MSRC CVRF, CISA KEV - free)	Microsoft Defender VM (reuse client's)

10.3

Standards crosswalk

NEED	STANDARD
Which technique, assessment or exploitation?	NIST SP 800-115 (§3 review · §4 analysis · §5 validation)
Engagement shape	PTES (seven phases)
Web/API method & checklist	OWASP WSTG · ASVS · Top 10 · API Top 10
Attacker framing	MITRE ATT&CK
Baseline ("what good looks like")	NIST SP 800-53 Rev 5
Hardening settings	CIS Benchmarks (licence permitting)
Executive roll-up	NIST CSF 2.0
Client regimes	ISO 27001 · SOC 2 · PCI DSS · GDPR

NEED	STANDARD
Machine-readable evidence	NIST OSCAL

10.4

Glossary

Switch / Router	Connects devices within one network (L2, by MAC) / connects different networks (L3, by IP). segmentation vs reachability
VLAN / zone	A logical division separating traffic; the basis of segmentation. guest vs corporate
MAC / IP · CIDR · NAT	Local vs routable address; range notation (`/24`); many private hosts sharing one public address. IP ≠ identity
TCP 3-way handshake	SYN → SYN-ACK → ACK; port scanning manipulates it. open/closed/filtered
Jump-box / bastion	A single hardened host routed through for controlled, logged in-network presence. internal delivery
VPN tunnel	Encrypted path making two ends behave as one local network; how internal testing reaches private ranges. Part III
Two-pass method	Unauthenticated (attacker view) then authenticated (read-only creds, exact state). §4.1
NIST SP 800-115 / PTES	The technique guide (assessment vs exploitation line) / the seven-phase lifecycle. methodology
NIST 800-53 / CIS / ISO 27001	Control catalogue (baseline) / hardening settings / client audit regime. what a finding violates
CVE / CVSS / EPSS / KEV / CWE	Flaw ID / severity vector / exploit probability / exploited-now list / weakness class. scoring stack
Credentialed scan	Logs in and reads the truth rather than guessing; verify auth actually succeeded. accuracy
WinRM / Kerberos / NTLM	Windows remote mgmt (5985/6) / domain ticket auth (needs FQDN) / older relay-prone auth. Windows plumbing
SMB signing	Session integrity; required (not merely enabled) blocks relay. SMB02/03

LLMNR / NBT-NS · Responder	Legacy name fallbacks, poisonable to capture credentials. NET01/02
Kerberoasting / AS-REP / PtH / DCSync	Offline-crack SPN tickets / pre-auth-disabled accounts / auth with a hash / replicate secrets from a DC. AD primitives
LSA / WDigest / LAPS / Credential Guard	Credential store / cleartext-cache legacy protocol / per-host admin-password mgmt / VBS credential isolation. CRED rules
MachineAccountQuota	AD default letting any user join 10 machines. domainpolicy
Servicing level (UBR) / MSRC CVRF	Build+revision = exact patch state / Microsoft's free per-CVE fixed-build feed. patch engine
SHA-256	One-way file fingerprint; the evidence seal. integrity
802.11 / SSID / BSSID / PMF	Wi-Fi standard / network name / AP-radio MAC / 802.11w protected management frames (defeats deauth). wireless basics
4-way handshake / PMKID	The PSK key exchange (an offline oracle) / a handshake-free value some APs volunteer. capture then crack
WPA2 / WPA3-SAE / 802.1X-EAP	Shared-key AES / current PAKE (no offline crack) / per-user enterprise (hinges on cert validation). encryption
WPS / Pixie-Dust	The PIN feature; recovers the PSK regardless of its strength - check it's disabled. one-shot compromise
Monitor-mode adapter	Chipset+driver (not brand) that hears all frames and injects; RF work needs it, on-site. the field card
BOLA / BFLA	Reach another user's object / a function above your role - the dominant API risks, manual to find. API Top 10
Shared responsibility	Provider secures the cloud; customer secures what's in it - cloud assessment targets the customer's half. §6.3
Baseline + overlay	Map once to 800-53, overlay each client's framework via crosswalks. test once, map many
Batfish / Zeek	Model device configs (static) / parse traffic to structured logs (passive). quiet planes
Non-interactive	Started by a deployment tool, not a person - the path that matters, where subtle defects hide. how it runs
Black / grey / white-box	No prior knowledge / partial knowledge or creds / full knowledge. Grey-box is the usual choice. §2.6

Threat modeling	Prioritising testing by asset value and plausible threat before technical work. PTES phase 3
Phishing / social engineering	Testing the human layer; §5, separately authorised, never bolted onto a technical engagement. §2.6
IPv6 / SLAAC / dual-stack	128-bit addressing; auto-config; running v4+v6 at once leaves v6 an often-unmonitored surface. §1.7
DMZ / zero-trust	Isolated buffer for public services / never trust by network location - verify every request. architecture
Pivoting / tunnelling	Using a foothold or jump-box to reach otherwise-unroutable segments (proxychains, chisel, ligolo). §4.11
Password spraying	One common password against many accounts, slowly - evades lockout; know the policy first. §4.11
Null session / anonymous bind	Unauthenticated SMB/LDAP that still leaks structure - itself a finding. §4.10
SPN	“Service Principal Name” - the label that ties a service (like a database) to the account running it. Asking for its ticket and cracking it offline is Kerberoasting. AD service accounts
gMSA	A “group managed service account” - Windows generates and rotates a long random password for a service, so it can't be Kerberoasted. The fix for weak service passwords. remediation
EPA / channel binding · coercion · PetitPotam	Extra login checks that tie an authentication to the exact connection, defeating relay / an attacker *forcing* a server to log in to them (PetitPotam is one such trick). relay defence & abuse
golden / silver ticket · krbtgt · Pass-the-Ticket	Forged Kerberos tickets: a “golden” one (from the domain's master `krbtgt` key) impersonates anyone; a “silver” one targets a single service; Pass-the-Ticket reuses a stolen ticket. Total-compromise territory. domain dominance
ADCS / ESC1-ESC16 · certipy	Active Directory Certificate Services - the in-house system that issues login certificates. Misconfigured templates (the “ESC” issues) let a low user mint an admin certificate; `certipy` is the tool that finds them. privilege escalation
BloodHound / tier-0	A tool that maps “who can reach admin” as a graph. “Tier-0” means the crown-jewel systems (domain controllers) that must never be reachable from ordinary machines. attack-path mapping
mitm6 / DHCPv6 · RA Guard	An attack that answers a Windows machine's automatic IPv6 requests to become its DNS, then relays logins. The switch-level fix is RA Guard / DHCPv6 Guard - *not* disabling IPv6. the unwatched second stack

PBKDF2	The slow “stretching” maths that turns a Wi-Fi password into its key; WPA2 uses a weak 4,096 rounds with the network name as salt, which is why a strong passphrase matters. why WPA2 is crackable
RSN IE / AKM · hashcat 22000	The chunk of a Wi-Fi beacon that advertises its security (cipher and login method) - you read it passively. `hashcat` mode 22000 is the setting that cracks captured WPA handshakes/PMKIDs. wireless fingerprint & crack
802.1X-EAP / RADIUS · PEAP-MSCHAPv2 · EAP-TLS	Enterprise Wi-Fi where each user logs in to a central server (RADIUS). PEAP-MSCHAPv2 is crackable if the client doesn't check the server's certificate; EAP-TLS (certificates both ways) is the safe option. the enterprise trap
WPA3-SAE transition mode · Dragonblood (a WPA3-SAE flaw) · KRACK (a WPA2 handshake flaw)	WPA3's modern login (SAE); “transition mode” also accepts WPA2 and can be forced down to it. Dragonblood/KRACK are named implementation flaws (mostly patched), not password weaknesses. config exposure
JWT · alg:none · RS256→HS256	A signed login token an app trusts (JSON Web Token). Classic bugs: getting the app to accept an *unsigned* token (`alg:none`) or one signed with the public key it published (RS256→HS256 confusion). API auth
IDOR / SSRF / XXE	Reach another user's data by changing an ID (IDOR/BOLA) / make the server fetch a URL *you* choose (SSRF) / abuse XML parsing to read files (XXE). Common high-impact web bugs. OWASP
CORS	The browser rule for which other sites may read a response. Set too loosely, it lets a malicious site read a victim's data through their logged-in session. web misconfig
IMDSv1 / IMDSv2 · service principal · PIM / Conditional Access	The cloud metadata service a server queries for its own keys (v2 is the hardened form) / a non-human cloud identity for an app / just-in-time admin and login conditions in Azure. cloud identity
MASVS / MASTG	OWASP's mobile security standard and testing guide - the mobile equivalent of the web ASVS/WSTG. mobile
NetFlow / IPFIX · SPAN / TAP	Summaries of “who talked to whom” a network keeps (NetFlow) / ways to copy live traffic to a sensor (a switch SPAN port or a physical TAP). Passive evidence sources. §4.8
CIM / PSRP · second-hop	Windows management interfaces the collector can use (CIM/WMI, PowerShell Remoting). The “second-hop” problem is that a remote login can't automatically re-use its credentials to jump onward. collector transport
RID · PPL (RunAsPPL) · PrintNightmare	The numeric tail of a Windows account ID (survives a rename) / a protection that stops tools reading the credential store / a print-spooler flaw class fixed by restricting driver installs. host rules

WDAC / AppLocker · code-signing certificate · OSCAL / POA&M	Windows controls that only allow approved programs to run (our tool needs a signing certificate to pass them) / a machine-readable format for controls and a "plan of action & milestones" for open gaps. running at mature clients
ADCS · AKM · OWE	Active Directory Certificate Services, a Windows PKI often abused for privilege escalation / the authentication-and-key-management method a Wi-Fi client negotiates / Opportunistic Wireless Encryption, which encrypts an otherwise-open Wi-Fi network.
Roundup (CVSS)	CVSS always rounds a score *up* to one decimal - score it mechanically from the vector, never by eye. §4.6

10.5

The first hour - a command reference

The exact commands a junior reaches for in the first hour of an internal engagement, in order. Every one is *identification* - safe in a vulnerability-assessment scope; nothing here exploits. Run them quietly from the jump-box, and save everything (`-oA`) so it becomes re-checkable evidence.

```
# 1 · Live hosts on this subnet - ARP, which cannot be firewalled at Layer 2
arp-scan --localnet
nmap -sn 10.10.10.0/24

# 2 · The four-port "is this soft?" check across the whole subnet
nmap -sS -sU -p T:88,445,3389,U:161 --open 10.10.10.0/24   # 161 is UDP: without -sU it is

# 3 · Full TCP on a target: all ports, versions, default scripts, save every format
nmap -sS -p- -sV -sC -T4 -oA scans/host1 10.10.10.5

# 4 · Targeted UDP on the high-value services (never scan all UDP - too slow)
nmap -sU -p 53,161,123,500 --open -T4 10.10.10.5

# 5 · Is SMB signing required? (the relay-exposure question)
nmap -p 445 --script smb2-security-mode 10.10.10.0/24
netexec smb 10.10.10.0/24            # shares, signing, null sessions

# 6 · Directory: does it allow an anonymous (no-login) read?
ldapsearch -x -H ldap://10.10.10.10 -s base namingcontexts

# 7 · The host plane: credentialed read over WinRM (CONFIRM auth actually succeeded)
#     run the collector with the client's read-only admin account
```

HOW TO USE IT

Steps 1-2 tell you the **shape** of the network and whether it is soft; the rest is depth on what you found. Always confirm the credentialed read in step 7 actually authenticated (§4.1) before you trust a quiet result - an empty report from a failed login looks exactly like “all clear.”

10.6

Common findings and their one-line fixes

The recurring findings on almost every internal engagement, with the severity they usually carry and the single change that closes each. Severity is always confirmed from the CVSS vector against the client's real exposure (§4.6) - treat this as a memory aid, not a scoreboard.

FINDING	USUALLY	ONE-LINE FIX
Missing security updates (on the KEV list)	**Critical**	Patch now; prioritise KEV-listed CVEs first.
SMB signing not *required*	High	Require SMB signing by Group Policy.
WDigest caching cleartext passwords	High	Set `UseLogonCredential = 0`; patch legacy OS.
No LAPS / one shared local-admin password	High	Deploy LAPS - a unique password per host.
Exposed management (RDP/SSH/DB/iLO)	High	Restrict to a management VLAN or VPN only.
Kerberoastable service accounts	High	gMSA or long random passwords; AES, not RC4.
Flat network - no segmentation	High	VLANs with firewall rules between zones.
Guest Wi-Fi not isolated from corporate	High	Separate guest VLAN; enable client isolation.
Over-privileged cloud role (`*:*`)	High	Scope the role to least privilege.
Broken access control (IDOR / BOLA)	High	Enforce authorization server-side, per object.
LLMNR / NBT-NS enabled	Medium	Disable via GPO; remove WPAD.
Weak/legacy TLS or invalid certificate	Medium	Enforce TLS 1.2+; install a valid certificate.
Default SNMP community `public`	Medium	Change the community; restrict SNMP access.
Anonymous LDAP bind / SMB null session	Medium	Require authentication; disable anonymous access.
`MachineAccountQuota = 10`	Medium	Set the quota to 0.
No 802.11w (PMF) on Wi-Fi	Medium	Enable PMF; move toward WPA3.

HOW TO USE IT

When you see one of these, you already know the likely severity and the fix - which means you can write the finding (§8.5) in minutes and give the client a clear remediation. But always re-confirm the severity for **their** environment; a "High" behind other controls can be a Medium in context.

10.7

Reading a CVSS vector at a glance

A CVSS score is not typed by hand - it is calculated from a *vector string*, and the score always rounds up (§4.6). Learn to read the string and you can sanity-check any tool's number in seconds. Each letter is one question about how the vulnerability behaves.

METRIC	VALUES	PLAIN MEANING
`AV` Attack Vector	N · A · L · P	Where from: **N**etwork, **A**djacent, **L**ocal, **P**hysical. Network is worst.
`AC` Attack Complexity	L · H	How hard to pull off: **L**ow or **H**igh.
`PR` Privileges Required	N · L · H	What access you need first: **N**one, **L**ow, **H**igh.
`UI` User Interaction	N · R	Does a victim have to do something? **N**one or **R**equired.
`S` Scope	U · C	Can it break out to other systems? **U**nchanged or **C**hanged.
`C / I / A` Impact	N · L · H	Damage to **C**onfidentiality, **I**ntegrity, **A**vailability.

Worked read of `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` = **9.8, Critical**: reachable *over the network*, *easy*, needs *no login* and *no user action*, with *high* impact to confidentiality, integrity and availability all at once. You can see it is severe without a calculator - which is exactly the point of learning to read the string.

10.8

The engagement at a glance

The whole lifecycle on one line each - what you do at every phase, the tools you reach for, and what comes out the other end. If you can hold this table in your head, you can run an engagement.

PHASE	WHAT YOU DO	KEY TOOLS	OUTPUT
Pre-engagement	Scope, rules of engagement, prerequisites (§8.6)	-	Signed ROE

PHASE	WHAT YOU DO	KEY TOOLS	OUTPUT
Recon (external)	Map the public footprint, touch nothing (§4.2)	whois, crt.sh, Shodan	Target list
Discovery	Find live hosts in scope (§4.4)	nmap -sn, arp-scan	Host list
Enumeration	Open ports, services, versions (§4.10)	nmap -sV, netexec	Service map
Identification	Find and confirm weaknesses (§4.1)	Greenbone, the toolkit	Raw findings
Validation	Senior review, score from the vector (§8.1)	manual, CVSS	Verified findings
Reporting	Write, prioritise, two audiences (§11.6)	-	Exec + technical report
Retest	Prove each fix actually works (§8.2)	re-run the exact tests	Closed findings

THE THROUGH-LINE

Notice the shape: gather broadly, narrow to what is real, let a human confirm and score it, explain it in the client's language, and prove the fix. Every tool and technique in this handbook hangs off one of these eight steps.

PART XI

Lessons from the field - ten years of network testing

Everything before this Part is method and fact. This Part is judgement - the things experience teaches that no checklist can. Read it once now, and again after your first few real engagements; it will read very differently the second time. None of it is optional: most engagements are won or lost on these habits, not on tool skill.

11.1

Scoping without regret

An engagement is won or lost before you send a single packet. A vague scope wastes days and creates real legal risk. When a client says “just test everything,” that is not a scope - it is a trap, because “everything” always turns out to include a system you were never allowed to touch. Pin the boundaries down, in writing, before you agree to start.

> **IN PLAIN WORDS**
> Scope is the fence around the job. Inside the fence you may test; one step outside is unauthorised access - a crime, not initiative. Your entire authority to touch someone else's systems comes from this one signed document, so it has to be exact.

01 **Exact targets**

The precise IP ranges, domains and URLs - not “the network.” A range you cannot read back to the client is not scoped.

02 **Explicit exclusions**

Which systems, accounts and techniques are off-limits. Fragile kit, production databases, anything third-party-hosted.

03 **Window and time zone**

When testing may run, in whose clock. “Business hours” without a time zone is how you scan at 2 a.m. and wake the on-call team.

04 **Emergency stop**

A named person and number to reach instantly if something breaks, and the agreement that you will call.

05 **What “done” looks like**

The deliverable and the definition of success, agreed up front, so there is no argument at the end.

11.2

The pre-engagement conversation

Before any testing, have a real conversation - not just a signed form. Ten minutes here saves a day of firefighting later, and it is where you learn what the engagement is actually about.

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> Ask four questions and listen. “What are you actually worried about?” - the answer is where your value is. “What breaks if I touch it?” - that is your do-not-poke list. “What happened in past tests?” - you learn the sore spots and the politics. And “who needs to be told we're testing?” - so your scan is treated as a scheduled test, not a live breach that triggers an incident and a very awkward phone call.

11.3

First, do no harm

The fastest way to lose a client forever is to knock something over. A finding you didn't get is recoverable; an outage you caused is not. Default to the gentlest technique that answers the question, and earn the right to be louder only with written permission.

- Start slow and quiet; raise intensity only as needed, never by reflex.
- Never run “denial-of-service,” “brute” or “intrusive” tooling without an explicit written yes - some of it crashes services by design.
- Warn the client's security team before you begin, so your test isn't fought as a real attack.
- On fragile equipment - factory/OT gear, medical devices, old printers - *look, don't poke*. Passive only.
- Keep the emergency-stop contact open the whole time, and actually use it the moment something looks wrong.

11.4

Reading between the scan lines

The scanner is a junior assistant, not the tester. It finds the obvious and misses the interesting: the two low-risk issues that *chain* into a serious one, the business-logic flaw no tool understands, the dull "informational" note that is really the way in. Your job begins where the scan ends.

> **IN PLAIN WORDS**
> A tool tells you *what* is there. Only a person can tell you *so what* - what an attacker could actually do with it on this specific network. Verify every finding by hand, throw out the false alarms without guilt, and for each real one ask "what does this let someone do?" That single question is the entire craft.

11.5

Prioritising what actually matters

A client handed 300 findings fixes none of them. Your value is telling them the handful that matter this week, in their language. Order the work by real-world risk, not raw severity (see §4.6): anything being exploited in the wild first, then high-likelihood and high-impact, then everything else on a sensible schedule.

> **FROM THE FIELD - PRACTITIONER'S NOTE**
> Tie every top finding to **their** business, not to a score. "An attacker on your guest Wi-Fi could reach the customer database" gets fixed on Monday; "SMB signing is not required (CVSS 7.5)" gets filed and forgotten. Same finding - one sentence decides whether it is ever fixed.

11.6

Writing findings that get fixed

A report is not written for another hacker to admire; it is a stack of work orders for a tired engineer at 5 p.m. Write for that person. Two layers cover every reader: one plain sentence an executive understands, and the technical detail an engineer needs to make the change.

---

01 **What is wrong**

One clear sentence, no jargon in the headline.

02 **Where**

The exact host, URL or account - so no one has to hunt for it.

03 **Proof**

The real request and response, *copied* from the evidence, never described from memory. If it isn't in the evidence, it doesn't go in the report.

04 **The smallest fix**

The specific, cheapest change that closes it - and a link to the vendor's own guidance where one exists.

11.7

Talking to clients and executives

You will spend nearly as much time explaining as testing. Executives care about risk and money, not packets. Translate everything into what could happen to the business and what it would cost - that is the language decisions are made in.

- Lead with impact, not mechanism: what an attacker could reach, and why that hurts.
- Never use fear to upsell. Overstating risk gets you one contract and no second one.
- Say plainly what you did *not* test, and why. Honesty about gaps is what makes the rest believable.
- Be calm about bad news. You are the person who found the problem, not the person who caused it - act like it.

WHY IT MATTERS

The trust you build in the debrief is why they call you back next year. Repeat clients, not one-off heroics, are what a career is made of.

11.8

The mistakes everyone makes once

Every tester collects these scars. The goal is to make each mistake once, write it down, and never repeat it - and, better, to borrow the ones below so you skip them entirely.

LEARN THESE BEFORE THEY TEACH YOU

Scanning out of scope because you found more systems. **Trusting a version banner** and reporting a patched bug. **Calling a silent port "blocked"** with no positive control to prove it. **A careless password spray** that locks out the client's staff. **Pasting a scanner's severity** as your own instead of scoring it. **Reporting a rejected attempt as a bypass** - a failed attack is not a finding. **Losing the evidence** that proves a finding, so you can't defend it. **Forgetting to warn the security team**, and becoming the incident. Each one is avoidable, and each one is embarrassing exactly once.

11.9

How to keep learning - a skill ladder

Nobody starts expert. The path is well-worn, and the fundamentals you learn early (Part I) outlast every tool and trend. Climb it in order; skipping rungs is how people end up running tools they don't understand.

01 **Fundamentals**

Own the networking and methodology in this handbook cold - Part I especially. Everything else is built on it.

02 **Build a home lab**

A few virtual machines and a test Windows domain. Break things where breaking is free; you learn ten times faster with your hands on it.

03 **Practise on real methodology**

Structured labs and capture-the-flag exercises that follow the phases in Part II, not random tool-running.

04 **Shadow senior work**

Watch how an experienced tester scopes, decides and writes. Judgement is caught more than taught.

05 **Own small engagements end to end**

Scope, test, validate, write, present. Owning the whole loop is where a tester is actually made.

06

Specialise

Then go deep in one area - Active Directory, cloud, wireless or web. Certificates open doors; a lab and real curiosity make the tester.

THE ONE THAT NEVER EXPIRES

The field changes every year - new tools, new attacks, new clouds. The fundamentals in Part I do not. When something new confuses you, it almost always reduces to a basic you already know: an address, a port, a login, a trust. Go back to the base and it gets simple again.

PART XII

A first engagement, walked through

The whole handbook in one week. This is a narrated walk through a small internal network assessment - nothing invented that you haven't already met in the earlier Parts. Watch how little of it is clever exploitation and how much is scoping, reading the network, checking boring settings, and explaining risk in the client's language. Section references point back to where each step is taught.

12.1

The setup - the scope call

Your first internal assessment. A 200-person company wants to know how far an attacker could get from a laptop plugged into their office. On the scope call you nail down the essentials (§11.1): the exact subnets (`10.10.0.0/16`), what is off-limits (the payroll server, the CCTV VLAN), the window (this week, business hours, *their* time zone), and an emergency contact. You ask the four questions (§11.2) - they are worried about ransomware spreading, a past test "found nothing," and yes, they will warn their security team. You send your jump-box's IP to be allow-listed (§3.2). Nothing starts until the rules of engagement are signed (§2.5).

12.2

Getting in and looking around

Day one. You connect to the jump-box on their user VLAN (§3.3) and start quietly. First, who is here: `arp-scan` and `nmap -sn` map the live hosts on the local subnet (§4.4) - no need to be loud, because ARP cannot be firewalled at Layer 2 (§1.4). You spot a domain controller (port 88 answering, §1.5), a dozen servers, and a flat network where the workstation VLAN can reach the server VLAN - a segmentation smell you note for later (§4.7). Then the four-port glance (§1.5): `445` is open with signing not required across the board. That one line already tells you the network is soft.

12.3

Running the toolkit and reading results

The client has given you read-only admin credentials (§8.3). You point the collector at the in-scope hosts over WinRM (§7.3), *confirm authentication actually succeeded* on each one (§4.1 - the empty-report trap), and let it read the real state: patch level, configuration, accounts (§7.4). It seals the evidence

(§7.5), and the analyzer scores it against the 37 rules (§7.6) and Microsoft's patch feed (§7.7). Out come real findings: a dozen servers months behind on patches (two on the KEV list), SMB signing not required, and WDigest still caching cleartext passwords on the older boxes. Remember - a “Fail” is not yet a finding; you review each against the client's own baseline before it counts.

12.4

One finding, start to finish

Take the SMB-signing result. *What it is* (§4.3): signing is not required, so an attacker who poisons name resolution can relay a login to another server without cracking anything. *Where*: the whole server VLAN. *Proof*: you do **not** run the relay - that is exploitation, and it is out of scope - you record the exact setting from the sealed evidence and name the enabling condition. *The fix*: require SMB signing by Group Policy. You score it mechanically from the CVSS vector (§4.6, Roundup - never by eye), and because it pairs with the flat network you rank it high. You write it in four parts (§11.6): what, where, proof, smallest fix - one sentence an executive understands, the detail an engineer needs.

12.5

The debrief

You prioritise for them (§11.5): “An attacker on any user PC could reach and take over your servers - here are the five changes that break that chain this week.” You tie it to their actual fear: ransomware spreads through exactly this path. You say plainly what you did *not* test (the CCTV VLAN was out of scope). You do not oversell. They fix the top five; you retest and confirm each one actually fails now (§8.2), not because they said so. And they book you again next year.

> **THE WHOLE HANDBOOK IN ONE WEEK**
> Notice what carried this engagement: not a clever exploit, but scoping, reading the network, verifying boring settings, and explaining the risk in the client's language. Master those and the tools become details. That is what this handbook is for.

12.6

When something looked wrong

Halfway through, a server stopped responding and your stomach drops - did you break it? You check calmly (§11.3): your scan was not touching that host at that moment, and the emergency contact confirms it was a scheduled reboot. Panic over. But notice why it stayed calm: because you had told their security team you were testing (§11.2), they never blamed your traffic; and because you had an

emergency contact, you got an answer in two minutes. Without those two habits, a routine reboot becomes an hour of accusations and a damaged relationship.

12.7

What the report looked like

The deliverable was two things, not one. A one-page executive summary - “a laptop on your user network could take over your servers; five fixes close the main path” - and a detailed section with each finding written in the four parts (§11.6). It named only the real tools you drove by hand, never a scanner that ran unattended (§8.5). Every claim was backed by sealed, hashed evidence (§7.5), so anything could be shown on request. Nothing dramatic, nothing overstated - which is precisely why they believed it and acted on it.

WHERE TO GO NEXT

Re-read Part I until the fundamentals are automatic, build the lab in §11.9, and run this same walk on your own test network. When a new tool or attack appears, trace it back to the basics here - it is always an address, a port, a login, or a trust.

PART XIII

Protocols in depth: the language of the wire

Part I introduced the protocols you meet. This part is the reference that makes you fluent: every protocol an assessor touches, explained in plain words, with how it works, the port it lives on, how attackers abuse it, and exactly what you check. Learn a protocol properly once and you never have to look it up mid-engagement again. Each entry links to its authoritative specification so you can go as deep as you like.

13.1

How to read any protocol

> **IN PLAIN WORDS**
>
> A protocol is just an agreed set of rules for how two machines talk: what to say first, how to reply, and what each message means. If two programs agree on the rules, they can talk even if they were written by different people on different systems. That is all a protocol is, a shared language.

You do not memorise protocols by rote. You read each one against the same six questions, and after a dozen protocols the pattern becomes automatic. Every entry below answers exactly these:

QUESTION	WHY AN ASSESSOR ASKS IT
1. What is it for?	The job it does. If you know the job, you can guess the risk.
2. What port and transport?	Where to find it on a scan, and whether it rides TCP (reliable, connection-based) or UDP (fire-and-forget).
3. Is it encrypted?	A cleartext protocol can be read or changed by anyone on the path. This single fact drives half of all network findings.
4. How does it authenticate?	Who is allowed to talk, and how they prove it. Weak or absent authentication is the other half of the findings.
5. How is it abused?	The concrete attacks. Named so you recognise them in a report and in the wild.
6. What do we check?	The assessment action: the exact thing you verify, non-destructively.

MEMORY HOOK

Say it as a sentence: **“A protocol for [job], on [port/transport], [encrypted or not], authenticated by [method], abused by [attack], and we check [control].”** Fill that sentence in for any protocol and you understand it.

FROM THE FIELD

Ninety percent of network findings reduce to two questions from the list above: **is it encrypted**, and **how does it authenticate**. A cleartext admin protocol with a default password is not an exotic exploit; it is questions 3 and 4 both failing on the same service. Train your eye on those two and you will out-find people who memorised a hundred CVEs.

13.2

Layer 2 and 3: Ethernet, ARP, IP and ICMP

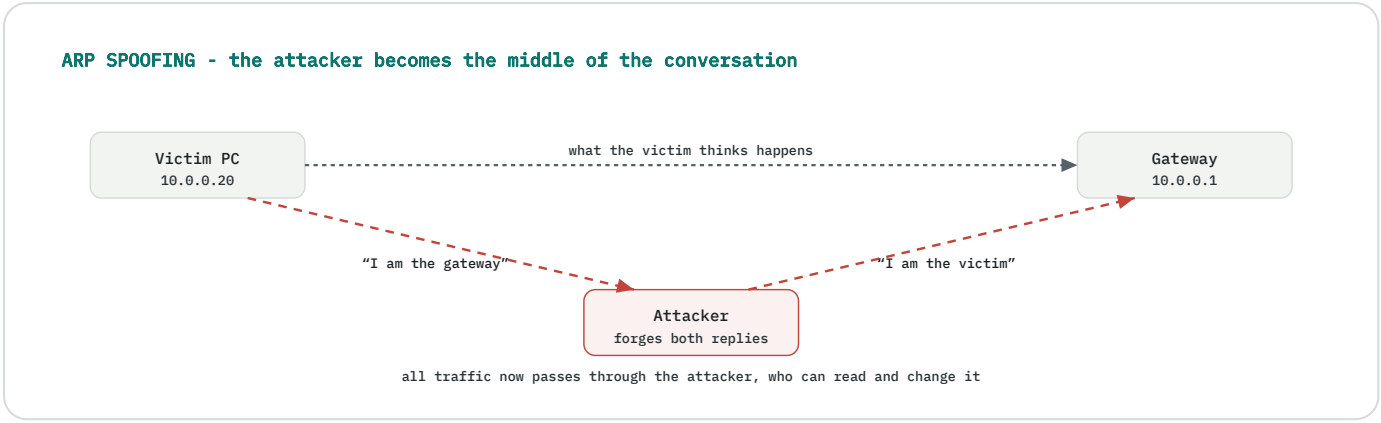
These are the protocols that get a packet from one machine to another. They were designed in an era of trust, so most have no authentication at all, which is why the local network is such fertile ground for an attacker who is already inside.

ARP (Address Resolution Protocol)

Job: on a local segment, machines talk by MAC address, but applications know only IP addresses. ARP is the lookup that answers *“who has IP 10.0.0.5? tell me your MAC.”* No transport, no ports, no authentication; it is Layer 2 broadcast. Spec: RFC 826.

How it is abused: because any host can answer any ARP query and hosts believe the last answer, an attacker sends forged replies (*ARP spoofing* / *ARP cache poisoning*) claiming to be the gateway. Traffic then flows through the attacker, who can read and alter it. This is the classic on-path (man-in-the-middle) attack and the engine behind tools like `bettercap` and `ettercap`.

What we check: is the segment flat (one broadcast domain for everything)? Are there dynamic ARP inspection (DAI) and port security on the switches? On an internal assessment, an unprotected segment where ARP can be poisoned is a real finding, because one compromised host then endangers every conversation on that VLAN.



ARP has no authentication, so the last reply wins. The attacker answers both sides, and both send their traffic through the attacker without noticing.

IP and ICMP

IP (RFC 791 for IPv4, RFC 8200 for IPv6) is the addressing and routing layer: it carries a packet from a source IP to a destination IP across routers, best-effort, with no delivery guarantee. **ICMP** (RFC 792) is IP's status-message channel: `ping` (echo request/reply) and `traceroute` both ride it. Assessors use ICMP for host discovery; defenders sometimes rate-limit or block it, which only slows discovery, it does not hide a host. IPv6 matters even when "nobody uses it": it is often on by default, unmonitored, and can offer a parallel unfiltered path to the same host.

> ASSESSOR'S NOTE
>
> Never assume a network is IPv4-only. Run discovery for both. A firewall rule set that is airtight on IPv4 and absent on IPv6 is a common, quietly serious finding.

13.3

Transport: TCP and UDP

> **IN PLAIN WORDS**
>
> TCP is a phone call: you dial, the other side answers, you both confirm you can hear each other, then you talk, and you say goodbye. UDP is a postcard: you write it, drop it in the box, and hope it arrives. TCP is used where every byte must arrive in order (web, email, file transfer); UDP where speed matters more than perfection (DNS lookups, voice, video).

TCP (RFC 9293) sets up a connection with the three-way handshake (SYN, SYN-ACK, ACK) shown in Part I, tracks every byte, retransmits what is lost, and tears down cleanly with FIN. Those control bits, the *flags*, are what a port scanner manipulates to learn a port's state without completing a full connection.

TCP FLAG	MEANING	WHY THE SCANNER CARES
SYN	Start a connection.	A lone SYN begins the fast “half-open” scan.
SYN-ACK	“I heard you, let us connect.”	A SYN-ACK back means the port is **open**.
RST	Refuse or reset.	An RST back means the port is **closed**.
ACK	Acknowledge received data.	Used to probe firewall rules (filtered vs unfiltered).
FIN	Finish, close politely.	Stealthy scans send unusual flag combos to slip past simple filters.

UDP (RFC 768) has no handshake and no flags, which makes UDP scanning slow and ambiguous: silence can mean open *or* filtered. But many high-value services are UDP only (DNS 53, SNMP 161, NTP 123, IKE/VPN 500), so skipping UDP misses them. UDP services are also the classic amplifiers in reflection DDoS, because a small request can trigger a large reply to a spoofed victim.

> **MEMORY HOOK**
>
> **Open answers, closed refuses, filtered says nothing.** SYN-ACK = open, RST = closed, silence = filtered. That one line explains most of what a port scan reports.

13.4

Naming and addressing: DNS and DHCP

These two services quietly run every network. Because everything trusts them, poisoning either is one of the most productive moves an internal attacker has.

DNS (Domain Name System)

Job: turn names people can remember (`intranet.example.com`) into IP addresses machines can route to. Runs on UDP 53 for lookups and TCP 53 for large answers and zone transfers. Classically cleartext (DNS-over-TLS and DNS-over-HTTPS are the encrypted successors). Spec: RFC 1035.

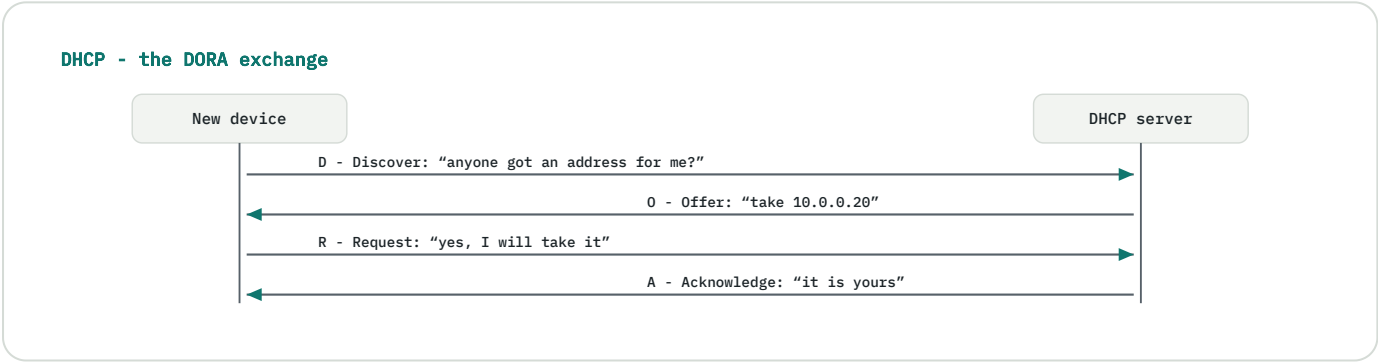
RECORD	MAPS	ASSESSOR USE
A / AAAA	name to IPv4 / IPv6	Resolve targets; find hosts.
MX	domain to mail server	Mail infrastructure, spoofing checks.
NS / SOA	domain to its DNS servers	Authority, zone-transfer target.
TXT	free text (SPF, DKIM, verifications)	Email security posture, leaked info.
PTR	IP back to name (reverse)	Map a range to hostnames.

RECORD	MAPS	ASSESSOR USE
CNAME	alias to another name	Spot cloud services, takeover risk.

How it is abused: a misconfigured server allows a *zone transfer* (AXFR), handing you the entire internal name list at once. *DNS cache poisoning* feeds a resolver a false answer so users go to the attacker's address. On Windows networks, when normal DNS fails, clients fall back to LLMNR and NBT-NS broadcasts, which an attacker answers to harvest credentials (see Part XIV). What we check: is AXFR restricted to authorised secondaries; is the resolver patched against poisoning; is LLMNR/NBT-NS disabled.

DHCP (Dynamic Host Configuration Protocol)

Job: hand a newly connected device its IP address, subnet mask, gateway and DNS server automatically. UDP ports 67 (server) and 68 (client). No authentication. Spec: RFC 2131. The exchange is four steps, remembered as *DORA*.



Discover, Offer, Request, Acknowledge. A rogue DHCP server answers Discover faster than the real one, handing the victim a gateway and DNS the attacker controls, another route to man-in-the-middle.

How it is abused: a *rogue DHCP server* races the legitimate one to answer, then gives victims the attacker's machine as their gateway and DNS. *DHCP starvation* floods the server with fake requests until it runs out of addresses, denying service. What we check: is DHCP snooping enabled on the switches (it drops offers from unauthorised ports)?

13.5

The web trio: HTTP, HTTPS and TLS

HTTP (RFC 9112, semantics in RFC 9110; HTTP/2 in RFC 9113) is the request-and-response protocol of the web, on TCP 80. A client sends a *method* and a path; the server returns a *status code* and a body. **HTTPS** is the same HTTP wrapped in **TLS** (RFC 8446) on TCP 443, which adds encryption (nobody on the path can read it), integrity (nobody can change it undetected) and identity (the certificate proves you reached the real server).

METHODS	STATUS FAMILIES	HEADERS THAT MATTER TO AN ASSESSOR
`GET` read, `POST` submit, `PUT` replace, `DELETE` remove, `OPTIONS` capabilities	**2xx** ok, **3xx** redirect, **4xx** client error (401 auth, 403 forbidden, 404 missing), **5xx** server error	`Set-Cookie` flags (HttpOnly, Secure), `Strict-Transport-Security`, `Content-Security-Policy`, `Server` version leaks

What we check on TLS: the protocol version (TLS 1.2 and 1.3 good; SSLv3, TLS 1.0 and 1.1 are findings), the cipher suites, and the certificate (expiry, trust chain, hostname match, key strength). A weak or expired certificate, or an old protocol still enabled, is one of the most common external findings because it is easy to test and easy to prove. Deeper web methodology is Part VI; the definitive testing reference is the OWASP Web Security Testing Guide and the OWASP Top 10 (2025).

13.6

The Windows enterprise stack: SMB, NTLM, Kerberos, LDAP, RDP

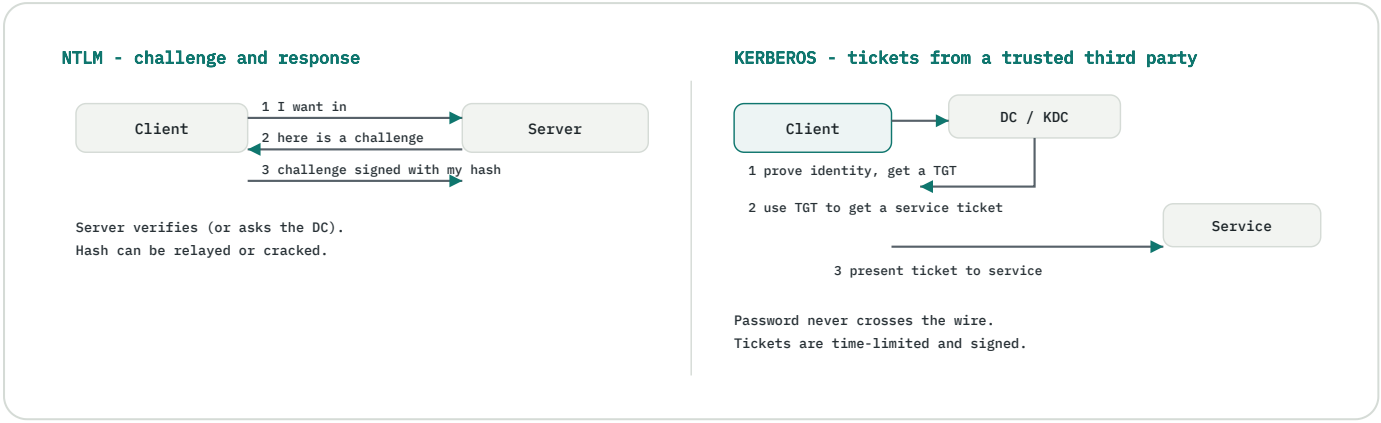
An internal Windows network runs on a small family of protocols. Understanding them is the difference between reading an Active Directory assessment and guessing at it.

SMB (Server Message Block)

Job: Windows file and printer sharing, and a transport for other services, on TCP 445 (legacy NetBIOS on 137-139). Open spec: MS-SMB2. **How it is abused:** the infamous EternalBlue (MS17-010) rode SMBv1; unsigned SMB enables relay attacks (Part XIV); readable shares leak data and credentials. **What we check:** is SMBv1 disabled; is SMB signing required (not just enabled); can a null or guest session list shares; are share permissions least-privilege.

NTLM and Kerberos (authentication)

These are the two ways a Windows identity is proven. **NTLM** is the old challenge-response scheme: it works without a domain controller but is vulnerable to relay and to offline cracking of captured hashes. **Kerberos** (RFC 4120) is the modern default: the domain controller issues time-limited tickets so the password never crosses the wire. Kerberos is stronger, but its ticket design opens specific attacks (Kerberoasting, AS-REP roasting) covered in Part XIV.



NTLM proves identity by signing a challenge with the password hash, which can be captured, relayed or cracked. Kerberos uses a trusted domain controller to issue signed, time-limited tickets, so the secret never travels.

LDAP, RDP, WinRM and MS-RPC

LDAP (RFC 4511, TCP 389, or 636 for LDAPS) is how you query Active Directory: users, groups, computers, policies. An *anonymous bind* that returns data is a finding, because it hands the directory to an unauthenticated attacker. **RDP** (TCP 3389) is remote desktop; exposed to the internet it is a top ransomware entry point, so we check for network-level authentication and whether it is reachable externally at all. **WinRM** (TCP 5985/5986) is remote PowerShell management. **MS-RPC** (TCP 135 plus dynamic ports) underlies much of Windows' internal machinery. What we check across all of them: are they exposed beyond where they need to be, are they patched, and do they enforce encryption and strong authentication.

13.7

Management and remote access: SSH, Telnet, SNMP, NTP

PROTOCOL	PORT	WHAT IT IS, AND THE FINDING TO LOOK FOR
SSH (RFC 4253)	TCP 22	Encrypted remote shell and file transfer, the Unix/Linux admin workhorse. Check for password auth where keys should be required, weak key exchange or ciphers, exposed to the internet, and any default or shared credentials.
Telnet	TCP 23	The *cleartext* ancestor of SSH. Its presence is itself a finding: everything, including the password, is readable by anyone on the path. Should not exist on a modern network.
SNMP (RFC 3411)	UDP 161/162	Monitors and manages network devices. SNMPv1/v2c use a cleartext *community string*; the defaults `public` and `private` are everywhere and leak, or even change, device config. Check for defaults, for v1/v2c where v3 (authenticated, encrypted) should be used, and for exposure.

PROTOCOL	PORT	WHAT IT IS, AND THE FINDING TO LOOK FOR
NTP (RFC 5905)	UDP 123	Time synchronisation. Matters more than it looks: Kerberos rejects tickets when clocks drift, and logs are worthless if timestamps disagree. Old NTP servers are also DDoS amplifiers (the `monlist` command). Check version and that it is not an open reflector.

FROM THE FIELD

Management protocols are where the fastest wins hide. A switch answering SNMP with the community string `public`, or a Telnet prompt on a printer, takes seconds to find and tells a story the client needs to hear: the boring interior was never hardened. You do not need an exploit; you need to notice.

13.8

File, mail and voice: FTP, SMTP, SIP

FTP (RFC 959, TCP 21) transfers files in cleartext, credentials included, and often allows anonymous access; prefer SFTP (over SSH) or FTPS. **SMTP** (RFC 5321, TCP 25, plus 587/465 for submission) moves email between servers; assessors check for an *open relay* (a server that will send mail for anyone, abused for spam and spoofing) and for the anti-spoofing records SPF, DKIM and DMARC. Companion mailbox protocols are IMAP (143/993) and POP3 (110/995). **SIP** (RFC 3261, UDP/TCP 5060, 5061 for TLS) sets up VoIP calls; check for exposed PBX interfaces, default extensions and toll-fraud exposure.

MEMORY HOOK

The cleartext family, avoid or wrap them: **Telnet, FTP, HTTP, SNMPv1/2c, SMTP without STARTTLS, POP3/IMAP without TLS**. Each has an encrypted replacement. Finding the plain version still in use is finding the report half-written.

13.9

Windows and Linux on the same wire

IN PLAIN WORDS

The same network usually carries both Windows and Linux machines, and they do the same jobs (log in, find each other, share files, be managed remotely) using different protocols. You need to recognise both, because an assessment touches both and the client runs both.

JOB	WINDOWS WAY	LINUX / UBUNTU WAY
Get an address	DHCP client (built in)	DHCP client (`dhclient`, `systemd-networkd`, NetworkManager)
Resolve names	DNS, plus legacy LLMNR / NBT-NS fallback	DNS via `/etc/resolv.conf`, `systemd-resolved`; sometimes mDNS/Avahi
Log in / identity	Active Directory: Kerberos, NTLM, LDAP	Local accounts, LDAP, SSSD joined to AD, or Kerberos
Share files	SMB (TCP 445)	NFS (2049), or SMB via Samba to interoperate with Windows
Remote admin	RDP (3389), WinRM (5985/5986)	SSH (22)
Config management	Group Policy from the domain controller	SSH plus Ansible/Puppet/Salt; cloud-init on first boot

FROM THE FIELD

Mixed shops are where interesting findings live. A Linux server joined to Active Directory via SSSD inherits the domain's weaknesses; a Samba share may allow the guest access its Windows equivalent would refuse; and a Windows admin who lives in RDP often forgets the Linux boxes still answer SSH with password auth. Map both stacks in the first hour and you will not be surprised in the third.

13.10

The protocol quick card

One table to scan before an engagement. If a service is cleartext or authenticates weakly, it earns a closer look.

PROTOCOL	PORT	TRANSPORT	ENCRYPTED?	FIRST THING TO CHECK
FTP	21	TCP	No	Anonymous access, cleartext creds
SSH	22	TCP	Yes	Password auth, weak ciphers, exposure
Telnet	23	TCP	No	Presence is the finding
SMTP	25	TCP	Optional	Open relay, SPF/DKIM/DMARC
DNS	53	UDP/TCP	Usually no	Zone transfer, poisoning, LLMNR fallback

PROTOCOL	PORT	TRANSPORT	ENCRYPTED?	FIRST THING TO CHECK
DHCP	67/68	UDP	**No**	Rogue server, DHCP snooping
HTTP	80	TCP	**No**	Redirect to HTTPS, headers
Kerberos	88	UDP/TCP	Yes	Roastable accounts, clock skew
NTP	123	UDP	No	Amplification, version
MS-RPC	135	TCP	Varies	Exposure beyond internal
NetBIOS	137-139	UDP/TCP	**No**	NBT-NS poisoning, null sessions
SNMP	161	UDP	v3 only	Default community strings, v1/v2c
LDAP	389	TCP	636=LDAPS	Anonymous bind, cleartext bind
HTTPS	443	TCP	Yes	TLS version, ciphers, certificate
SMB	445	TCP	Signing	SMBv1, signing, null session, shares
RDP	3389	TCP	Yes	Internet exposure, NLA
WinRM	5985/5986	TCP	5986=TLS	Exposure, auth

MEMORY HOOK

Read the card top to bottom once before every engagement. The red “No” rows are where the easy, defensible findings almost always are.

PART XIV

Network attacks and vulnerabilities: the catalogue

This is the field guide to how networks are actually attacked, grouped so the patterns stick. For each attack you get what it is, how it works, and the control that stops it, because our job is to *verify that control*, not to run the attack. Keep the standing rule in view: this assessment identifies and validates weaknesses (NIST SP 800-115 §3.4 and §4.3); exploitation (§5) needs separate written authorisation. You learn the attack so you understand exactly what you are checking and why it matters. The living references are MITRE ATT&CK and HackTricks.

14.1

How to think about attacks

IN PLAIN WORDS

There are not a thousand different attacks to memorise. There are a handful of *families*, and almost everything is a variation. Learn the families and you can place any new attack you meet, including ones invented after this handbook was written.

FAMILY	THE CORE IDEA	EXAMPLES IN THIS PART
Interception (on-path)	Get in the middle of a conversation to read or change it.	ARP spoofing, LLMNR/NBT-NS poisoning, DNS spoofing, rogue DHCP, SSL stripping
Credential	Obtain or reuse a login without knowing the password.	Spraying, hash capture and cracking, pass-the-hash, relay
Directory (Active Directory)	Abuse how Windows trust and tickets work.	Kerberoasting, AS-REP roasting, pass-the-ticket
Service exploitation	Use a flaw or default in an exposed service.	Known-CVE exploitation, default credentials, misconfiguration
Layer 2	Abuse the switch and the local segment itself.	VLAN hopping, MAC flooding, STP attacks
Availability (DoS)	Make a service stop answering.	Volumetric, protocol, application, amplification

MEMORY HOOK

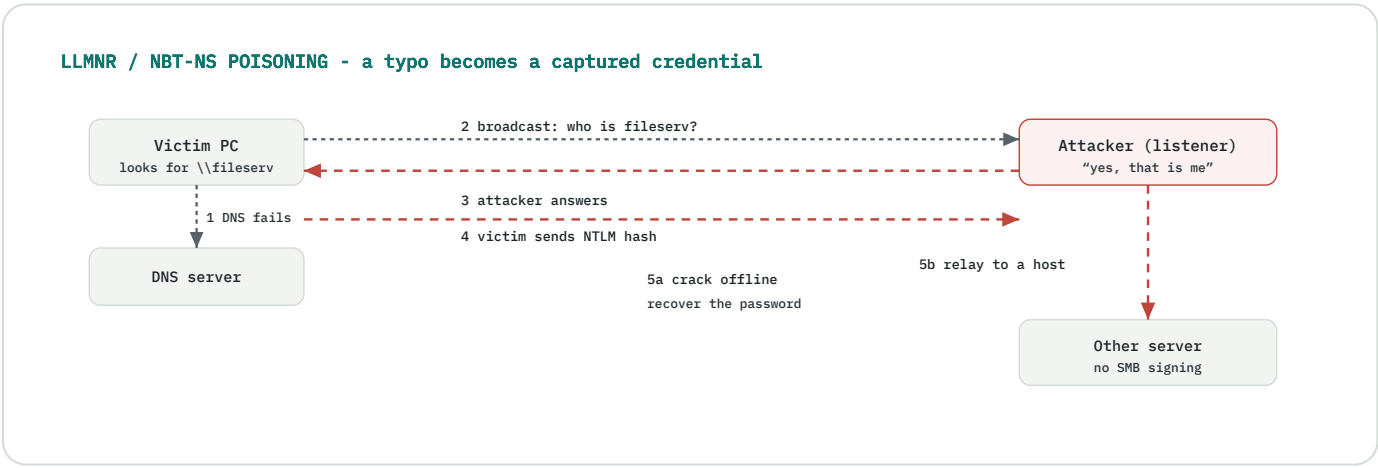
Six families: **Intercept, Credential, Directory, Service, Layer 2, Availability**. Every attack you will ever read about lands in one of these boxes.

14.2

Interception: poisoning, relay and man-in-the-middle

ARP spoofing (§13.2) is the classic, but on a Windows network the most productive interception attack is **LLMNR and NBT-NS poisoning**, mapped by MITRE as T1557.001 (LLMNR/NBT-NS Poisoning and SMB Relay).

How it works: when a Windows machine cannot resolve a name through DNS (a typo, a dead share, a stale mapping), it does not give up; it shouts to the whole local segment over LLMNR and NBT-NS: *"does anyone know where fileserv is?"* These fallback protocols have no authentication, so an attacker running a listener simply answers *"yes, that is me"*. The victim then tries to authenticate to the attacker, sending its NTLM challenge-response, which the attacker either cracks offline to recover the password, or immediately *relays* to another machine that does not require SMB signing, authenticating *as the victim* without ever knowing the password.



The attacker never breaks in. A failed name lookup hands over a credential, which is then cracked offline or relayed straight to a server that does not require SMB signing.

What we check (the controls): is LLMNR disabled by Group Policy; is NBT-NS disabled on adapters; is *SMB signing required* (this alone defeats the relay); is the segment monitored for rogue responders. Related interception attacks and their controls: *DNS spoofing* (patched resolvers, DNSSEC), *rogue DHCP* (DHCP snooping, §13.4), and *SSL stripping* (HSTS, redirect all HTTP to HTTPS).

FROM THE FIELD

On a fresh internal engagement, LLMNR/NBT-NS is often the first credential you obtain, sometimes within minutes, entirely passively, just by listening. That is exactly why it is worth a finding: if you can hear it, so can anyone who plugs into the network. The fix is two Group Policy settings and requiring SMB signing, and it closes a whole class of attacks at once.

14.3

Credential attacks: getting a login without the password

ATTACK	WHAT IT IS	THE CONTROL WE CHECK
Password spraying	Try *one* common password against *many* accounts, staying under the lockout threshold. Quiet and effective; the opposite of brute force.	Strong password policy, MFA, lockout tuned to catch spraying, alerting on many-accounts-one-password.
Brute force	Many passwords against one account. Loud; triggers lockouts. Rare against AD for that reason.	Lockout policy, rate limiting, MFA.
Credential stuffing	Reuse username/password pairs leaked from other breaches.	MFA, breached-password screening, no reuse.
Hash capture + cracking	Capture an NTLM hash (via poisoning, §14.2) and recover the password offline with Hashcat or John.	Long passphrases (cracking cost), stopping the capture in the first place.
Pass-the-hash	Use a captured NTLM hash *directly* to authenticate, no cracking needed, because NTLM accepts the hash as proof. MITRE T1550.002 (Pass the Hash).	Restrict local admin reuse (LAPS), credential guard, tiered admin, disable NTLM where possible.

MEMORY HOOK

Spray is one password, many users. **Brute** is many passwords, one user. **Stuffing** is reused leaks. **Pass-the-hash** skips the password entirely. MFA blunts all four.

14.4

Active Directory and Kerberos attacks

Active Directory runs the Windows enterprise, so its trust model is the prize. These attacks abuse how Kerberos issues tickets (§13.6). They are the reason the toolkit collects AD configuration: to find the

conditions that make them possible *before* an attacker does.

ATTACK	HOW IT WORKS	THE ENABLING WEAKNESS WE FLAG
Kerberoasting (T1558.003)	Any domain user can request a service ticket for an account that runs a service (an SPN). Part of that ticket is encrypted with the service account's password hash, so it can be cracked offline.	Service accounts with weak passwords and no gMSA; over-privileged service accounts.
AS-REP roasting	Accounts with "do not require Kerberos pre-authentication" hand out a crackable blob to *anyone* who asks, no credentials needed.	Accounts with pre-authentication disabled.
Pass-the-ticket	Steal a valid Kerberos ticket from memory and reuse it as that user.	Credential exposure on shared hosts; no credential guard.
Golden / silver ticket	With the domain's master key (krbtgt) or a service key, forge tickets at will. This is post-compromise persistence, shown here for context.	krbtgt never rotated; a domain already fully compromised.

The end-to-end attack path, foothold to domain dominance, is drawn in §4.3. The toolkit's role is upstream of all of it: identify the roastable accounts, the stale krbtgt, the excessive privileges, and the missing signing, so the chain never starts.

14.5

Service exploitation: CVEs, defaults and misconfiguration

Not every attack is clever. Many are simply an exposed service that is unpatched, left on a default password, or misconfigured.

- **Known-CVE exploitation** (T1210): a public vulnerability with a working exploit. The most dangerous are the ones on the CISA KEV catalogue, meaning attackers are using them right now. EternalBlue (MS17-010) against SMBv1 is the textbook example. We identify the missing patch and its KEV status; the fix is patching.
- **Default and shared credentials:** the printer on `admin/admin`, the switch on `public`, the app with the vendor's demo login. Fast to find, embarrassing to leave. Check every management interface.
- **Misconfiguration:** an exposed database with no authentication, a management port open to the internet, an anonymous file share, directory listing enabled. No exploit required; the door is simply open.

THE STANDING RULE

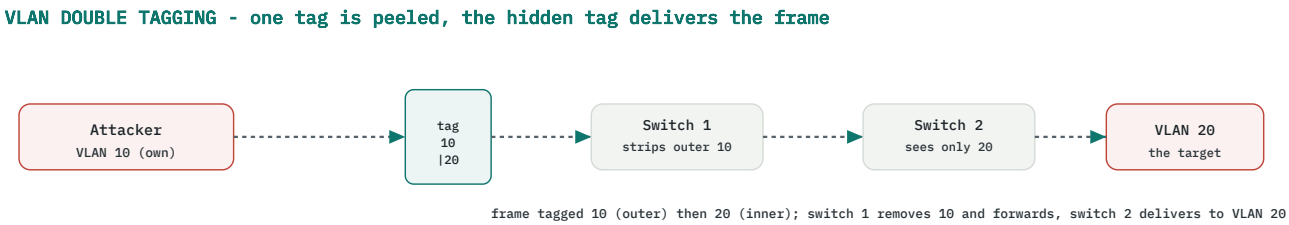
We prove the door is open and unlocked. We do not walk through it and rearrange the furniture. Identifying an exploitable condition is the assessment; running the exploit to prove impact is a penetration test, and needs its own written authorisation.

14.6

Layer 2 attacks: abusing the switch

Segmentation lives on the switch (§1.6). These attacks try to defeat it from a single connected port.

ATTACK	HOW IT WORKS	THE CONTROL WE CHECK
VLAN hopping (switch spoofing)	A port left on auto-trunk can be tricked into forming a trunk, giving the attacker access to every VLAN.	Disable DTP/auto-trunk; set access ports explicitly.
VLAN hopping (double tagging)	Two stacked VLAN tags; the first switch strips one and forwards the frame into a VLAN the attacker should not reach.	Change the native VLAN off the default; do not use VLAN 1.
MAC flooding	Flood the switch's address table until it fails open and broadcasts all traffic, like a hub, so the attacker sees everything.	Port security (limit MACs per port).
STP attack	Claim to be the root bridge so traffic reroutes through the attacker.	BPDU guard, root guard.



Double tagging exploits how a switch strips only the outermost VLAN tag. The hidden inner tag then carries the frame into a VLAN the attacker was never meant to reach. Moving the native VLAN off VLAN 1 defeats it.

14.7

Denial of service and amplification

A denial-of-service attack does not steal data; it removes availability, one of the three pillars alongside confidentiality and integrity. We do *not* run DoS in an assessment (it is destructive and out of scope by default); we identify the *exposure*.

TYPE	WHAT IT DOES
Volumetric	Flood the link with more traffic than it can carry (UDP/ICMP floods).
Protocol	Exhaust a device's connection state (SYN flood fills the connection table).
Application	Send requests that are cheap to make but expensive to answer (a slow, heavy search).
Reflection / amplification	Send a small spoofed request to a public service (DNS, NTP `monlist`, memcached) that replies with a large answer to the *victim*. A small input becomes a huge attack.

What we check: is any internal service reachable from the internet as an open amplifier (open DNS resolver, NTP with monlist, exposed memcached)? Those are real findings, because the client's own infrastructure can be turned into a weapon against others, and against themselves.

14.8

The web and application vulnerabilities, in one place

Web and API testing has its own methodology in Part VI; this is the orientation so the names are familiar. The authority is the OWASP Top 10 (2025), with the API Security Top 10 for APIs and the OWASP Cheat Sheet Series for defences.

CLASS	IN PLAIN WORDS
Broken access control	You can reach or change data that is not yours (the top category; now includes SSRF).
Injection	Your input is treated as code or a command (SQL injection, command injection).
Cryptographic failures	Sensitive data is unencrypted, or protected with something weak.
Security misconfiguration	Default settings, verbose errors, unnecessary features left on.
Authentication failures	Weak login, guessable sessions, no MFA.
Vulnerable components	A known-vulnerable library or framework in the stack (the host toolkit's software check, §7, is the network-side of this).

FROM THE FIELD

The single most common serious web finding, year after year, is broken access control: change an ID in the URL and you see another customer's record. It needs no exploit tool, only curiosity and Burp. Master that one idea and you will find real issues on almost every application you test.

14.9

The attack-to-control map

Every attack in this part is stopped by a control you can verify. This table is the whole part in one view: read the attack, know the control, and you know what to check.

ATTACK	WHAT ENABLES IT	THE CONTROL THAT STOPS IT
ARP spoofing	Flat segment, no L2 protection	Dynamic ARP inspection, segmentation
LLMNR/NBT-NS poisoning	Name-resolution fallback on	Disable LLMNR & NBT-NS; require SMB signing
Rogue DHCP	Any port can offer addresses	DHCP snooping
Password spraying	Weak passwords, no MFA	MFA, policy, spray-aware alerting
Pass-the-hash	Shared local admin, NTLM	LAPS, tiered admin, Credential Guard
Kerberoasting	Weak service-account passwords	gMSA, long passphrases, least privilege
Known-CVE exploitation	Missing patch on exposed service	Patch (prioritise KEV), reduce exposure
Default credentials	Vendor defaults unchanged	Change defaults, credential management
VLAN hopping	Auto-trunk, native VLAN 1	Disable DTP, move native VLAN
Amplification DoS	Open UDP reflector exposed	Close/restrict resolver, patch NTP

MEMORY HOOK

Notice that most controls are configuration, not products: disable a fallback, require a signature, change a default, move a VLAN. That is the good news you carry to the client, and the reason a network assessment is worth so much: most of what it finds is fixable this week, for free.

PART XV

Databases on the network

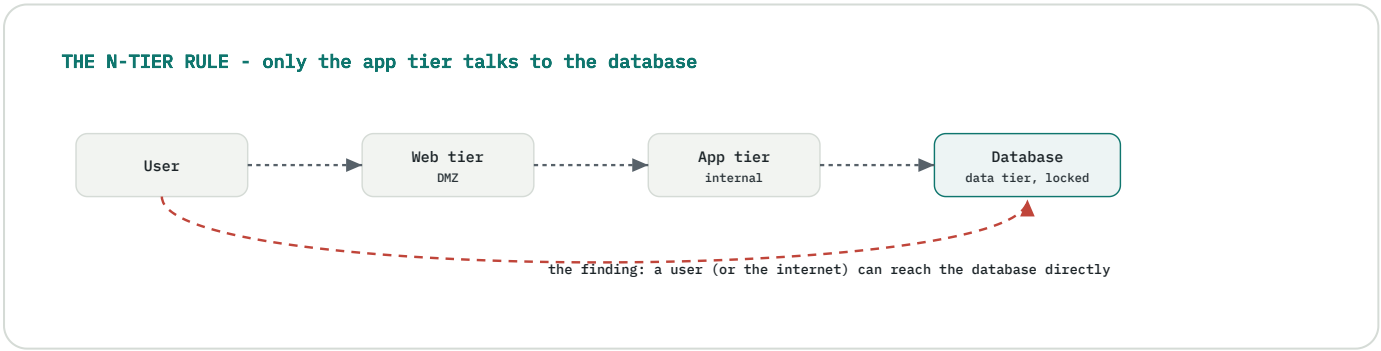
Databases hold the thing the whole engagement is really about: the data. They are also, with dispiriting regularity, the easiest serious finding on an internal network, because a database that should only ever hear from one application server is instead answering the entire subnet, often with no password at all. This part is how to recognise them, where they belong, and what to check.

15.1

Where a database belongs

IN PLAIN WORDS

In a well-built application, the database is the innermost room. The user talks to the web server, the web server talks to the application, and only the application talks to the database. The database should never take a call directly from a user's PC, and certainly never from the internet. When it does, the room with all the valuables has been left unlocked.



The database should be reachable only from the application tier. A path straight from a user zone or the internet to the database port is one of the highest-value findings on any internal test.

15.2

The database quick reference

Learn the ports. On a scan, these numbers tell you a data store is present, and each has a signature weakness.

ENGINE	PORT	SIGNATURE WEAKNESS TO CHECK
Microsoft SQL Server	TCP 1433 (UDP 1434 browser)	Default/weak sa password; `xp_cmdshell` enabled; exposed browser service.
MySQL / MariaDB	TCP 3306	Root with no or weak password; remote root allowed; unencrypted connections.
PostgreSQL	TCP 5432	Trust authentication in `pg_hba.conf`; default `postgres` account; broad listen address.
Oracle	TCP 1521	Default accounts; TNS listener with no password; SID guessing.
MongoDB	TCP 27017	**No authentication by default in older setups**; bound to all interfaces.
Redis	TCP 6379	**No auth by default**; exposed instance allows config write and, classically, host compromise.
Elasticsearch	TCP 9200	Open HTTP API with no auth; full data readable in a browser.
Memcached	TCP/UDP 11211	No auth; a major UDP amplification reflector when internet-exposed.

MEMORY HOOK

The “no password out of the box” trio, **MongoDB, Redis, Elasticsearch**, has caused some of the largest data leaks on record, every one of them just an exposed port. If you see 27017, 6379 or 9200 answering from a user subnet, look immediately.

15.3

How databases are reached and abused

- **Exposed with no authentication:** the NoSQL classic. Connect to the port, read everything. No exploit, no credentials.
- **Default or weak credentials:** `sa` with a blank or guessable password on SQL Server; `root` with nothing on MySQL.
- **Unencrypted connections:** if the app-to-database traffic is cleartext, an on-path attacker (§14.2) reads queries and results, credentials included.
- **SQL injection at the application:** the database is reached *through* the web app when input is treated as query code (§14.8). This is an application finding whose blast radius is the whole database.
- **Feature abuse for host access:** SQL Server's `xp_cmdshell`, PostgreSQL's programmable functions, or a writable Redis instance can turn database access into operating-system access.

- **Exposed backups:** a `.bak`, `.sql` or dump file on an open share is the whole database with none of the access controls.

15.4

What we check

CHECK	WHY IT MATTERS
Reachability	From which zones is the database port answerable? It should be the app tier only (§15.1). A path from a user VLAN or the internet is the finding.
Authentication	Is a login required at all, and are defaults changed? Test with published defaults, non-destructively.
Encryption in transit	Is the connection TLS-protected, or cleartext on the wire?
Least privilege	Does the application account have only the rights it needs, or is it effectively an administrator?
Patch level	Version against known vulnerabilities; cross-check the KEV catalogue.

FROM THE FIELD

You rarely need to touch the data to write the finding. “Your customer database on 10.4.2.9:27017 accepts connections from the staff Wi-Fi with no password” is complete, provable from a single connection banner, and impossible to argue with. Prove reachability and the absent control; never dump or alter the records. That restraint is exactly what keeps the client trusting you.

PART XVI

Tools in depth: what to run, and when

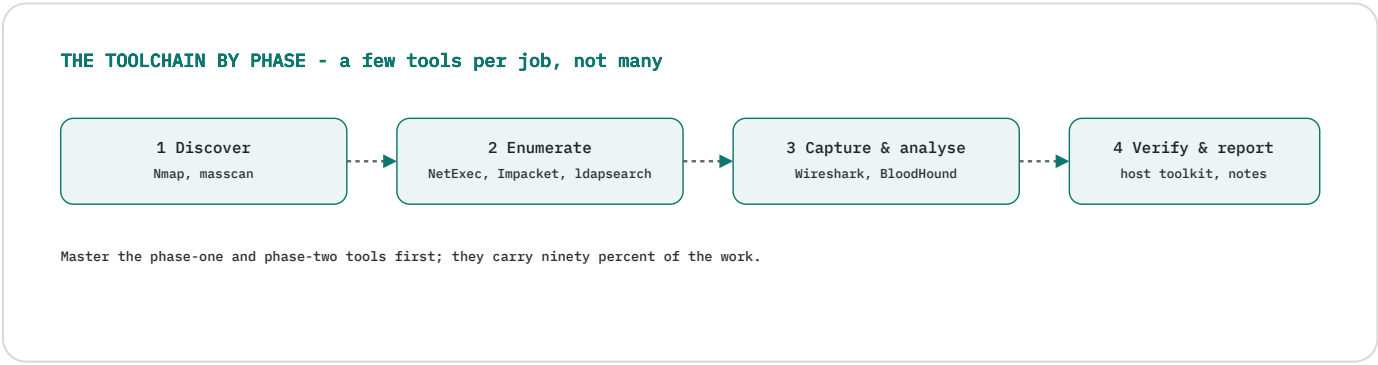
Tools do not make an assessor; understanding does. But the right tool, driven by someone who knows what its output means, is how understanding turns into evidence. This part covers the tools a network assessor reaches for most, what each is for, one safe example, and the official home for each so you always download from source. Everything here is used for discovery, enumeration and verification. The exploitation frameworks are named for awareness only; running them is penetration-test scope with separate authorisation (§14.5).

16.1

How to think about tooling

IN PLAIN WORDS

Do not collect tools like trophies. For any engagement you only ever need a handful, and they line up with the phases of the work: find what is there, ask each thing what it is, look closely at the interesting ones, and record it. Learn one tool per job deeply rather than ten shallowly.



Every tool below fits one of these four jobs. When you are unsure what to run, ask which phase you are in.

16.2

Discovery and scanning: Nmap

Nmap is the one tool to master. It finds live hosts, open ports, the service and version behind each port, and (with the scripting engine) a great deal more. Deep coverage is in §4.4; here is the working shape.

```
# discover live hosts on a subnet, no port scan
nmap -sn 10.20.4.0/24
# full TCP port + service/version, saved in every format
nmap -sS -p- -sV -oA scans/internal 10.20.4.0/24
# the top UDP services people forget
nmap -sU --top-ports 50 10.20.4.10
# safe, non-intrusive scripts for a host
nmap -sV --script "default and safe" 10.20.4.10
```

masscan scans enormous ranges fast when you need a first sweep, then you confirm with Nmap. The rule: sweep wide, confirm deep. Never trust a single fast scan's port list as final.

16.3

Capture and analysis: Wireshark and tcpdump

Wireshark (graphical) and tcpdump (command line) record and decode the actual packets on the wire. They are how you *prove* a protocol is cleartext, watch a handshake, or confirm what a device is really doing. In an assessment you capture your own traffic and broadcast traffic to characterise the segment; you do not intercept others' traffic without explicit authorisation.

```
# capture on an interface to a file for later analysis
tcpdump -i eth0 -w capture.pcap
# see cleartext credentials cross the wire (your own test login)
# open capture.pcap in Wireshark, filter: http.request or ftp or telnet
```

FROM THE FIELD

A thirty-second capture is often the most persuasive evidence in the whole report. Showing a client their own admin password sitting in plain text inside a Telnet packet ends the debate about whether cleartext protocols matter.

16.4

Windows and Active Directory enumeration

TOOL	WHAT IT DOES
NetExec (nxc)	The maintained successor to CrackMapExec. Sweeps a range over SMB/WinRM/LDAP to enumerate hosts, shares, sessions, users and policy, and to check where credentials are valid. Its enumeration is assessment; its module runner can be exploitation, so mind the scope.
Impacket	A Python toolkit of small, sharp scripts (`GetUserSPNs`, `secretsdump`, `smbclient`) that speak Windows protocols directly. The Swiss-army knife of AD work.
BloodHound	Collects directory relationships and *graphs* the attack paths, showing how a low-privilege user could reach Domain Admin. Turns a heap of AD data into a picture. (Attack-path graphing is a future workstream; the toolkit collects the configuration today, §7.8.)
kerbrute	Fast, low-noise username enumeration and password spraying against Kerberos, without the lockout risk of hammering SMB.
ldapsearch / enum4linux-ng	Query LDAP and SMB for users, groups and shares; the fastest way to spot anonymous binds and null sessions.
Responder	Listens for and answers LLMNR/NBT-NS/mDNS to demonstrate poisoning (§14.2). Powerful and noisy; use its analyse-only mode first, and only capture within authorised scope.

16.5

Credential and hash tooling

Hashcat (GPU) and John the Ripper (flexible) recover passwords from captured hashes offline. In an assessment their purpose is to *demonstrate password weakness*: a hash that cracks in seconds proves the policy is inadequate. You report that a percentage cracked and how fast, never the plaintext passwords themselves in the deliverable.

> ASSESSOR'S NOTE
>
> Offline cracking touches nothing on the client network, it runs on your own machine against data you already captured, which is why it is safe assessment activity. The finding is “these credentials are weak,” evidenced by crack rate, not a password list.

16.6

Web and API tooling

TOOL	WHAT IT DOES
Burp Suite	The intercepting proxy at the centre of web testing: see and modify every request, replay, and probe access control. The core web tool, and one of the tools named in client reports. Free training: Web Security Academy.
ffuf / gobuster	Content and directory discovery: find hidden endpoints, parameters and virtual hosts by fuzzing.
Nikto	Quick checks for common web-server misconfigurations and known issues; a fast first look, not a full test.
sqlmap	Detects and demonstrates SQL injection. Detection is assessment; automated data extraction is intrusive and needs authorisation. Confirm carefully, do not dump.

16.7

Wireless tooling

Covered in method in Part V. The staples: Aircrack-ng (the suite for capturing and testing Wi-Fi encryption) and hcxdumptool (efficient PMKID and handshake capture). Both require a wireless adapter that supports monitor mode and packet injection, which is why wireless is an on-site, not a remote, assessment.

16.8

Vulnerability scanners and reference databases

Where a broad automated network scan is in scope, the established engines are Nessus, Greenbone / OpenVAS and Nuclei. They cast a wide net and always need a human to confirm what is real. For host-level depth, the toolkit performs *credentialed* assessment from the inside (Part VII), which is more accurate than unauthenticated scanning. Keep these reference databases open while you work:

REFERENCE	USE IT TO
CISA KEV	Know which vulnerabilities are being exploited right now (fix these first).
NVD + CVSS calculator	Look up a CVE and score it mechanically.
GTFOBins / LOLBAS	Which built-in Unix / Windows binaries can be abused.
LOLDrivers	Known vulnerable or malicious drivers.

REFERENCE	USE IT TO
PayloadsAllTheThings / HackTricks	Technique reference for almost anything you meet.

AWARENESS ONLY

Metasploit and mimikatz are exploitation and credential-theft frameworks. You must recognise them and understand what they do, because attackers use them and reports reference them, but running them is penetration-test scope with its own written authorisation, not default assessment activity.

16.9

The tool selection map

YOUR GOAL	REACH FOR	WHAT IT TELLS YOU
What hosts and ports exist?	Nmap	The live attack surface
What is this service, exactly?	Nmap -sV, banner grab	Product and version to check
Is this protocol cleartext?	Wireshark / tcpdump	Proof on the wire
What can these credentials reach?	NetExec	Where a login is valid
How does AD trust flow?	BloodHound	The path to Domain Admin
Are service accounts weak?	Impacket GetUserSPNs, kerbrute	Roastable, sprayable accounts
Is the web app authorising properly?	Burp Suite	Broken access control
Is Wi-Fi encryption sound?	Aircrack-ng, hcxdumptool	Handshake / PMKID strength

MEMORY HOOK

The tool is never the finding. “Nmap found port 445” is not a finding; “SMB signing is not required, so captured credentials can be relayed to this host” is. The tool gathers evidence; **you** write the finding.

PART XVII

How real client networks are built, and how you get in

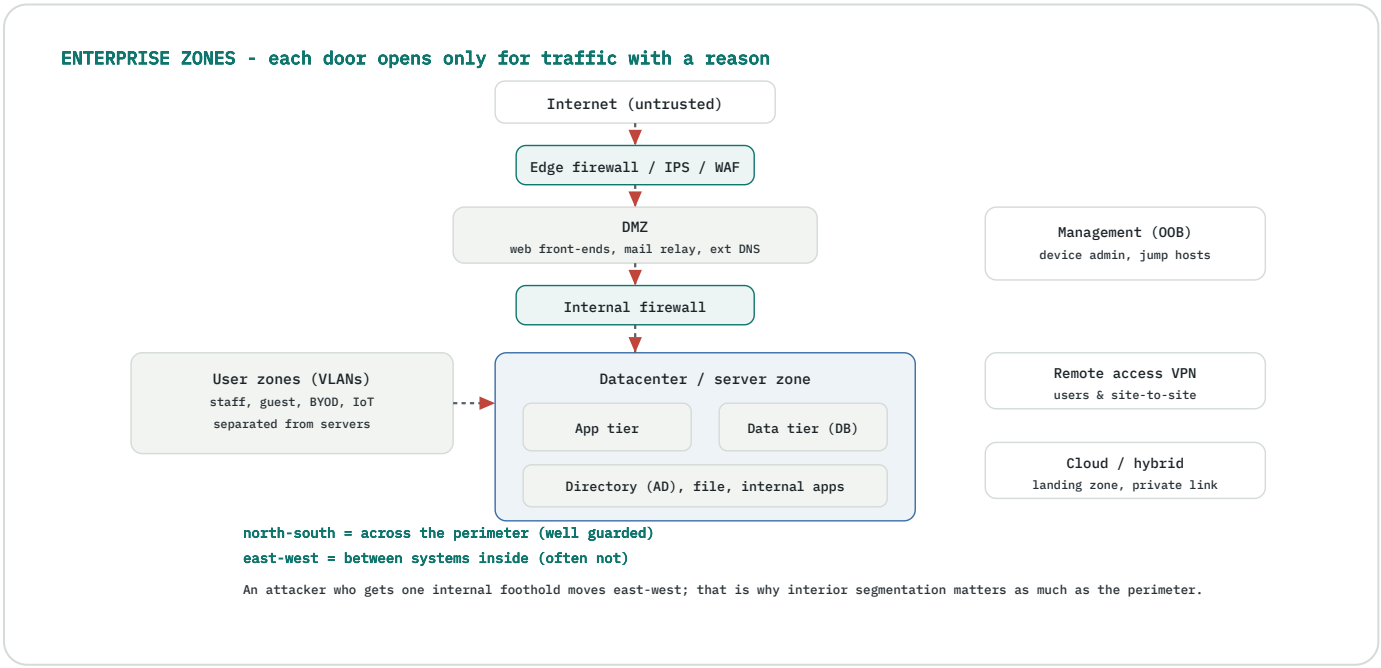
Everything so far has been the how. This part is the where: what a real corporate or banking network actually looks like, why it is shaped that way, and the practical business of getting a tester into it, over the internet, down a VPN, or sitting in the client's office. When you can picture the client's network before you arrive, you scope better, test faster, and speak their language in the debrief. Everything here is generic and standards-based; no real client's design is shown.

17.1

The zone model of an enterprise network

IN PLAIN WORDS

A big network is not one open space. It is a set of rooms with locked doors between them, and each door only opens for traffic that has a written reason to pass. The rooms are called zones; the doors are firewalls and access lists. Your job is to learn the rooms, then test whether the doors actually hold.



The zones from the internet inward, with the management network kept out-of-band and remote access and cloud entering through controlled points. Reference: Cisco SAFE.

ZONE	WHAT LIVES THERE	WHY THE ASSESSOR CARES
Internet edge	Border routers, edge firewall, IPS, WAF, DDoS scrubbing.	The outer door. Everything an external test starts against.
DMZ	Internet-facing services so outsiders never touch the inside directly.	Single vs dual-firewall design; whether the DMZ can reach the internal network.
Application / data tiers	The n-tier split: app logic, then the database, each more restricted (§15.1).	Can a user zone or the internet reach the data tier? A top finding.
User zones	Staff, guest, BYOD, IoT, split by VLAN.	Is guest truly isolated? Can a user PC reach servers it should not?
Management (OOB)	Device admin interfaces, hypervisor management, jump hosts, kept off the production path.	If production compromise reaches management, it is game over. Check the separation.
Remote access / cloud	VPN termination and cloud landing zones.	VPNs land in a controlled zone, not straight onto the flat LAN; cloud mirrors on-prem tiering.

17.2

Segmentation, micro-segmentation and zero-trust

The zones only mean something if the doors are enforced. **Segmentation** is the coarse version: firewalls and VLANs between zones. **Micro-segmentation** pushes policy down to the individual workload, so even two servers in the same datacenter cannot talk unless a rule says so; this is what contains east-west movement. **Zero-trust** (NIST SP 800-207) is the principle behind it: never trust by location, authenticate and authorise every request. Its three logical parts are the Policy Engine (decides), the Policy Administrator (enacts), and the Policy Enforcement Point (the door that actually blocks or allows).

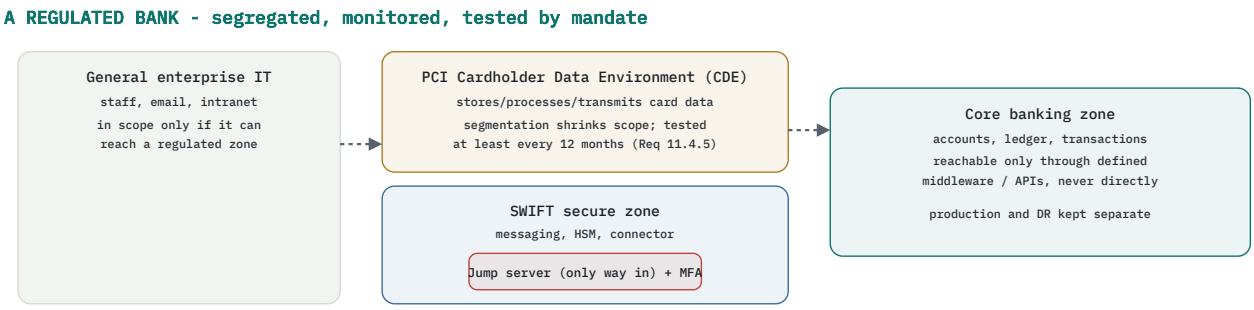
> **MEMORY HOOK**
> Old model: hard shell, soft centre, once you are in, you can roam. Zero-trust: every door is locked, every time, no matter where you are standing. Segmentation testing measures which model the client actually has.

17.3

Banking and regulated networks

Banks are the most heavily segmented networks you will meet, because overlapping mandates require it and the assets are high value. You do not need a real bank's map; you need the control requirements that

shape every one of them.



Regulated zones are deliberately walled off from general IT and from each other. References: PCI DSS and the SWIFT Customer Security Programme.

CONTROL AREA	WHAT IT MEANS FOR THE NETWORK
PCI CDE segmentation	The cardholder data environment is isolated so unrelated systems fall *out of scope*. Segmentation is not strictly mandatory, but without it the whole network is in scope. Where used to reduce scope, it must be penetration-tested at least every 12 months and after changes (Req 11.4.5), confirming out-of-scope segments genuinely cannot reach the CDE.
SWIFT secure zone	SWIFT components sit in a segregated zone (CSCF Control 1.1), entered only through a hardened *jump server* with MFA. Administrators connect through it, never directly, so the boundary and the logging hold.
Core banking isolation	The core (accounts, ledger, transactions) is the innermost zone, reachable only through defined middleware and APIs, never from user or internet zones.
ATM & branch / WAN	ATMs and branches connect over private WAN or encrypted tunnels into dedicated segments, kept apart from general corporate traffic.
Production / DR separation	Production and disaster-recovery environments are distinct, often geographically, with controlled replication paths.

FROM THE FIELD

On a regulated engagement, half the value is confirming the walls are real. The client **says** the card environment is segmented; your reachability test (§17.6) either proves it or finds the one forgotten firewall rule that puts the whole thing back in scope. That single result can be worth the entire engagement to them.

17.4

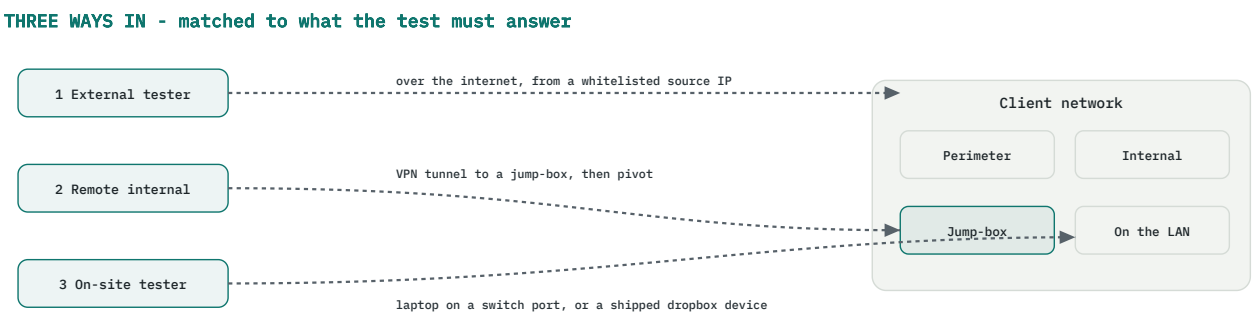
When there is OT: the Purdue model

Banks, utilities and manufacturers often run operational technology (building management, industrial control) alongside IT. These are organised by the **Purdue model**: Level 0 (sensors and actuators), Level 1 (controllers, PLCs), Level 2 (supervisory HMI/SCADA), Level 3 (site operations), a Level 3.5 IT/OT DMZ, and Levels 4 and 5 (business IT). **IEC 62443** adds the zones-and-conduits model: group assets into zones, allow traffic only through defined, secured conduits. The assessor's rule for OT is caution: these systems are fragile and safety-critical, so testing is gentle, often passive, and only with explicit sign-off. Active probing that is routine on IT can halt a physical process.

17.5

How a tester actually gets in

There is no single way. The access method is chosen to match the question the engagement is answering, and it is written into the rules of engagement.



External tests the perimeter; VPN-to-jump-box and on-site test the interior. The method is chosen to answer the engagement's real question, and always pre-authorised.

METHOD	WHAT IT SIMULATES / WHEN TO USE
Black-box external	An internet attacker with no access. Tester works from a defined, whitelisted source IP so defenders can tell the test from a real attack. Tests the perimeter and public services.
Authenticated / grey-box	An attacker who phished a user or an insider. Given a standard login or token; far deeper coverage per day than pure black-box.
Remote internal (VPN + jump-box)	The client stands up a VPN to a bastion inside; the tester pivots from there into scoped segments. Internal testing without travel.
On-site	Tester plugs a laptop into a LAN port, seeing the network as a connected device does. Needed for wireless and physical-layer work.
Dropbox / implant	A small pre-configured device is shipped to site, plugged in, and calls back, giving remote internal access with no on-site tester. Tracked and removed at the end.

METHOD	WHAT IT SIMULATES / WHEN TO USE
Assumed breach	Start from a foothold already inside (a low-priv account or workstation) and measure lateral movement, escalation and detection, rather than spending the engagement on initial access.
Credentialed scanning	The scanner (or the host assessment toolkit, Part VII) logs in to read patch and config from the inside, far more accurate than unauthenticated scanning.

TWO GATES YOU WILL MEET

PAM / jump servers: testers are often granted access *through* a Privileged Access Management system or bastion, scoped to just the in-scope hosts and fully logged (tester to jump server to target). **NAC / 802.1X:** port-based access control can block an unknown testing laptop or quarantine it; where it must be worked around for the test to proceed, the client pre-authorises the device. NAC itself is worth testing, can it be bypassed by MAC spoofing?

17.6

Inside versus outside, and the reachability matrix

The outside attacker sees only what the perimeter exposes: public IPs, DMZ services, the VPN portal, published APIs, and any accidental exposure. The inside foothold reaches whatever segmentation allows, and then pursues lateral movement toward the high-value zones. The gap between the two is the whole point: a strong perimeter can hide a flat, soft interior.

The reachability matrix (introduced in §4.7) is how you prove it. From representative points in each zone, you test what can open connections to what, source zone against destination zone against allowed ports, and you produce a grid. Every cell that allows a path which policy says should be blocked is a segmentation finding. This is the concrete test behind PCI's “confirm segmentation is effective” and behind any micro-segmentation claim.

MEMORY HOOK

Outside asks “how do I get in?” Inside asks “now that I am in, where can I go?” A complete assessment answers both, and the reachability matrix is the answer to the second, on one page the client can act on.

17.7

The engagement, on the ground

> **IN PLAIN WORDS**
> “Going to sit at the client” is less dramatic than it sounds. You arrive, you confirm the rules and the scope in person, you are given the access that was agreed (a network port, a VPN, a set of credentials), you test only what is in scope during the agreed window, you check in daily, and you leave the network as you found it. Professionalism is most of the job.

- **Before:** the ROE is signed, the scope and IP ranges are fixed, an emergency contact is named, and your testing source is whitelisted (§11.1, §11.2).
- **Arrival:** confirm scope and timing face to face, collect the access method (port, VPN profile, or credentials), and tell the security team you are starting so your traffic is not mistaken for a real attack (§11.3).
- **During:** work the methodology (Parts IV to VI), stay strictly inside scope, capture evidence as you go, and give a short daily status. If something looks like it broke, stop and call the contact (§12.6).
- **After:** remove any dropbox or test accounts, hold an out-brief in the client's language (§12.5), and deliver findings backed by sealed evidence (§7.5). Then retest the fixes rather than taking “done” on trust (§8.2).

> **FROM THE FIELD**
> Clients remember two things: that you found what mattered, and that you were easy and safe to have on site. Turn up on time, keep them informed, never touch out-of-scope systems, and leave nothing behind. The testing skill gets you the first engagement; the professionalism gets you the next five.

PART XVIII

Use cases and mini-scenarios

Part IX walks five engagements end to end. This part is different: eighteen short cases, the kind you meet weekly, each in three lines, *what you see*, *what you do*, and *what you learn*. Read them until the pattern is reflex. Together they are a rehearsal of the whole handbook against real situations, so that when one appears on a live engagement, you have already seen it here.

18.1

Discovery and recon

1 · The host that was not on the list

You see	A ping sweep returns a live host at `.240` that is not in the client's asset inventory.
You do	Confirm it is in scope, then service-scan it: an old, forgotten test server running an unpatched web app.
You learn	The inventory is never complete. Discovery finds what the client forgot they own, and forgotten hosts are where the worst findings live.

2 · The airtight firewall with a side door

You see	IPv4 scanning is clean; the perimeter looks solid.
You do	Run discovery over IPv6 as well, and reach a management interface that the IPv4 rules blocked.
You learn	Never assume IPv4-only. A rule set that is perfect on one protocol and absent on the other is a real, quiet gap (§13.2).

3 · The banner that told the whole story

You see	A service banner reveals an exact software version.
You do	Look it up against the KEV catalogue; it is on the exploited-now list.
You learn	Version plus KEV is a finding you can prioritise instantly. The boring banner grab is often the highest-value second (§16.8).

18.2

Internal and Active Directory

4 · A credential from thin air

You see	Minutes after plugging in, your listener shows an NTLM hash arriving, unprompted.
You do	Confirm LLMNR/NBT-NS is enabled and SMB signing is not required; capture proves the exposure.
You learn	The first internal credential usually costs no effort at all. If you can hear it, so can anyone on the network (§14.2).

5 · The service account with a weak password

You see	A domain account runs a service and has an SPN.
You do	Request its service ticket and test the crack offline; it falls in minutes.
You learn	Kerberoasting needs only a normal user and a weak service password. The fix is gMSA and length, not a new product (§14.4).

6 · One password, every account

You see	A large user list and a lockout policy.
You do	Spray a single seasonal password across all of them, under the threshold; several accounts open.
You learn	Spraying beats brute force because it respects lockout. MFA is what actually stops it (§14.3).

18.3

Segmentation and Layer 2

7 · The guest Wi-Fi that reached the servers

You see	You are on the guest network, which should be isolated.
You do	Try to open a connection to a server VLAN; it succeeds on a port that policy says is blocked.
You learn	“Isolated” is a claim until the reachability matrix proves it. One missing rule undoes the whole design (§17.6).

8 · The card environment that was not walled off

You see	The client states the PCI CDE is segmented.
You do	Test reachability from a general-IT segment; one forgotten path reaches the CDE.
You learn	That single result puts the whole network back in PCI scope, and is worth the engagement to them (§17.3).

9 · The access port that became a trunk

You see	A wall port in a meeting room.
You do	Check for auto-trunk negotiation; the port forms a trunk and exposes multiple VLANs.
You learn	VLAN hopping is a switch-config problem. Disabling DTP and setting access ports explicitly closes it (§14.6).

18.4

Configuration and services

10 · The switch that answered "public"

You see	SNMP is open on a network device.
You do	Query it with the default community string `public`; it returns the full configuration.
You learn	Default community strings leak, and sometimes let you change config. Seconds to find, easy to fix (§13.7).

11 · The database with no password

You see	Port 27017 answering from a staff subnet.
You do	Connect; the database accepts you with no authentication at all.
You learn	The no-auth trio (Mongo, Redis, Elastic) causes the biggest leaks. Prove reachability and the missing control; never dump the data (§15.2).

12 · The printer running Telnet

You see	Port 23 open on an office printer.
You do	Note it; a capture shows the admin session, password included, in cleartext.
You learn	The presence of a cleartext protocol is itself the finding. The interior was never hardened (§13.8).

18.5

Web, cloud and wireless

13 · The ID you could change

You see	A URL ending `?account=1042`.
You do	Change it to another number in Burp; another customer's record loads.
You learn	Broken access control is the most common serious web finding, and needs only curiosity, not an exploit tool (§14.8).

14 · The storage bucket left open

You see	A cloud storage URL referenced in a page.
You do	Request its index; it lists files to anyone, no credentials.
You learn	Cloud findings are usually misconfiguration, not exploits: an over-broad permission or a public setting (§6.3).

15 · The Wi-Fi with the weak passphrase

You see	A WPA2 corporate SSID on site.
You do	Capture the handshake/PMKID and test the passphrase strength offline; it is a dictionary word.
You learn	Wireless is a physical-presence test, and a weak passphrase turns the car park into a network port (§16.7).

18.6

Judgement and professionalism

16 · The server that went quiet

You see	Mid-scan, a host stops responding and your stomach drops.
You do	Check calmly: your scan was not touching it; the emergency contact confirms a scheduled reboot.
You learn	Because you announced the test and had a contact, a routine reboot stayed routine, not an accusation (§12.6).

17 · The finding that was not there

You see	A scanner flags a critical vulnerability.
You do	Try to confirm it by hand; it does not reproduce. You drop it and record why.
You learn	An unconfirmed scanner hit is not a finding. Verifying by hand is what separates an assessment from a scan (§16.1).

18 · The report they actually read

You see	Ten findings, varied severity, and a nervous executive.
You do	Lead with the one chain that matters, in their language, tied to their real fear.
You learn	The finding is only useful if it is fixed, and it is only fixed if the reader understood it (§8.5, §11.7).

THE PATTERN BEHIND ALL EIGHTEEN

Almost none of these needed an exploit. They needed you to notice a default, a cleartext protocol, a missing signature, an open door, or an unverified claim, and then to prove it cleanly and explain it plainly. That is the whole craft, and it is why theoretical mastery, not tool collecting, is what this handbook builds.

PART XIX

Resources and further reading

Every link here was checked and is current. When you want to go deeper than this handbook, start from these primary sources, not from a search engine. They are grouped so you can find the right one fast, and each is the authoritative home for its topic. Bookmark this page.

19.1

Standards and methodology

SOURCE	WHAT IT IS
NIST SP 800-115	The technical guide to security testing. The methodology baseline for this handbook.
NIST SP 800-53	The control catalogue findings map back to.
NIST SP 800-207	Zero Trust Architecture: the model behind micro-segmentation.
PCI DSS	Cardholder data protection and segmentation testing (Req 11.4.5).
SWIFT CSP	The Customer Security Controls Framework for financial messaging.
CIS Benchmarks	Consensus hardening baselines the host toolkit maps to.
Cisco SAFE	Enterprise security reference architecture (zones and domains).

19.2

Protocol specifications

The primary sources for Part XIII. When behaviour is disputed, the RFC settles it.

PROTOCOL	SPEC	PROTOCOL	SPEC
ARP	RFC 826	Kerberos	RFC 4120
IPv4 / IPv6	791 / 8200	LDAP	RFC 4511
ICMP	RFC 792	SNMP	RFC 3411

PROTOCOL	SPEC	PROTOCOL	SPEC
TCP	RFC 9293	NTP	RFC 5905
UDP	RFC 768	SSH	RFC 4253
DNS	RFC 1035	FTP	RFC 959
DHCP	RFC 2131	SMTP	RFC 5321
HTTP/1.1	RFC 9112	SIP	RFC 3261
HTTP/2	RFC 9113	TLS 1.3	RFC 8446
SMB	MS-SMB2		

19.3

Vulnerability intelligence

SOURCE	USE IT TO
NVD	Look up any CVE and its details.
CVSS v3.1 calculator	Score a finding mechanically (never eyeball it).
CISA KEV	Know what is being exploited now; fix these first.
MITRE ATT&CK	The shared language of attacker techniques.

19.4

Web and application

SOURCE	WHAT IT IS
OWASP Top 10 (2025)	The ten most critical web risks, current edition.
OWASP WSTG	The definitive web testing methodology.
OWASP API Security Top 10	The API-specific risk list.
OWASP Cheat Sheet Series	Concise defensive guidance per topic.

SOURCE	WHAT IT IS
PortSwigger Web Security Academy	Free, hands-on web security labs.

19.5

Tool homepages

Always download from the official source. Full treatment is in Part XVI.

TOOL	HOME	TOOL	HOME
PostureKit	The open-source host assessment toolkit specified in Part VII: collector, sealed evidence, analyzer, rule set and importers (MIT).		
Nmap	nmap.org	BloodHound	SpecterOps
Wireshark	wireshark.org	Hashcat	hashcat.net
tcpdump	tcpdump.org	John the Ripper	openwall.com
NetExec	Pennyw0rth	Burp Suite	portswigger.net
Impacket	fortra	ffuf / gobuster	ffuf / gobuster
Responder	lgandx	Nikto	sullo
kerbrute	ropnop	sqlmap	sqlmap.org
Aircrack-ng	aircrack-ng.org	hcxdumptool	ZerBea
Nessus	tenable.com	Greenbone/OpenVAS	greenbone.net
Nuclei	projectdiscovery	masscan	masscan

19.6

Technique references and learning

SOURCE	WHAT IT IS
HackTricks	The community reference for almost any technique. Start here when you meet something new.

SOURCE	WHAT IT IS
PayloadsAllTheThings	Payloads and methodology per vulnerability class.
GTFOBins / LOLBAS	Abusable built-in Unix / Windows binaries.
LOLDrivers	Known vulnerable and malicious drivers.
SANS white papers	Thousands of practitioner deep-dives, free.

19.7

A self-study path

IN PLAIN WORDS

You do not become expert by reading alone. Read a section here, then do the matching thing in a lab you control, then read the next. A week of that beats a month of passive reading.

1. **Weeks 1-2, foundations.** Re-read Parts I and XIII until addressing, layers and the protocol quick card are automatic. Build the home lab from §11.9.
2. **Weeks 3-4, the attacks.** Work Part XIV against your own lab (never a live network). Watch each attack in Wireshark so you see what it looks like on the wire.
3. **Weeks 5-6, the web.** Work through the PortSwigger Academy labs alongside Part VI; broken access control first.
4. **Weeks 7-8, the method.** Run the full pipeline of Parts IV to VIII against the lab, and write a real report from it using §8.5.
5. **Ongoing.** When any new tool or attack appears, trace it back to the basics here. It is always an address, a port, a login, or a trust.

THE LAST WORD

Everything in twenty parts reduces to one habit: understand what you are looking at, prove it cleanly, and explain it plainly. Master that and you are not someone who runs tools. You are an assessor.

baseline (800-53), client frameworks overlaid. · Raw evidence is change-evident after hashing; reporting is downstream of senior validation. · Client reports name only real analyst-driven tools.